Portfolio Deep-Dive

Technical Reference & Interview Preparation

A comprehensive analysis of every decision, formula,
and architectural choice across 7 data analyst projects

Andrés González Ortega

Actuarial Science – UNAM, Mexico

Projects covered: Airbnb CDMX, Insurance Claims, E-Commerce Cohorts, A/B Testing, Executive KPI Report, Financial Portfolio, Operational Efficiency, Shared Infrastructure

Tech stack: Python, SQL, R, Next.js, FastAPI, Recharts, D3, Docker, Cloud Run, Cloudflare Pages, GitHub Actions

Focus: Mathematical derivations, decision justifications, architecture patterns, interview challenges

March 2026

Built with LaTeX – Source at `projects/latex-portfolio-deepdive/`

Contents

Chapter 1

Airbnb CDMX Dashboard

Source files: `projects/00-demo-aestehtics/data-pipeline/airbnb_etl.py`, `projects/00-demo-aeste`
`projects/00-demo-aestehtics/src/components/AirbnbDashboard.tsx`, `projects/00-demo-aestehtic`
`projects/00-demo-aestehtics/src/app/globals.css`

This chapter covers the lightest project in the portfolio in terms of statistical complexity, but one of the most instructive in terms of frontend architecture and visualization design decisions. The Airbnb CDMX dashboard is a Next.js 14 application that presents Mexico City's short-term rental market to a non-technical audience using five chart types, a dark/light mode system, and a static-data architecture that requires no backend at runtime.

1.1 Business Context

Mexico City (CDMX) is one of Latin America's top tourist and business travel destinations. The short-term rental market served by Airbnb spans 16 of the city's boroughs (*alcaldías*), from the dense historic centre to peripheral residential areas with very different supply-demand dynamics.

The primary audience for this dashboard is a market analyst at a short-term rental investment firm or a policy researcher studying housing supply effects. Key questions the dashboard is designed to answer:

- Which boroughs concentrate supply, and which are underserved relative to demand signals such as pricing and review scores?
- Is CDMX's Airbnb market driven by casual individual hosts or by professional property management companies?
- What is the price distribution by accommodation type, and where do outliers cluster geographically?

These questions do not require predictive modeling. They are answered through descriptive aggregations and interactive charts — the core of a data analyst's toolkit.

1.2 Data Source

Source files: `projects/00-demo-aestehtics/data-pipeline/airbnb_etl.py` (lines 23–33)

The data comes from **Inside Airbnb** (http://insideairbnb.com), an independent project that scrapes public Airbnb listing pages and publishes city snapshots under a Creative Commons licence. The CDMX snapshot used here is from March 2025.

Table 1.1: Dataset overview — Inside Airbnb CDMX, March 2025

Property	Value
Raw file	`listings.csv.gz`
Total rows	27,051 listings
Total columns	79
Columns selected	15 (see ETL)
Price null rate	12.9%
Rating null rate	12.6%
Geographic granularity	16 alcaldías (borough level, not neighbourhood)
Licence	Creative Commons CC0

The null rates in price and rating are structurally expected: new listings have no reviews, and listings with draft prices or unpublished pricing appear with null values in the scrape. The pipeline drops nulls for the specific aggregation that requires the field rather than imputing them — a deliberate choice to avoid distorting distribution shapes.

Key Insight

Inside Airbnb prices reflect the *published* nightly rate, not the final price paid. Cleaning fees, service fees, and taxes can raise the effective checkout price by 20–40%. Any pricing recommendation derived from this data should note this limitation explicitly.

1.3 ETL Pipeline

Source files: `projects/00-demo-aestehtics/data-pipeline/airbnb_etl.py`

The ETL is a single Python script (`airbnb_etl.py`) that reads the compressed CSV, applies transformations, and writes five JSON files to `public/data/airbnb/`. The script uses only `pandas` and `numpy` — no database, no streaming, no orchestration framework. At 27K rows this is appropriate: pandas loads the file in under a second.

1.3.1 Price String Cleaning

Airbnb exports prices as formatted strings: `$1,200.00` or `$14,500.00`. The pipeline strips the currency symbol and thousand separators before casting to float:

Listing 1.1: Price cleaning in `airbnb_etl.py` (lines 18–20)

```
def clean_price(series: pd.Series) -> pd.Series:
    """Strip $, commas and convert to float."""
    return (series
        .str.replace("$", "", regex=False)
        .str.replace(",", "", regex=False)
        .astype(float))
```

A simpler regex like `[\$,]` would work but is harder to read at a glance. The two-step replacement makes the intent explicit to any reviewer.

1.3.2 Host Segmentation Logic

Hosts are classified into three segments based on how many listings they operate. The thresholds are defined in the `segment()` closure inside `build_host_segmentation()`:

Listing 1.2: Host segmentation thresholds (lines 109–113)

```
def segment(n):
    if n == 1:
        return "casual"
    elif n <= 5:
        return "professional"
    return "enterprise"
```

Decision Justification

Why these thresholds? The 1 / 2–5 / 6+ split maps to observable market behaviours: a single-listing host is almost certainly an individual renting their own home or spare room. Hosts with 2–5 listings are likely managing investment properties part-time. Hosts with 6+ listings operate at a scale that requires systematic pricing, availability management, and cleaning coordination — this is a commercial operation regardless of legal form.

The thresholds are not empirically calibrated to CDMX data; they are heuristic conventions used by Airbnb researchers. A rigorous segmentation would cluster on multiple variables (listing count, price variance, response rate). For a portfolio dashboard the heuristic is sufficient and defensible.

1.3.3 Geo Sampling

The raw dataset has 27K coordinate pairs. Rendering all of them in a browser scatter chart would produce an illegible overplotted mass and degrade performance. The pipeline samples 3,000 points at build time:

Listing 1.3: Geo sampling in `airbnb_etl.py` (lines 65–68)

```
def build_geo_heatmap(df: pd.DataFrame, max_points: int = 3000) -> dict:
    valid = df.dropna(subset=["latitude", "longitude", "price"]).copy()
    if len(valid) > max_points:
        valid = valid.sample(max_points, random_state=42)
```

The fixed `random_state=42` ensures the sample is reproducible between pipeline runs. The 3,000-point limit is a UX decision: enough density to reveal the spatial clustering of Cuauhtémoc and Miguel Hidalgo without overwhelming the browser's SVG renderer.

1.3.4 Output Files

The pipeline produces five JSON files totalling under 500 KB:

Table 1.2: ETL output files written to `public/data/airbnb/`

File	Builder function	Consumed by
`kpis.json`	`build_kpis()`	KPI bar
`price_distribution.json`	`build_price_distribution()`	`PriceHistogram`
`geo_heatmap.json`	`build_geo_heatmap()`	`GeoScatter`
`neighborhood_ranking.json`	`build_neighborhood_ranking()`	`NeighborhoodBar`
`host_segmentation.json`	`build_host_segmentation()`	`HostSegmentation`

1.4 Architecture

Source files: `projects/00-demo-aestehtics/src/app/airbnb/page.tsx`, `projects/00-demo-aestehtics/`
`projects/00-demo-aestehtics/package.json`

1.4.1 Static JSON Pattern

The architecture of the Airbnb dashboard is deliberately the simplest possible: pre-computed JSON files are read at server-render time using Node's `fs` module and passed as props to client components. There is no API route, no database connection, and no runtime data fetching.

Listing 1.4: Static JSON loading in `src/app/airbnb/page.tsx` (lines 11–14)

```
function readJson<T>(filename: string): T {
  const filePath = join(process.cwd(), 'public', 'data', 'airbnb',
    filename)
  return JSON.parse(readFileSync(filePath, 'utf-8')) as T
}
```

The page component calls `readJson()` five times — once per dataset — and forwards all values as typed props to `AirbnbDashboard`. Because the page is a React Server Component (no `'use client'` directive), the file I/O happens once on the server during the request and the result is serialized into the initial HTML payload. The client receives fully-hydrated data and never issues a network request for chart data.

Decision Justification

Why static JSON instead of an API? Three reasons:

1. **Data size**: Five JSON files total under 500 KB. This is small enough to embed in a server render without latency penalties.
2. **Update frequency**: The Inside Airbnb dataset is a monthly snapshot. Data is not changing in real time. An API that queries a database to return the same pre-aggregated values on every request adds infrastructure cost with zero user-facing benefit.
3. **Deployment simplicity**: The dashboard is a static export merged into the portfolio's Cloudflare Pages project. No Cloud Run service, no database provisioning, no secret management beyond what already exists for the rest of the portfolio.

The trade-off is that updating the data requires re-running `airbnb_etl.py` and committing the new JSON files. For a monthly-refresh dataset served to a portfolio audience this is an acceptable workflow.

1.4.2 Next.js App Router and Proxy Configuration

The project uses Next.js 14.2 with the App Router. The route `/airbnb` is handled by `src/app/airbnb/page.tsx`. A separate section of the app (`/olist`) connects to a FastAPI backend via a server-side rewrite proxy defined in `next.config.js`:

Listing 1.5: Rewrite proxy for the Olist backend in `next.config.js`

```
async rewrites() {
  const backend = process.env.OLIST_BACKEND_URL ||
    'http://localhost:2050'
  return [
    {
```

```
      source: '/api/olist/:path*',
      destination: `${backend}/:path*`,
    },
  ]
},
```

The Airbnb section of the app does not use this proxy at all — it is purely static. The proxy exists for the Olist (e-commerce) section that shares the same Next.js process, demonstrating that both patterns (static JSON and live API) can coexist in the same project.

1.4.3 Dependency Choices

Table 1.3: Key runtime dependencies and their roles

Package	Version	Role
`recharts`	2.13	All chart rendering
`framer-motion`	11.0	KPI counter animations
`lucide-react`	0.454	Icon system (theme toggle, back arrow)
`swr`	2.4	Data fetching for Olist section (not Airbnb)
`clsx`	2.1	Conditional CSS class composition
`tailwind-merge`	2.5	Conflict-safe Tailwind class merging

Recharts was chosen over D3.js, Victory, and Nivo. D3 gives maximum control but requires imperative DOM manipulation that conflicts with React's declarative model unless carefully managed. Recharts wraps D3's scales in React components, maintaining the familiar `<BarChart>` / `<Scatter>` declarative API while exposing enough customization hooks (custom shapes, custom tooltips, the `Customized` component for SVG overlays) to handle the geo-label overlay on the scatter chart.

1.5 Visualization Design

Source files: `projects/00-demo-aestehtics/src/app/globals.css`, `projects/00-demo-aestehtics/src/`
`projects/00-demo-aestehtics/src/components/ui/ThemeToggle.tsx`

1.5.1 Dark/Light Mode Implementation

The theme system uses a CSS custom properties (variables) approach rather than duplicating color values across components. Light and dark tokens are declared in `globals.css`:

Listing 1.6: CSS token system in `src/app/globals.css` (lines 5–47)

```
:root {
  --background: #FAFAF8;   /* warm white, not pure #FFFFFF */
  --foreground: #1A1A1A;
  --chart-tick:   #6B6B6B;
  --chart-label:  #1A1A1A;
  --chart-grid:   #E5E4DF;
  --bar-rank-1:   #1E3A5F; /* dark navy -- top-ranked bars */
  --bar-rank-2:   #2B527F;
  --bar-rank-3:   #9AB0C8; /* lighter blue -- lower-ranked bars */
}

.dark {
  --background: #141414;
  --foreground: #F0EFEB;
  --chart-tick:   #9A9A9A; /* bumped from #6B6B6B for WCAG AA */
  --chart-label:  #F0EFEB;
  --chart-grid:   #2a2a2a;
  --bar-rank-1:   #6BA3C8; /* inverted: lighter in dark mode */
  --bar-rank-2:   #4A7FA8;
  --bar-rank-3:   #9AB0C8;
}
```

Chart components reference these tokens via `var(-chart-tick)` in Recharts `tick` and `label` props. The dark class is toggled on `document.documentElement` by `ThemeToggle.tsx`, which reads the user's system preference via `window.matchMedia` on first render and persists the explicit choice to `localStorage`.

1.5.2 WCAG Contrast Considerations

The muted text color `#6B6B6B` has a contrast ratio of 3.45:1 against the dark background `#141414` — below WCAG AA's 4.5:1 requirement for body text. The fix is a single CSS override:

Listing 1.7: Muted text contrast fix in `globals.css` (lines 64–67)

```
/* #6B6B6B gives 3.45:1 on #141414 -- fails AA for small text.
   Override to #9A9A9A which gives 6.6:1. */
.dark .text-muted {
  color: #9A9A9A;
}
```

A subtler issue arises with Recharts: SVG `fill` and `stroke` attributes are *not*

affected by CSS `color` inheritance. A hardcoded `fill="#6B6B6B"` in a tick label will fail in dark mode because the SVG attribute takes precedence over any CSS rule targeting the element. The solution is to use `fill='var(-chart-tick)'` in every Recharts `tick` prop, as done consistently across all chart components in this project.

1.5.3 Chart Type Rationale

Table 1.4: Chart type selection rationale for each section

Chart	Type	Rationale
Price distribution	Stacked `BarChart` (histogram)	Bins the continuous price variable to reveal the bimodal structure (entire home vs. private room floors). Stacked bars by room type show composition without requiring a separate facet.
Geographic distribution	`ScatterChart` (lon/lat)	Recharts `ScatterChart` maps longitude to x and latitude to y, producing an approximate map. A true tile-based map (Leaflet/Mapbox) was deferred to avoid token management overhead for a portfolio demo. The `Customized` component overlays borough labels as SVG text.
Neighborhood ranking	Horizontal `BarChart`	Long category names (borough names in Spanish) fit better on a horizontal axis with `layout="vertical"`. Three sort options (listing count, avg price, avg rating) are toggled client-side without re-fetching data.
Host segmentation	Vertical `BarChart` + table	Three segments (casual/professional/enterprise) map cleanly to three bars. The adjacent summary table surfaces host counts and top operators that a bar chart alone cannot convey.

1.5.4 The Geo Scatter Limitation

Using `ScatterChart` to approximate a map is a pragmatic compromise with known failure modes. Recharts maps data coordinates linearly to SVG coordinates, so the

output is an *equirectangular* projection. For CDMX's latitude range (19.15°N to 19.6°N) this introduces roughly 0.4% east-west distortion — visually imperceptible. The main limitation is the absence of base-map tiles: there is no coastline, no street grid, no borough boundary polygon. Borough labels are drawn at hardcoded centroid coordinates defined in `GeoScatter.tsx` (lines 22–38). This is adequate for communicating spatial clustering to a non-specialist audience but would not survive scrutiny from a GIS professional.

1.5.5 Color Encoding on the Geo Scatter

Point colors encode price tier using a three-level threshold function:

Listing 1.8: `src/components/charts/GeoScatter.tsx` (lines 47–51)

```
function priceToColor(price: number): string {
  if (price < 800) return '#6B9DB8'   // steel blue -- budget
  if (price < 2000) return '#1E3A5F'  // dark navy -- mid range
  return '#C4841D'                    // amber -- premium
}
```

The three-color encoding was chosen deliberately over a continuous gradient. A continuous sequential scale would require a colorbar legend and demands that viewers compare gradient shades at small dot sizes — a perceptually difficult task. Three discrete tiers (budget / mid / premium) allow pre-attentive detection of spatial patterns: the viewer can immediately see that amber dots cluster along the western edge without needing to read a colorbar.

The amber accent `#C4841D` is the same highlight color used across the portfolio's chart palette, establishing visual consistency between projects.

1.6 Key Findings

Key Insight

Cuauhtémoc concentration: 12,514 listings (46% of total) sit in a single borough. Roma Norte, Condesa, and Centro Histórico within Cuauhtémoc drive this cluster. The runner-up borough (Miguel Hidalgo) has approximately half the listing count. Tláhuac, at the far southeastern edge of the city, has 40 listings — a 300× gap from the leader.

Key Insight

Enterprise host concentration: The host segmentation reveals a structurally professional market. Enterprise hosts (6+ listings) represent only 7% of all hosts yet control 40% of total supply. The top three enterprise operators are Blueground (221 listings), Mr. W (164), and Clau (156). Casual hosts — those with a single listing — number 8,370 but collectively hold fewer units than the enterprise segment controls per host on average.

Key Insight

Peripheral pricing premium: Despite far fewer listings, Tlalpan and Cuajimalpa show the highest average nightly prices in the dataset (MXN 2,493 and MXN 2,151 respectively). This suggests undersupplied premium inventory: listings in those boroughs can command high prices because competition is thin. A new host entering Tlalpan faces a different competitive environment than one entering Cuauhtémoc.

Key Insight

Room type pricing floors: The price distribution is not unimodal. Entire home/apartment listings peak in the MXN 1,000–1,500 bin. Private rooms form a secondary mass concentrated near MXN 500. This bimodal shape disappears when the two room types are aggregated — a reminder that averaging across heterogeneous product categories produces a mean that is representative of neither group.

Median availability of 16 days per 30 confirms strong demand pressure — listings are occupied roughly half the time. The 87% of listings with review scores above 4.5 suggests either genuine quality or a platform selection effect (underperforming listings are delisted before accumulating reviews).

1.7 Challenge and Interview Questions

Challenge Question

Static JSON vs. live API — when does the pattern break?

The dashboard reads pre-computed JSON files from the filesystem at request time. Identify three scenarios where this architecture would require replacement with a live API, and describe what changes to the codebase and infrastructure would be needed for each.

Consider: update frequency requirements, personalisation needs, dataset size growth, and the cost of adding a database versus the cost of stale data.

Challenge Question

SVG fill inheritance and dark mode

A developer adds a new chart component and writes:

```
<XAxis tick={{ fill: '#6B6B6B' }} />
```

Explain why this breaks in dark mode even though the dark mode CSS class is correctly applied to `document.documentElement`. Describe two ways to fix it, and explain which approach is more maintainable in a multi-chart project.

Challenge Question

Geo scatter vs. choropleth map

The current geographic visualization plots individual listing coordinates as colored dots. An alternative would be a choropleth map where each borough polygon is filled with a color proportional to listing count or average price.

Compare the two approaches on: (a) information density, (b) accessibility for colorblind users, (c) implementation complexity given the current Recharts stack, and (d) suitability for the three audience questions listed in Section 1.1.

Interview Question

Explain the host segmentation design to a product manager.
You have classified hosts into casual (1 listing), professional (2–5 listings), and enterprise (6+ listings). A PM asks: "Why those thresholds? Could the segmentation be done better?"
Give a one-minute answer that acknowledges the heuristic nature of the current approach, proposes a data-driven alternative (such as clustering), and explains the trade-off between analytical rigor and interpretability for a dashboard audience.

Interview Question

Stakeholder question: why are enterprise hosts cheaper than casual hosts?
The host segmentation data shows that enterprise hosts have a lower average nightly price than casual hosts. A stakeholder finds this counterintuitive and asks for an explanation.
Walk through at least two alternative hypotheses (e.g., volume pricing strategy, geographic mix, accommodation type mix) and explain how you would distinguish between them using the available data without additional data collection.

Chapter 2

Insurance Claims Dashboard

Key Insight

This project answers the two questions every P&C insurance CFO needs to know: *Are we reserving enough?* and *Are we pricing correctly?* It applies two actuarial reserving methods – Chain-Ladder and Bornhuetter-Ferguson – to real NAIC Schedule P regulatory data, decomposing losses across six lines of business (LOBs) over accident years 1988–1997. The project is the most actuarially rigorous in the portfolio and is the natural bridge between the UNAM actuarial science curriculum and data-analyst practice.

2.1 Business Context

Source files: `README.md`, `data-pipeline/04_compute_reserves.py`

A property-and-casualty (P&C) insurer collects premiums today and pays claims tomorrow – sometimes years or decades later. The regulatory and financial reporting challenge is to estimate, at any given *valuation date*, how much has been incurred but not yet paid. This quantity is called the **IBNR reserve** (Incurred But Not Reported), and it flows directly onto the insurer's balance sheet as a liability.

The CFO's question. "How much should we be holding in reserve for unpaid claims, and which lines of business are actually profitable?" This question requires both *actuarial reserving* (estimating future claim payments) and *profitability analysis* (comparing those estimates to earned premium).

Regulatory context. In the United States, every P&C insurer must file **NAIC Schedule P** as part of its annual statement. Schedule P reports cumulative paid and incurred losses by accident year and development lag for each line of business – exactly the data structure needed to build loss triangles. The CAS Loss Reserving Database

is a publicly available dataset of historical Schedule P filings, making it ideal for a reproducible analytical project.

Why this matters for an actuary. Reserve adequacy is an existential question for insurers. Under-reserving produces inflated earnings and potential insolvency; over-reserving unnecessarily constrains capital deployment. State insurance departments, rating agencies (AM Best, S&P), and auditors all scrutinize reserve adequacy. An actuarial science graduate entering a data-analyst role at an insurer, reinsurer, or consulting firm will encounter loss triangles and combined ratio analysis in the first week.

2.2 CAS Loss Reserving Database

Source files: `data-pipeline/01_download_cas.py`, `data-pipeline/02_clean_triangles.py`

The data originates from the Casualty Actuarial Society's publicly hosted repository of NAIC Schedule P filings. Six CSV files are downloaded directly from `casact.org`:

Filename	LOB	Actuarial tail
`ppauto_pos.csv`	Private Passenger Auto	Short
`comauto_pos.csv`	Commercial Auto	Medium
`wkcomp_pos.csv`	Workers Compensation	Long
`medmal_pos.csv`	Medical Malpractice	Very long
`othliab_pos.csv`	Other Liability	Long
`prodliab_pos.csv`	Product Liability	Long

Schema. Each row in the raw CSV represents one company-accident year-development year observation. The key columns are:

- `GRCODE` / `GRNAME` – company identifier
- `AccidentYear` – the year the loss event occurred (1988–1997)
- `DevelopmentYear` – the calendar year of the evaluation
- `DevelopmentLag` – `DevelopmentYear - AccidentYear + 1` (1 = first 12 months of development, 10 = tenth year)
- `IncurLoss` – cumulative incurred losses (paid + case reserves)
- `CumPaidLoss` – cumulative paid losses
- `EarnedPremDIR` – direct earned premium

Data quirk: column suffixes. Each CAS file uses a *different* suffix on the numeric columns (e.g., `IncurLoss_B`, `IncurLoss_C`, `IncurLoss_F2`). The cleaning script

`02_clean_triangles.py` detects the suffix by inspecting the `IncurLoss*` column name and strips it uniformly, enabling a single code path for all six files.

Company selection. The full dataset contains roughly 30 companies per LOB. To keep the analysis tractable and representative, the pipeline selects the top 5 companies per LOB by total earned premium (`COMPANIES_PER_LOB = 5`), yielding approximately 3,000 triangle rows in `data/processed/triangles.parquet`.

Valuation date. Although the CAS data extends through 2006 (enabling comparison to known ultimate outcomes), the reserving analysis truncates development at `DevelopmentYear <= 1997`. This simulates the perspective of an actuary evaluating reserves *as of 12/31/1997* – the methodologically honest way to demonstrate that the projections are out-of-sample predictions, not fitted values.

2.3 Synthetic Claims Generation

Source files: `data-pipeline/03_generate_claims.py`

Schedule P is aggregate data: one row per company-accident year-development year. There are no individual claim records. To build the claim-level visualizations shown on the dashboard (severity distributions, report lag histograms, open/closed status charts), the pipeline generates approximately **50,000 synthetic claims** calibrated to match the CAS aggregate patterns.

2.3.1 Distributional Assumptions

Severity – LogNormal model. Claim severity (the loss amount per claim) is modeled as:

$$S \sim \text{LogNormal}(\mu, \sigma) \tag{2.1}$$

The LogNormal distribution is standard practice for claim severity because loss amounts are always positive, right-skewed (a few very large claims dominate total losses), and multiplicative effects (inflation, social inflation) are natural in log space. Each LOB has calibrated parameters:

LOB	μ	σ
Private Passenger Auto	8.5	1.2
Commercial Auto	9.0	1.4
Workers Compensation	9.2	1.3
Medical Malpractice	10.5	1.8
Other Liability	9.5	1.6
Product Liability	10.0	1.7

Medical Malpractice has both the highest mean ($\mu = 10.5$, median severity $\approx e^{10.5} \approx$ \$36,000) and highest dispersion ($\sigma = 1.8$), reflecting its long-tail, high-severity nature.

Frequency – Poisson process. The number of claims per LOB per accident year follows a Poisson distribution (constant rate), an appropriate model for independent, relatively rare events. The total target is 50,000 claims, allocated across LOBs by weight: 30% Auto, 25% Workers Comp, 15% Commercial Auto, 12% Other Liability, 10% Med Mal, and 8% Product Liability.

Report lag – Shifted exponential. The delay between accident date and report date is a critical driver of IBNR. Each claim's report lag is drawn from:

$$L \sim \text{Exponential}\left(\lambda = \frac{1}{\bar{L}}\right) \tag{2.2}$$

where $\bar{L}$ is the mean report lag in days. Auto claims report quickly ($\bar{L} = 30$ days) while Medical Malpractice claims take much longer ($\bar{L} = 90$ days), producing significantly more IBNR for that LOB.

Settlement time – Gamma distribution. The time from report to claim closure follows:

$$T \sim \text{Gamma}(\alpha, \beta) \tag{2.3}$$

with shape α and scale β (in days) calibrated by LOB. For Medical Malpractice, $\alpha = 4.0$ and $\beta = 365$ days, yielding a mean settlement time of $4 \times 365 = 1{,}460$ days (4 years).

Open/closed status. A claim is `Closed` if its projected close date (report date plus settlement days) falls on or before the 12/31/1997 valuation date; otherwise it is `Open` with a case reserve equal to `incurred - paid`.

Decision Justification

Why not use real claim data? Individual claim records from NAIC Schedule P are not publicly available – only the aggregated triangles. Synthetic data generated from actuarially-realistic distributions allows the dashboard to demonstrate claim-level insights (report lag analysis, severity bands, status mix) that would otherwise be impossible. The distributions are calibrated to produce aggregate statistics consistent with the CAS data, so the synthetic dataset is analytically honest even if not legally real.

2.4 Loss Triangle Construction

Source files: `data-pipeline/04_compute_reserves.py`, `sql/04_triangle_pivot.sql`, `backend/insuran`

2.4.1 Formal Definition

A **cumulative loss triangle** is a two-dimensional array $C = \{C_{i,k}\}$ where:

- $i \in \{1, 2, \ldots, n\}$ indexes accident years (here $i = 1988, \ldots, 1997$)
- $k \in \{1, 2, \ldots, K\}$ indexes development lags (here $k = 1, \ldots, 10$ years)
- $C_{i,k}$ is the *cumulative* incurred (or paid) loss as of lag k for accident year i

The triangle structure follows from the valuation date constraint: cell (i, k) is observed only when $i + k - 1 \leq$ valuation year. This creates the characteristic upper-left populated / lower-right missing pattern shown in Figure 2.1.

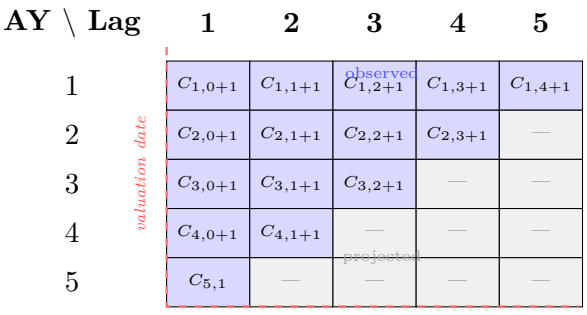


Figure 2.1: Structure of a loss development triangle. Blue cells are observed; gray cells lie beyond the valuation date and must be projected. The diagonal boundary represents the valuation date.

2.4.2 From Long Format to Triangle

The raw CAS data is in *long format* (one row per accident year × development lag). The function `build_triangle()` in `04_compute_reserves.py` pivots this into the standard

matrix form using `pandas.pivot_table`:

Listing 2.1: Triangle construction in `04_compute_reserves.py`

```python
def build_triangle(df, lob, value_col="IncurLoss"):
    lob_data = df[
        (df["line_of_business"] == lob)
        & (df["DevelopmentYear"] <= VALUATION_YEAR)   # 1997 cutoff
    ].copy()
    pivot = lob_data.pivot_table(
        index="AccidentYear",
        columns="DevelopmentLag",
        values=value_col,
        aggfunc="sum",
    )
    return pivot  # NaN in lower-right = future cells
```

The SQL equivalent is in `sql/04_triangle_pivot.sql`, which uses PostgreSQL's conditional aggregation pattern to pivot lags 1–10 into separate columns:

Listing 2.2: Triangle pivot in SQL (`04_triangle_pivot.sql`)

```sql
SELECT
    line_of_business, accident_year,
    MAX(incur_loss) FILTER (WHERE development_lag = 1) AS
    incurred_lag_01,
    MAX(incur_loss) FILTER (WHERE development_lag = 2) AS
    incurred_lag_02,
    -- ... lags 3-10 ...
    ROUND(incurred_lag_02 / NULLIF(incurred_lag_01, 0), 4) AS ata_01_02
FROM triangles
GROUP BY line_of_business, accident_year;
```

2.5 Chain-Ladder Method

Source files: `data-pipeline/04_compute_reserves.py` (lines 59–135), `backend/insurance_backend/route` (lines 67–93)

The **Chain-Ladder method** (also called the *Development Method*) is the actuarial workhorse for loss reserving. It assumes that loss development follows a stable, repeatable pattern captured by historical age-to-age factors.

2.5.1 Age-to-Age Development Factors

For each transition from development lag k to lag $k+1$, the **volume-weighted age-to-age factor** is:

$$\hat{f}_k = \frac{\sum_{i:\, C_{i,k+1} \text{ observed}} C_{i,k+1}}{\sum_{i:\, C_{i,k+1} \text{ observed}} C_{i,k}} \tag{2.4}$$

Volume-weighting (rather than simple averaging) gives larger, more-developed accident years greater influence – appropriate because they represent more statistical credibility.

Code implementation. The function `chain_ladder()` implements Equation (2.4) directly:

Listing 2.3: Age-to-age factor computation (`04_compute_reserves.py`, lines 70–88)

```
for j in range(n_lags - 1):
    col_curr = dev_lags[j]
    col_next = dev_lags[j + 1]
    numerator = 0.0
    denominator = 0.0
    for year in years:
        val_curr = triangle.loc[year, col_curr]
        val_next = triangle.loc[year, col_next]
        if pd.notna(val_curr) and pd.notna(val_next) and val_curr > 0:
            numerator += val_next
            denominator += val_curr
    factors[f"{col_curr}-{col_next}"] = numerator / denominator
```

2.5.2 Cumulative Development Factors (CDFs)

The **cumulative development factor** (CDF) from lag k to ultimate is the product of all age-to-age factors from lag k onward:

$$\widehat{\text{CDF}}_k = \prod_{j=k}^{K-1} \hat{f}_j \tag{2.5}$$

where K is the maximum observed development lag (10 in this project). A tail factor of 1.0 is implicitly assumed (i.e., no further development beyond lag 10), which is conservative for short-tail lines but may underestimate for long-tail lines like Medical Malpractice.

Listing 2.4: CDF computation (`04_compute_reserves.py`, lines 91–97)

```
factor_values = list(factors.values())
cdfs = {}
for j in range(n_lags):
    cdf = 1.0
    for k in range(j, n_lags - 1):
        cdf *= factor_values[k]
    cdfs[dev_lags[j]] = cdf
```

2.5.3 Projecting Ultimate Losses

For accident year i with the latest available observation at lag k, the projected ultimate loss is:

$$\hat{U}_i = C_{i,k} \times \widehat{\mathrm{CDF}}_k \tag{2.6}$$

2.5.4 IBNR Reserve Estimation

The **IBNR reserve** for accident year i is the difference between projected ultimate and current reported losses:

$$\widehat{\mathrm{IBNR}}_i = \max\Big(0,\ \hat{U}_i - C_{i,k}\Big) \tag{2.7}$$

The $\max(0, \cdot)$ floor prevents negative IBNR from arising due to favorable development (reserve releases).

Numerical example. For Private Passenger Auto, paid development factors (from the `main()` output) illustrate the method:

Lag transition	**Factor** $\hat{f}_k$	**CDF** $\widehat{\mathrm{CDF}}_k$
$1 \to 2$	1.80	$1.80 \times \cdots$
$2 \to 3$	1.25	$\cdots$
$\vdots$	$\vdots$	$\vdots$
$k \to$ ultimate	1.00	1.00

The 1997 accident year has only lag-1 data available; its CDF is the product of all nine factors, making the IBNR estimate highly leveraged on historical patterns – exactly why BF was developed as a complement.

Key Insight

The portfolio's total IBNR on a paid Chain-Ladder basis is approximately $20.4M, with Private Passenger Auto contributing 76% due to its volume. Medical Malpractice carries paid CDF values of 6.1x at lag 1–2, meaning that only $1/6.1 \approx 16\%$ of ultimate losses have been paid by end of year 2 – illustrating why long-tail lines require large IBNR reserves.

2.6 Bornhuetter-Ferguson Method

Source files: `data-pipeline/04_compute_reserves.py` (lines 138–180), `backend/insurance_backend/ro` (lines 122–145)

2.6.1 Motivation

The Chain-Ladder method relies entirely on observed data. For recent accident years (e.g., 1997, which has only one year of development), the CL estimate is highly volatile – a single unusual claim year extrapolates to a large IBNR. The **Bornhuetter-Ferguson method** (BF) blends CL-derived development patterns with an *a priori* expected loss ratio, moderating the influence of sparse data.

2.6.2 Derivation

Partition ultimate losses into two components:

1. Losses already reported (observed): $C_{i,k}$
2. Losses yet to emerge (unreported): estimated from an a priori expectation

   The fraction of losses *yet to be reported* as of lag k is:

$$q_k = 1 - \frac{1}{\widehat{\text{CDF}}_k} \tag{2.8}$$

   The **a priori expected loss** for accident year i is:

$$\text{EL}_i = \text{Premium}_i \times \text{ELR} \tag{2.9}$$

where ELR (*expected loss ratio*) is the actuary's a priori assumption about profitability – typically derived from pricing actuaries, industry benchmarks, or long-run historical averages.

The **BF IBNR** is:

$$\begin{aligned}\widehat{\text{IBNR}}_i^{\text{BF}} &= \text{EL}_i \times q_k \\ &= \text{Premium}_i \times \text{ELR} \times \left(1 - \frac{1}{\widehat{\text{CDF}}_k}\right)\end{aligned} \tag{2.10}$$

The **BF ultimate** is the sum of observed losses and BF IBNR:

$$\hat{U}_i^{\text{BF}} = C_{i,k} + \text{Premium}_i \times \text{ELR} \times \left(1 - \frac{1}{\widehat{\text{CDF}}_k}\right) \tag{2.11}$$

2.6.3 Code Implementation

The project uses `apriori_lr = 0.65` (65% expected loss ratio), a reasonable starting assumption for a mixed P&C portfolio:

Listing 2.5: Bornhuetter-Ferguson (`04_compute_reserves.py`, lines 159–177)

```
def bornhuetter_ferguson(triangle, earned_premium, cl_cdfs,
                         apriori_lr=0.65):
    for year in sorted(triangle.index):
        latest_lag = available.index.max()
        reported   = float(available[latest_lag])
        cdf        = cl_cdfs.get(latest_lag, 1.0)
        prem       = earned_premium.get(year, 0)
        expected_loss = prem * apriori_lr

        pct_unreported = max(0, 1 - 1/cdf) if cdf > 1 else 0
        ibnr_bf        = expected_loss * pct_unreported
        ultimate_bf    = reported + ibnr_bf
```

2.6.4 CL vs. BF Comparison

Criterion	Chain-Ladder	Bornhuetter-Ferguson
Data requirement	Development pattern only	Pattern + a priori loss ratio
Mature years (early lags)	Accurate; data-rich	Converges to CL as $q_k \to 0$
Immature years (late AYs)	Volatile; highly leveraged	Moderate; a priori anchors estimate
Regulatory preference	Standard; widely accepted	Often preferred for most-recent years
Sensitivity to one bad year	High (contaminates all factors)	Low (a priori is independent)
When to use	Long-run stable portfolios with credible history	New books of business; recent AYs; volatile LOBs

Key Insight

CL produces larger IBNR estimates for the most recent accident years (1996–1997) where little development data is available; BF moderates these projections toward the a priori expectation. This difference is visible in the `/api/v1/cl-vs-bf` endpoint, which returns `pct_diff` (the percentage by which BF IBNR differs from CL IBNR) for each accident year and LOB.

2.7 Profitability Analysis

Source files: `sql/02_loss_ratio_by_lob.sql`, `sql/03_frequency_severity.sql`, `sql/05_combined_rat`
`data-pipeline/04_compute_reserves.py` (lines 288–294)

2.7.1 Loss Ratio

The **loss ratio** is the primary underwriting profitability metric:

$$\text{LR}_i = \frac{\text{Incurred Losses}_i}{\text{Earned Premium}_i} \tag{2.12}$$

An LR below 100% means the line collected more premium than it paid in losses (before expenses). The SQL in `02_loss_ratio_by_lob.sql` computes both a reported LR and an ultimate LR (using CL projections):

Listing 2.6: Loss ratio computation (`02_loss_ratio_by_lob.sql`)

```
annual_loss_ratio AS (
```

```
    SELECT
        line_of_business, accident_year,
        ROUND(100.0 * incur_loss
              / NULLIF(earned_prem_net, 0), 2) AS loss_ratio_pct,
        -- Rolling 3-year average to smooth volatility
        ROUND(AVG(loss_ratio_pct) OVER (
            PARTITION BY line_of_business
            ORDER BY accident_year
            ROWS BETWEEN 2 PRECEDING AND CURRENT ROW
        ), 2)                                    AS rolling_3yr_lr_pct
    FROM latest_eval
)
```

Profitability flags are assigned: `PROFITABLE` ($< 100\%$), `MARGINAL` (90–100%), `UNPROFITABLE` ($> 100\%$).

2.7.2 Combined Ratio

The **combined ratio** is the gold-standard underwriting metric, incorporating both claim costs and operational expenses:

$$\text{CR} = \text{LR} + \text{ER} = \frac{\text{Incurred Losses} + \text{Expenses}}{\text{Earned Premium}} \tag{2.13}$$

where ER is the expense ratio. A CR below 100% means underwriting profit; above 100% means the insurer relies on investment income to be profitable.

Combined Ratio	Interpretation
$< 95\%$	Strong underwriting profit
$95 - 100\%$	Marginal profit
$100 - 110\%$	Marginal loss; investment income may compensate
$> 110\%$	Significant loss

The project assumes a flat 30% expense ratio (U.S. industry average for P&C):

$$\text{CR} = \text{LR} + 0.30 \tag{2.14}$$

This assumption is made explicit in the SQL via a `params` CTE to facilitate sensitivity analysis.

2.7.3 Frequency-Severity Decomposition

The fundamental identity of insurance pricing:

$$\text{Pure Premium} = \text{Frequency} \times \text{Severity} \tag{2.15}$$

where:

$$\text{Frequency} = \frac{\text{Claim Count}}{\text{Exposure}} \tag{2.16}$$

$$\text{Severity} = \frac{\text{Total Losses}}{\text{Claim Count}} \tag{2.17}$$

This decomposition is critical for claims management decisions: rising frequency signals underwriting/risk selection problems; rising severity signals claims management or trend issues.

The SQL in `03_frequency_severity.sql` computes year-over-year growth rates for both components and flags the *primary loss driver*:

Listing 2.7: Loss driver identification (`03_frequency_severity.sql`)

```sql
CASE
    WHEN ABS(frequency_yoy_pct) > ABS(severity_yoy_pct)
        THEN 'FREQUENCY DRIVEN'
    WHEN ABS(severity_yoy_pct) > ABS(frequency_yoy_pct)
        THEN 'SEVERITY DRIVEN'
    ELSE 'BALANCED'
END AS primary_loss_driver
```

Since the project uses synthetic claims, exposure is proxied by earned premium, yielding a *pure premium rate* (loss per dollar of premium) rather than traditional frequency per policy.

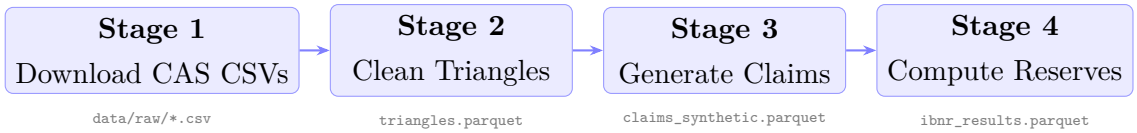
Key Insight

From the README findings: **Private Passenger Auto** and **Product Liability** are the only profitable lines (combined ratio $< 100\%$). **Medical Malpractice** shows the highest loss ratio ($\approx 280\%$) with the longest development tails – claims take 3–5$\times$ longer to report than Auto, creating the largest IBNR uncertainty of all six LOBs.

2.8 System Architecture

Source files: `backend/insurance_backend/main.py`, `backend/insurance_backend/data_loader.py`, `src/components/insurance/InsuranceDashboard.tsx`

2.8.1 4-Stage ETL Pipeline



Stage 1 – `01_download_cas.py` Downloads six CAS CSVs from `casact.org`. Idempotent: skips files that already exist.

Stage 2 – `02_clean_triangles.py` Detects and strips per-file column suffixes, tags each row with a human-readable LOB name, selects top 5 companies per LOB by premium volume, and computes derived columns (`loss_ratio`, `paid_to_incurred`). Output: `data/processed/triangles.parquet`.

Stage 3 – `03_generate_claims.py` Generates 50,000 synthetic claims using LOB-calibrated lognormal severity, exponential report lag, and gamma settlement distributions. Output: `data/processed/claims_synthetic.parquet`.

Stage 4 – `04_compute_reserves.py` Builds valuation-date triangles for both incurred and paid bases, runs chain-ladder on each, runs BF on paid losses with `apriori_lr = 0.65`, and aggregates LOB-level summaries. Outputs: `data/processed/ibnr_results.parquet` and `data/processed/lob_summary.parquet`.

2.8.2 FastAPI Backend

The backend loads all four parquet files at startup via `backend/insurance_backend/data_loader.py` and exposes five API routers under the `/api/v1` prefix:

Router module	Endpoint
`loss_triangle.py`	`GET /api/v1/loss-triangle`
	`GET /api/v1/cl-vs-bf`
`frequency_severity.py`	`GET /api/v1/frequency-severity`
`loss_ratios.py`	`GET /api/v1/loss-ratios`
`combined_ratio.py`	`GET /api/v1/combined-ratio`
`claim_distribution.py`	`GET /api/v1/claim-distribution`

All endpoints accept optional query parameters `lob`, `company`, `year_start`, and `year_end` for dashboard filtering. The `/api/v1/loss-triangle` endpoint additionally accepts `type` (`incurred`/`paid`) and `method` (`cl`/`bf`), computing CL factors on-the-fly from the filtered triangle and overriding the IBNR estimates from the pre-computed parquet when BF is requested.

2.8.3 Next.js Frontend

The dashboard runs at `localhost:3051` and proxies all API calls through Next.js rewrites to avoid CORS issues. Browser traffic never reaches the FastAPI port directly.

Key React components in `src/components/insurance/`:

- `LossTriangleHeatmap.tsx` – interactive heatmap; toggle incurred/paid and CL/BF
- `IBNRWaterfall.tsx` – waterfall: Paid + Case Reserve + IBNR = Ultimate
- `FrequencySeverityChart.tsx` – dual-axis: claim count (bars) and avg severity (line)
- `LossRatioByLOB.tsx` – reported vs. ultimate LR bar chart
- `CombinedRatioTrend.tsx` – stacked area: loss ratio + expense ratio
- `CLvsBFTable.tsx` – tabular comparison of CL and BF IBNR by year
- `InsuranceFilterBar.tsx` – LOB / company / year-range selectors

Decision Justification

Why Next.js instead of Power BI? Power BI was the original plan, but it requires a Windows machine with PBI Desktop and a licensed workspace for embedding. Next.js + Recharts gives full control over dark/light theming (the dashboard supports both modes with LOB-specific CSS variables like `-lob-auto`, `-lob-workers`), a static export that merges into the shared portfolio hub, and the ability to call FastAPI endpoints through an edge proxy rather than exposing them. The actuarial calculations remain in Python, which is far more transparent than DAX measures.

2.9 SQL Queries

Source files: `sql/01_claims_summary.sql`, `sql/02_loss_ratio_by_lob.sql`, `sql/03_frequency_severi`
`sql/04_triangle_pivot.sql`, `sql/05_combined_ratio.sql`

Five analytical SQL queries cover the full story from raw claims to underwriting decision support. All are written in PostgreSQL dialect with CTEs for readability.

Query 01 – Claims portfolio snapshot. *Business question: What does our claims landscape look like by LOB and status?*

Produces count, open/closed split, average severity (paid and incurred), total case reserves, and severity-band distribution per LOB. Uses three CTEs: `claims_by_lob_status` → `lob_pivot` (conditional sums for open/closed) → `portfolio_totals` (UNION ALL grand total row). Key pattern: `NULLIF(denominator, 0)` guards every division.

Query 02 – Loss ratio by LOB. *Business question: Which lines of business are profitable?*

Joins triangles (most mature lag per accident year via `DISTINCT ON`) to `ibnr_results` for ultimate loss ratios. Adds a rolling 3-year window average:

Listing 2.8: Rolling window loss ratio (`02_loss_ratio_by_lob.sql`)

```
AVG(loss_ratio_pct) OVER (
    PARTITION BY line_of_business
    ORDER BY accident_year
    ROWS BETWEEN 2 PRECEDING AND CURRENT ROW
) AS rolling_3yr_lr_pct
```

Assigns `CHRONIC ISSUE` when the rolling 3-year average exceeds 100%, directing management attention to persistently unprofitable lines.

Query 03 – Frequency-severity decomposition. *Business question: Are losses driven by more claims or larger claims?*

Joins `claims_synthetic` to `triangles` for an exposure proxy, computes frequency per $1M of earned premium and incurred severity, then uses `LAG()` window functions to produce year-over-year growth rates. The `primary_loss_driver` flag guides management action: frequency-driven losses suggest underwriting intervention; severity-driven losses suggest claims management improvements.

Query 04 – Triangle pivot. *Business question: What does the loss development pattern look like?*

Pivots the long-format triangles table into the standard actuarial wide layout using `FILTER (WHERE development_lag = k)` conditional aggregation. Also computes individual age-to-age factors per accident year and volume-weighted average factors in a separate CTE – the equivalent of what the Python chain-ladder code does, but expressible in pure SQL.

Query 05 – Combined ratio analysis. *Business question: Is the portfolio profitable after expenses?*

Uses a `params` CTE (`assumed_expense_ratio = 0.30`) to make the flat expense assumption explicit and easy to change. Computes reported CR, ultimate CR (including IBNR from chain-ladder), a 3-year rolling CR, and year-over-year CR change. Profitability bands (`STRONG PROFIT` through `SIGNIFICANT LOSS`) and trend flags (`IMPROVING`, `DETERIORATING`, `STABLE`) provide dashboard-ready text labels.

2.10 Challenge and Interview Questions

Challenge Question

Triangle completeness and the valuation date. The code sets `VALUATION_YEAR = 1997`. If you moved the valuation date to 1993, which cells in the triangle would be removed? How would the chain-ladder factors change, and in which direction would IBNR estimates move for the 1993 accident year? Explain using Equation (2.4).

Challenge Question

Tail factor sensitivity. The project assumes a tail factor of 1.000 beyond lag 10 for all LOBs. For Medical Malpractice (a very long-tail line), this is arguably wrong. If you applied a tail factor of 1.15 (15% additional development beyond lag 10), how would the IBNR change for the oldest accident year (1988)? Show the algebraic adjustment to Equations (2.5) and (2.6).

Challenge Question

BF convergence. Prove algebraically that as $k \to K$ (i.e., as an accident year approaches full development), the BF ultimate in Equation (2.11) converges to the observed value $C_{i,K}$. What does this imply about when BF and CL agree?

Challenge Question

Volume weighting vs. simple average. The code computes volume-weighted factors (Equation (2.4)). Construct a numerical example with three accident years where the simple-average factor and the volume-weighted factor differ by more than 10%. Which is more appropriate and why? When might an actuary deliberately choose a simple average or an "all-but-one-year" factor selection?

Interview Question

Explain IBNR to a non-actuarial CFO. The CFO asks: "Our loss triangle shows \$15M of paid losses. Your report says our ultimate is \$20.4M. Where did the extra \$5.4M come from, and do we actually owe it?" Give a 2-minute plain-language answer.

Interview Question

When would you choose BF over Chain-Ladder? The pricing team just launched a new commercial auto product six months ago. You have half a year of loss data. Which method would you use to estimate IBNR and why? What a priori loss ratio would you select, and where does it come from?

Interview Question

Combined ratio interpretation. Medical Malpractice shows a combined ratio of approximately 310% in this project. Does that mean the insurer is bankrupt? What information would you need to determine whether the company is financially viable despite this underwriting loss?

Interview Question

Frequency-severity trade-offs in pricing. Your analysis flags a line as "SEVERITY DRIVEN" because average incurred severity increased 12% year-over-year while frequency was flat. What are three possible root causes of rising severity? For each, describe the management response that would be appropriate.

Chapter 3

E-Commerce Cohort Analysis

Source files: `projects/02-ecommerce-cohort-analysis/` — notebooks (4), SQL scripts (5), Streamlit app (6 pages)

This chapter documents the analytical work behind the Olist Brazilian E-Commerce cohort analysis. The central business question is deceptively simple: with only 3% of customers making a repeat purchase, what distinguishes the returning minority? The project delivers statistical rigour — survival curves, odds ratios, Gini coefficients — through a multi-layer stack of SQL, Python notebooks, and a Streamlit executive dashboard deployed to Cloud Run.

3.1 Business Context

3.1.1 The 3% Repeat Purchase Problem

Olist is a Brazilian marketplace intermediary: sellers list products on Olist's platform and Olist handles the customer-facing experience. The dataset covers 99,441 orders placed between September 2016 and October 2018. Of 93,358 unique customers, only 2,801 — roughly 3.0% — ever placed a second delivered order.

This figure is the analytical anchor of the entire project. It frames every downstream analysis as a search for signal within extreme class imbalance. Repeat customers are worth **1.92x** a one-time customer in *total* revenue (R$309 against R$161), but the reason is order count, not basket size: they place roughly 2.1 orders. *Per order* they spend **0.91x** as much (R$146 against R$161), and their first order is about 10% smaller than a one-and-done customer's. The high spenders are disproportionately the ones who never come back, which inverts the naive reading — retention is valuable because of frequency, not because returning customers are richer.

Key Insight

A 3% base rate means that standard accuracy metrics are meaningless for classification tasks. Any model predicting "never repeat" would achieve 97% accuracy while being analytically useless. The project therefore avoids binary classification in favour of survival analysis (which respects censoring) and logistic regression odds ratios (which quantify relative risk between groups).

3.1.2 The `customer_id` vs. `customer_unique_id` Distinction

Olist's schema contains two customer identifiers that are easily confused:

- `customer_id`: generated per order. A single person placing three orders will have three distinct `customer_id` values.
- `customer_unique_id`: the true person-level identifier, stable across all orders.

Using `customer_id` directly would cause three critical errors: (1) it would inflate the apparent customer count, (2) every order would appear as a first purchase, and (3) the repeat rate would compute to zero by construction. All SQL scripts and Python code in this project join through `customer_id` only to resolve the link to `customer_unique_id`, then group exclusively on the latter.

Listing 3.1: Wrong vs. correct customer grouping

```sql
-- WRONG: one row per order, not per person
SELECT customer_id, COUNT(*) AS orders
FROM olist_orders
GROUP BY customer_id;

-- CORRECT: one row per real customer
SELECT c.customer_unique_id, COUNT(*) AS orders
FROM olist_orders  o
JOIN olist_customers c ON o.customer_id = c.customer_id
GROUP BY c.customer_unique_id;
```

3.1.3 Olist Marketplace Model

Because Olist is an intermediary, the customer experience depends on three parties: Olist (platform, review system, logistics coordination), the individual seller (product quality, dispatch time), and the carrier. This creates a confounding structure for causal claims: a poor delivery experience may reflect the seller, the carrier, or geographic distance — not Olist platform quality per se. The analysis treats delivery days and review scores as observed outcomes without attributing causality to any single party.

3.2 Olist Dataset

Source files: `streamlit/pages/5_metodologia.py` (tab "Fuente de Datos"), `notebooks/01_data_ingesti`

3.2.1 Nine CSV Files, One Analytical Table

The raw dataset consists of nine CSV files joined into a single master table (`data/processed/orders_enr` during notebook 01. The join graph is star-shaped: `olist_orders` is the central fact table, with dimension tables for customers, items, payments, reviews, products, sellers, geolocation, and a category translation lookup.

Table 3.1: Olist dataset files and approximate row counts

CSV file	Content	Rows (
`olist_orders_dataset.csv`	Orders: status, timestamps, customer_id	
`olist_order_items_dataset.csv`	Line items: price, freight, seller	
`olist_customers_dataset.csv`	customer_id → unique_id, state	
`olist_products_dataset.csv`	Category, dimensions, weight	
`olist_sellers_dataset.csv`	Seller state and city	
`olist_order_payments_dataset.csv`	Payment type, value, instalments	
`olist_order_reviews_dataset.csv`	Review score 1–5, free text	
`olist_geolocation_dataset.csv`	ZIP → lat/lon	
`product_category_name_translation.csv`	Portuguese → English category	

The enriched parquet adds 30 derived columns including `delivery_days`, `is_late_delivery`, `cohort_month`, `months_since_cohort`, and aggregated payment totals. The analysis filters to 96,478 delivered orders only; cancelled, unavailable, and invoiced orders are excluded because their customer journey is incomplete.

3.2.2 Right-Censoring Problem

The observation window closes at October 2018. Any customer who made their first purchase in, say, August 2018 has had at most two months to return. We cannot observe whether they return in November 2018 or later. This is right-censoring in the survival analysis sense: the event (second purchase) may have happened after the observation period ended.

Decision Justification

Three approaches were considered for handling censoring:

1. **Ignore it**: count non-repeaters as definitive churners. This underestimates the true repeat rate for recent cohorts.
2. **Truncate**: only analyse cohorts old enough to have a full 12-month window. This wastes 40%+ of the data and introduces selection bias toward early cohorts.
3. **Kaplan-Meier** (chosen): treat non-repeaters as right-censored observations. This correctly accounts for the fact that recent customers have had less time to return, without discarding them.

The Kaplan-Meier approach is implemented via the `lifelines` library. The `event_observed` column is 1 for confirmed repeaters and 0 for right-censored customers.

3.3 Cohort Retention

Source files: `sql/01_cohort_assignment.sql`, `sql/02_retention_matrix.sql`, `notebooks/03_cohort_ret` `streamlit/pages/2_retencion_cohortes.py`

3.3.1 Cohort Assignment Logic

A cohort is defined by the month of a customer's first *delivered* order. The SQL implementation uses two CTEs: `customer_first_purchase` computes the minimum `order_purchase_timestamp` per `customer_unique_id`, and `cohort_assignment` computes `months_since_cohort` for every subsequent order.

Listing 3.2: Cohort assignment and tenure calculation (from `sql/01_cohort_assignment.sql`)

```
customer_first_purchase AS (
    SELECT customer_unique_id,
           MIN(order_purchase_timestamp) AS first_purchase_date
    FROM delivered_orders
    GROUP BY customer_unique_id
),
cohort_assignment AS (
    SELECT d.customer_unique_id,
           DATE_TRUNC('month', fp.first_purchase_date) AS cohort_month,
           (EXTRACT(YEAR FROM AGE(
               DATE_TRUNC('month', d.order_purchase_timestamp),
               DATE_TRUNC('month', fp.first_purchase_date)
           )) * 12
```

```
            + EXTRACT(MONTH FROM AGE(
                DATE_TRUNC('month', d.order_purchase_timestamp),
                DATE_TRUNC('month', fp.first_purchase_date)
            )))::INT AS months_since_cohort
    FROM delivered_orders AS d
    JOIN customer_first_purchase AS fp
      ON d.customer_unique_id = fp.customer_unique_id
)
```

Note that `DATE_TRUNC` is applied before `AGE` to ensure tenure is measured in whole months, not fractional months. The final `SELECT` also adds `ROW_NUMBER() OVER (PARTITION BY customer_unique_id ORDER BY order_purchase_timestamp)` to assign a sequential order number, which is used downstream to isolate first orders in the activation analysis.

3.3.2 Retention Matrix Construction

The retention matrix pivots cohort-month pairs into a wide format using `CASE-WHEN` expressions. Each cell reports the percentage of the cohort's customers who placed a delivered order in that period:

$$\text{Retention}_{c,k} = \frac{|\{u \in C_c : \text{purchase in month } k\}|}{|C_c|} \tag{3.1}$$

where C_c is the set of customers in cohort c and k is the number of months since cohort formation. By construction, $\text{Retention}_{c,0} = 100\%$ for all cohorts.

Listing 3.3: Retention matrix pivot (excerpt from `sql/02_retention_matrix.sql`)

```
SELECT
    s.cohort_month,
    s.cohort_size,
    ROUND(MAX(CASE WHEN m.months_since_cohort = 0
        THEN m.active_customers * 100.0 / s.cohort_size END), 2) AS
    month_0,
    ROUND(MAX(CASE WHEN m.months_since_cohort = 1
        THEN m.active_customers * 100.0 / s.cohort_size END), 2) AS
    month_1,
    ROUND(MAX(CASE WHEN m.months_since_cohort = 2
        THEN m.active_customers * 100.0 / s.cohort_size END), 2) AS
    month_2
    -- ... through month_11
FROM cohort_sizes AS s
LEFT JOIN monthly_activity AS m ON s.cohort_month = m.cohort_month
```

```
GROUP BY s.cohort_month, s.cohort_size
ORDER BY s.cohort_month;
```

3.3.3 Count-Based vs. Revenue-Based Retention

The Streamlit dashboard (page 2) offers a toggle between two retention modes:

- **Count-based**: the number of distinct customers active in month k divided by cohort size. Stored in `data/processed/cohort_retention_matrix.parquet`.
- **Revenue-based**: the total payment value from returning customers in month k divided by month-0 revenue. Stored in `data/processed/cohort_revenue_retention.parquet`.

Revenue retention amplifies the signal of high-value returners. A cohort where one high-spend customer returns could show high revenue retention but low count retention. The divergence between the two modes is itself informative: large gaps suggest that repeat purchases are concentrated among a few high-value customers rather than broadly distributed.

The Streamlit code normalises both matrices identically:

Listing 3.4: Normalisation to retention percentages (`streamlit/pages/2_retencion_cohortes.py`, line 90)

```
data_pct = data.div(data.iloc[:, 0], axis=0) * 100
```

3.4 Kaplan-Meier Survival Analysis

Source files: `notebooks/03_cohort_retention.ipynb`, `streamlit/pages/2_retencion_cohortes.py` (tab "Kaplan-Meier"), `streamlit/pages/5_metodologia.py` (tab "Supervivencia Kaplan-Meier")

3.4.1 Estimator Definition

The Kaplan-Meier (KM) estimator computes the probability that the event of interest — here, a second purchase — has *not* yet occurred by time t. In this project, $S(t)$ is the probability that a customer has not returned within t days of their first purchase:

$$\hat{S}(t) = \prod_{t_i \leq t} \left(1 - \frac{d_i}{n_i}\right) \tag{3.2}$$

where:

- t_i are the ordered distinct event times (days when second purchases occurred),
- d_i is the number of second purchases at time t_i ("deaths"),

- n_i is the number of customers still at risk at time t_i (neither purchased again nor censored before t_i).

3.4.2 Preparing the Survival Dataset

The `survival_data.parquet` file contains one row per customer with two key columns:

Listing 3.5: Survival data construction (from `streamlit/pages/5_metodologia.py`)

```
survival["duration_days"] = np.where(
    survival["is_repeat_customer"],
    (survival["second_purchase_date"] -
    survival["first_purchase_date"]).dt.days,
    (DATASET_END_DATE - survival["first_purchase_date"]).dt.days,
)
survival["event_observed"] = survival["is_repeat_customer"].astype(int)
```

For the 97% of customers who never returned, `event_observed = 0` and `duration_days` measures how long we watched them without seeing a repeat purchase — the censoring time, not the time-to-event.

3.4.3 Fitting the Model in Streamlit

The dashboard fits a `KaplanMeierFitter` from the `lifelines` library (version `>=0.29`) interactively:

Listing 3.6: KM fitting, originally in `streamlit/pages/2_retencion_cohortes.py`; now precomputed by `data-pipeline/05_build_web_json.py`

```
from lifelines import KaplanMeierFitter

kmf = KaplanMeierFitter()
kmf.fit(
    durations=survival_clean["duration_days"],
    event_observed=survival_clean["event_observed"],
)
km_df = kmf.survival_function_.reset_index()
ci = kmf.confidence_interval_survival_function_.reset_index()
median_surv = kmf.median_survival_time_   # returns inf: S(t) never
    reaches 0.5
```

Key Insight

That last line is a trap worth naming. `median_survival_time_` returns `inf` on this dataset, because $\hat{S}(t)$ plateaus near 0.95 and never crosses 0.5. Printing it as "the median" produces a headline figure that does not exist. The quotable statistic is conditional on returning, and has to be computed separately over the uncensored rows.

The dashboard also supports segmented KM curves: the user selects `payment_type` or `customer_state` as a stratification variable, and the page fits a separate KM curve per group. This directly supports the log-rank test finding that voucher payers have statistically different survival curves from credit card and boleto payers.

3.4.4 Interpreting the Curve in Context

For Olist, $\hat{S}(t)$ starts at 1.0 (all customers are "surviving" — i.e., have not yet reordered) and decays slowly, plateauing near 0.95. The decay *decelerates but never stops*: the curve loses 1.23 percentage points in the first quarter and roughly 0.6 points in every quarter thereafter, out to two years.

Key Insight

This corrects an earlier reading of this same dataset, which described reorder probability as "practically nil" after six months. The curve does not support that. Repurchase keeps trickling in for the full two-year observation window; it merely slows. The distinction matters for the recommendation, because a six-month cutoff would write off customers who are still converting.

Because $\hat{S}(t)$ never crosses 0.5, **the median survival time does not exist** — it is undefined, not merely uninterpretable. What can be quoted is a conditional statistic: among the 3% who *did* return, the median wait was **82 days**. That is a different quantity and must be labelled as such.

Key Insight

The KM curve shows where the density is, not where the opportunity ends. Among customers who returned, the median wait was 82 days and 27% came back within 30 — so the first quarter is the right place to concentrate an intervention. But since the curve keeps declining for two years, a later touch is not wasted effort, and framing day 90 as a cutoff would be a misreading of the same data that motivates the campaign.

3.5 RFM Segmentation

Source files: `sql/03_rfm_segmentation.sql`, `notebooks/04_rfm_ltv_activation.ipynb`, `streamlit/pages/3_segmentos_clientes.py` (tabs "Distribucion" through "Lorenz")

3.5.1 RFM Dimensions and Quintile Scoring

RFM scores every customer on three behavioural dimensions relative to the analysis reference date (one day after the last observed purchase):

Table 3.2: RFM dimension definitions and scoring direction

Dimension	Metric	Direction	Score 5 means
Recency (R)	Days since last purchase	Lower is better	Most recent 20%
Frequency (F)	Count of distinct orders	Higher is better	Top 20% by orders
Monetary (M)	Total lifetime spend (R$)	Higher is better	Top 20% by revenue

Quintile scoring uses `NTILE(5)` in PostgreSQL. Recency requires reversed ordering because fewer days since last purchase is better:

Listing 3.7: Quintile scoring in `sql/03_rfm_segmentation.sql`

```
NTILE(5) OVER (ORDER BY recency_days DESC)  AS r_score,  -- 5=most recent
NTILE(5) OVER (ORDER BY frequency ASC)      AS f_score,
NTILE(5) OVER (ORDER BY monetary ASC)       AS m_score
```

3.5.2 Seven Segment Definitions

Score combinations are mapped to seven business segments through a `CASE` statement. The segment rules below are implemented identically in both SQL (`sql/03_rfm_segmentation.sql`) and Python (`streamlit/pages/5_metodologia.py`):

Table 3.3: RFM segment assignment rules

Segment (English)	R score	F score	M score
Champions	≥ 4	≥ 4	≥ 4
Loyal Customers	any	≥ 4	≥ 3
Potential Loyalists	≥ 4	2–3	any
New Customers	≥ 4	1	any
At Risk	2–3	≥ 3	any
Hibernating	2–3	1–2	any
Lost	1	any	any

3.5.3 Why RFM over K-Means for This Use Case

Decision Justification

K-Means clustering was considered as an alternative segmentation approach. Three arguments favour RFM quintiles for this dataset:

1. **Interpretability**: Business stakeholders can understand and act on "Champions" and "At Risk" segments. K-Means cluster labels are arbitrary without post-hoc labelling.
2. **Class imbalance**: With 97% single-purchase customers, K-Means on raw RFM values would produce clusters dominated by F=1 customers with little sub-group differentiation. Quintile scoring spreads customers across scores regardless of the raw distribution.
3. **Operational directness**: Each segment maps to a specific marketing action (reactivation for "At Risk", retention for "Champions"). K-Means clusters require interpretation before action; RFM segments are pre-labelled for action.

The downside of NTILE-based RFM is that quintile boundaries are not stable: they shift if the customer base grows. For production use, fixed absolute thresholds (e.g., R<30 days = score 5) would be more stable but require domain calibration.

3.6 Activation Analysis

Source files: `sql/04_ltv_activation.sql` (Part 2), `notebooks/04_rfm_ltv_activation.ipynb`, `streamlit/pages/3_segmentos_clientes.py` (tab "Activacion")

3.6.1 Logistic Regression Setup

The activation analysis estimates the association between first-order characteristics and the probability of making a repeat purchase. The binary outcome is `is_repeat` (1 if two or more delivered orders, 0 otherwise). The model is a standard logistic regression:

$$\log\left(\frac{p}{1-p}\right) = \beta_0 + \beta_1 x_1 + \cdots + \beta_k x_k \tag{3.3}$$

where $p = \mathbb{P}(\text{repeat} = 1 \mid \mathbf{x})$ and predictors x_i include payment type (dummy-coded), first-order value (continuous), item count (categorical), and product category (top categories dummy-coded). The model is fitted with `statsmodels` (`Logit` class) to obtain coefficient estimates with standard errors and Wald test p-values.

3.6.2 Odds Ratios and Their Interpretation

Each logistic regression coefficient $\hat{\beta}_i$ is exponentiated to yield an odds ratio:

$$\widehat{\text{OR}}_i = \exp(\hat{\beta}_i) \tag{3.4}$$

An odds ratio greater than 1 indicates that the feature is associated with higher odds of repeat purchase, holding other features constant. The stored artefact `data/processed/activation_co` contains columns `feature`, `odds_ratio`, `p_value`, `ci_lower`, and `ci_upper` (95% Wald confidence intervals on the log-odds scale, exponentiated).

Key Insight

Voucher OR = 1.42: customers who paid their first order with a voucher (gift card or discount code) have 1.42 times the odds of returning compared to other payment methods, after controlling for order value, item count, and category. This is the strongest statistically significant predictor in the model.
Business interpretation: voucher users may have lower price sensitivity or may have been acquired through promotional channels that attract higher-intent buyers. The recommendation is to expand voucher/discount programmes targeting first-time buyers.

The dashboard renders the odds ratios on a $\log_2$ scale (forest plot style) so that symmetric bar lengths represent equivalent fold-changes above and below the null (OR = 1). Green bars indicate significantly positive associations ($p < 0.05$, OR > 1); red bars indicate significantly negative associations.

3.6.3 SQL Activation Rate Table

The SQL script `sql/04_ltv_activation.sql` (Part 2) computes raw repeat rates by feature group — a fast univariate view complementing the multivariate logistic regression. It uses a `UNION ALL` to stack results for payment type, item count bucket, and order value quartile:

Listing 3.8: Activation rates by payment type (excerpt from `sql/04_ltv_activation.sql`)

```
SELECT
    'payment_type'                          AS feature,
    first_payment_type                      AS feature_value,
    COUNT(*)                                AS customers,
    SUM(is_repeat)                          AS repeat_customers,
    ROUND(100.0 * SUM(is_repeat) / COUNT(*), 2) AS repeat_rate_pct
FROM first_order_features
GROUP BY first_payment_type
```

3.7 Revenue Concentration: Lorenz Curve and Gini Coefficient

Source files: `streamlit/pages/3_segmentos_clientes.py` (tab "Lorenz"), `notebooks/04_rfm_ltv_activat`

3.7.1 Lorenz Curve Construction

The Lorenz curve plots cumulative revenue share against cumulative customer share, both sorted from lowest to highest spender. At point $(p, L(p))$, the bottom p fraction of customers account for $L(p)$ fraction of total revenue. Perfect equality would yield $L(p) = p$ (the diagonal).

The Python implementation in `streamlit/pages/3_segmentos_clientes.py` computes the curve directly from the `total_revenue` column of `rfm_segments.parquet`:

Listing 3.9: Lorenz curve and Gini calculation (`streamlit/pages/3_segmentos_clientes.py`, lines 329–335)

```
revenue_sorted = np.sort(rfm_raw["total_revenue"].values)
n = len(revenue_sorted)
cum_revenue = np.cumsum(revenue_sorted) / revenue_sorted.sum()
cum_population = np.arange(1, n + 1) / n

gini = 1 - 2 * np.trapezoid(cum_revenue, cum_population)
```

3.7.2 Gini Coefficient Formula

The Gini coefficient is twice the area between the Lorenz curve and the line of equality:

$$G = 1 - 2 \int_0^1 L(p)\, \mathrm{d}p \tag{3.5}$$

In the discrete implementation, the integral is approximated with the trapezoidal rule (`np.trapezoid` in NumPy $\geq$ 2.0, falling back to `np.trapz` for earlier versions). $G = 0$ means perfect equality; $G = 1$ means one customer holds all revenue.

3.7.3 Business Interpretation

The Olist customer revenue distribution is highly concentrated. Quantifying the top-20% contribution directly:

Listing 3.10: Top-20% revenue share calculation

```
top_20_rev = cum_revenue[int(0.8 * n)]
```

```
top_20_share = (1 - top_20_rev) * 100  # % held by top 20%
```

The Champions segment (R $\geq$ 4, F $\geq$ 4, M $\geq$ 4) — approximately 0.1% of customers — generates outsized lifetime value (R$ 545 per customer at 12 months vs. R$ 160 average). This concentration justifies a differentiated retention programme for the top revenue tier.

3.7.4 LTV Curves by Segment

Cumulative LTV is computed in `sql/04_ltv_activation.sql` (Part 1) using a running sum window function:

Listing 3.11: Cumulative LTV per customer (from `sql/04_ltv_activation.sql`)

```
SUM(cr.monthly_revenue) OVER (
    PARTITION BY cr.cohort_month
    ORDER BY cr.months_since_cohort
    ROWS UNBOUNDED PRECEDING
) / cs.cohort_size  AS ltv_per_customer
```

The Streamlit page loads `data/processed/ltv_curves.parquet` via `load_ltv_curves()` and renders a multi-line chart with one series per RFM segment, coloured by the shared categorical palette from `streamlit/utils/charts.py`.

3.8 Dashboard Architecture: From Streamlit to a Static Export

Source files: `streamlit/app.py`, `streamlit/utils/data_loader.py`, `streamlit/utils/filters.py`, `streamlit/pages/` (6 page files)

This dashboard was built twice, and the reason for the rebuild is the most transferable thing in the chapter. The first implementation was a six-page Streamlit app on Cloud Run; the second is a six-page Next.js static export served from the portfolio hub at `data-analyst.gonor.me/cohorts`, with no backend at all. Both sections below are kept: the Streamlit design is described first because the rebuild only became visible *through* it.

3.8.1 First Implementation: Streamlit Page Structure

The original dashboard was a six-page Streamlit multi-page app launched via:

Listing 3.12: Run command (from `Dockerfile`)

```
CMD ["streamlit", "run", "streamlit/app.py",
```

```
    "--server.port=8501", "--server.address=0.0.0.0",
    "--server.headless=true"]
```

Table 3.4: Streamlit pages and their analytical content

Page file	Title	Key content
1_resumen_ejecutivo.py	Resumen Ejecutivo	KPI cards, revenue trend, retention funnel
2_retencion_cohortes.py	Retención por Cohortes	Heatmaps, KM survival, cohort comparator
3_segmentos_clientes.py	Segmentos de Clientes	RFM scatter, LTV, activation OR, Lorenz
4_analisis_geografico.py	Análisis Geográfico	State rankings, delivery-retention scatter
5_metodologia.py	Metodología	Definitions, calculation logic, data docs
6_proceso_tecnico.py	Proceso Técnico	Embedded Jupyter notebooks as HTML ifra

3.8.2 Data Loading and Caching (Streamlit)

All data loading was centralised in `streamlit/utils/data_loader.py`. Each parquet file has a dedicated `@st.cache_data` function so that repeated page navigations do not trigger re-reads from disk. The eight loader functions are:

Listing 3.13: Cached loaders in `streamlit/utils/data_loader.py`

```
@st.cache_data
def load_orders()          -> pd.DataFrame | None: ...  #
    orders_enriched.parquet
def load_customers()       -> pd.DataFrame | None: ...  #
    customers_summary.parquet
def load_cohort_retention()-> pd.DataFrame | None: ...  #
    cohort_retention_matrix.parquet
def load_cohort_revenue()  -> pd.DataFrame | None: ...  #
    cohort_revenue_retention.parquet
def load_survival()        -> pd.DataFrame | None: ...  #
    survival_data.parquet
def load_rfm()             -> pd.DataFrame | None: ...  #
    rfm_segments.parquet
def load_ltv_curves()      -> pd.DataFrame | None: ...  #
    ltv_curves.parquet
def load_activation()      -> pd.DataFrame | None: ...  #
    activation_coefficients.parquet
```

Functions `load_rfm()` and `load_ltv_curves()` additionally apply `_translate_segments()` to map English segment names to their Spanish equivalents for display (e.g., "Champions" → "Alto Valor").

3.8.3 Global Filter System — and What It Revealed

Filters were maintained in `st.session_state` and applied by helper functions in `streamlit/utils/filters.py`. Three global filters are available:

1. **Date range** (`date_start` / `date_end`): applied to orders via `apply_date_filter()` and to customers via `apply_date_filter_customers()` (which filters on `cohort_month` rather than `order_purchase_timestamp`).
2. **Minimum cohort size** (`min_cohort_size`, default 50): `apply_cohort_size_filter()` removes cohorts with fewer than `min_cohort_size` customers in month 0 from the retention matrices.
3. **RFM segment selection** (`selected_segments`): `apply_segment_filter()` subsets the RFM dataframe to selected segments.

Active filters are shown via `render_active_filter_badges()`, which renders coloured HTML badges without recomputing any data.

Key Insight

Read those three filters again and note what none of them does: recompute anything from the 96,478 delivered orders. A date range *selects columns* of an already-aggregated retention matrix; a minimum cohort size *drops rows* from it; a segment selection *subsets* an RFM table that was scored once, offline. Every one is a projection over a precomputed aggregate.

That is the entire argument for the rebuild. A stateful Python server is only required when a user action changes the computation. Here it never did — so the server was doing nothing that a browser could not do over a small JSON payload, while costing a container, a cold start, and a second design language on the same domain.

3.8.4 Second Implementation: Zero-Backend Static Export

The rebuild pre-aggregates 36 MB of parquet into **276 KB of JSON** (roughly 38 KB gzipped) via `data-pipeline/05_build_web_json.py`, and ships a Next.js application compiled with `output: 'export'`. The six pages, the filters and the interactivity all survive; the Cloud Run service does not.

Two consequences are worth stating because they generalise beyond this project:

1. **Correctness has to be proved, not assumed.** Moving the computation from request time to build time means a silent divergence between the parquet and the published JSON would be invisible. `data-pipeline/06_verify_parity.py` re-derives every published figure from the parquet and checks the Kaplan–Meier curve and both confidence bounds against `lifelines`. It cannot run in CI, because the parquet lives only in GCS.

Table 3.5: What moved where in the rebuild

Concern	Streamlit	Static export
Aggregation	per request, in Python	once, in the build pipeline
Filter state	`st.session_state`	React state in the browser
Kaplan–Meier	fitted per page load	precomputed, shipped as points
Transport	Cloud Run HTTP	static JSON under `public/cohorts/data/`
Notebooks	`components.v1.html` iframe	same HTML, served statically
Cold start	∼10 s	none

2. **The data became a build input, not an artifact.** The repository's global `*.json` ignore rule excluded the very files the dashboard renders from. The build still succeeded and would have deployed a dashboard with no numbers on it — a failure mode that returns HTTP 200. It is now covered by an explicit negation in `.gitignore`, a gate test asserting a rendered figure, and a health probe on `/cohorts/data/meta.json`.

3.8.5 Publishing the Notebooks ("Proceso Técnico")

Page 6 publishes the analytical pipeline inside the dashboard itself. The four Jupyter notebooks are pre-converted to static HTML using:

Listing 3.14: Notebook export command

```
jupyter nbconvert --to html \
  --output-dir=streamlit/notebooks_html \
  notebooks/*.ipynb
```

The HTML files are committed to git. Under Streamlit they were `COPY`-ed into the Docker image and rendered with `streamlit.components.v1.html()`:

Listing 3.15: Notebook rendering in `streamlit/pages/6_proceso_tecnico.py`, lines 113–116

```
with open(path, "r", encoding="utf-8") as f:
    html_content = f.read()

components.html(html_content, height=800, scrolling=True)
```

The static export keeps the pattern and drops the container: the same HTML now lives under the app's own `public/` slug and is rendered in a plain `<iframe>`. Either way, each tab shows one notebook with a description card explaining its inputs and outputs, giving portfolio reviewers full visibility into the analytical pipeline without leaving the dashboard.

Key Insight

Assets must live under the application's own slug (`public/cohorts/...`), never at the root. Seven dashboards share one origin in the merged hub, and a root-level asset from one of them would collide with another's. See Chapter 8.

3.9 SQL Scripts Reference

Source files: `sql/01_cohort_assignment.sql`, `sql/02_retention_matrix.sql`, `sql/03_rfm_segmentati`
`sql/04_ltv_activation.sql`, `sql/05_geographic_retention.sql`

The five SQL scripts are written for PostgreSQL and share a common structure: delivered orders are always the base CTE (filtering `order_status = 'delivered'`), and `customer_unique_id` is consistently used as the person-level key.

The geographic retention script uses the `FILTER` aggregate modifier to compute on-time delivery rate and repeat customer count in a single pass:

Listing 3.16: Conditional aggregation with FILTER (from `sql/05_geographic_retention.sql`)

```
ROUND(100.0 * COUNT(*) FILTER (
    WHERE d.order_delivered_customer_date <=
    d.order_estimated_delivery_date
) / COUNT(*), 2)                          AS on_time_pct,

COUNT(DISTINCT customer_unique_id) FILTER (
    WHERE order_count >= 2
)                                         AS repeat_customers
```

States with fewer than 50 customers are filtered out of the ranking to avoid noise from low sample sizes (`WHERE total_customers >= 50`).

3.10 Key Findings Summary

3.11 Challenge and Interview Questions

Challenge Question

Retention matrix normalisation. The retention matrix is constructed by dividing each row by its month-0 count. What happens if a customer places two orders in month 0? Are they counted twice in the denominator? Trace through the SQL in `02_retention_matrix.sql` to determine how this is handled.

Challenge Question

Censoring bias direction. The dataset ends in October 2018. For cohorts formed in September 2018, customers have had at most two months to reorder. In which direction does ignoring censoring bias the estimated repeat rate — and for which cohorts is the bias largest?

Challenge Question

Gini coefficient edge case. If all customers have exactly equal revenue, $G = 0$. If one customer holds all revenue, $G = 1$. Verify that the discrete trapezoidal approximation in the code converges to these limits for $n = 100$ customers.

Challenge Question

NTILE stability. The RFM scores are computed as quintiles of the current customer base. If 10,000 new customers join with low recency scores, how do the quintile boundaries shift, and what happens to existing Champions?

Interview Question

Why Kaplan-Meier rather than a simple proportion? A hiring manager asks: "Why not just compute repeat rate as 2801 / 93358 = 3%?" Explain in two sentences what information the KM curve adds that the simple proportion does not.

Interview Question

Interpreting OR = 1.42. Explain to a non-technical product manager what it means that voucher payment has an odds ratio of 1.42 for repeat purchase. Avoid the word "odds". Frame it in terms of risk or probability.

Interview Question

Confounding in the activation analysis. A colleague points out that voucher users may also have placed higher-value first orders, and higher order value is itself associated with return probability. How does the multivariate logistic regression address this concern, and what residual confounding might remain?

Interview Question

Lorenz curve business case. The Gini coefficient shows high revenue concentration. A VP asks: "Should we focus on acquiring more Champions or on converting Hibernating customers?" Walk through the data-driven argument for each position and state which you would recommend.

Table 3.6: SQL scripts, key techniques, and business questions

Script	Business question answered	Key technique
`01_cohort_assignment.sql`	Which cohort does each customer belong to?	`ROW_NUMBER()`, `DATE_TRUNC`, `AGE`
`02_retention_matrix.sql`	What % of each cohort returns each month?	`CASE-WHEN` pivot, `COUNT(DISTINCT)`
`03_rfm_segmentation.sql`	How do customers segment by RFM?	`NTILE(5)`, `CASE` segment mapping
`04_ltv_activation.sql`	What drives repeat purchase (activation)?	Running `SUM() OVER`, `DISTINCT ON`, `UNION ALL`
`05_geographic_retention.sql`	Which states retain better and why?	`FILTER (WHERE ...)`, `RANK() OVER`

Table 3.7: Key quantitative findings from the Olist analysis

Finding	**Value**	**Implication**
Repeat purchase rate	3.0%	Extreme retention challenge
Repeat customer total revenue	1.92x	Driven by 2.1 orders, not basket size
Repeat customer *per order*	0.91x	Returning buyers spend *less* per order
Voucher payment OR	1.42	Expand voucher first-order incentives
Late delivery rate	8.1%	Delivery accuracy is a retention lever
Median wait among returners	82 days	Concentrate, do not cut off, at Q1
Champions LTV (12 months)	R$ 545 vs. R$ 160 avg	Prioritise top segment

Chapter 4

A/B Test Analysis

Source files: `projects/03-ab-test-analysis/backend/abtest_backend/stats_engine.py`, `projects/03-ab-test-analysis/backend/abtest_backend/routers/`, `projects/03-ab-test-analysis/da` `projects/03-ab-test-analysis/src/components/abtest/`

This chapter documents the most statistically dense project in the portfolio: a full-stack A/B test analysis platform built for an e-commerce landing page experiment. The project implements four distinct statistical frameworks – Frequentist, Bayesian, Power Analysis, and Sequential Testing – each as pure Python functions in a single `stats_engine.py` module, served through a FastAPI backend and visualised in a Next.js 14 dashboard.

The central business question is: *Did a landing page redesign improve conversion rates sufficiently to justify shipping it to all users?* The answer turns out to be nuanced: the aggregate result is positive, but segment-level analysis reveals Simpson's Paradox – the treatment actively *hurts* returning users.

4.1 Business Context

Source files: `README.md`, `src/components/abtest/VerdictCard.tsx`

Every A/B test ends with one of three decisions: ship, do not ship, or collect more data. The cost of each wrong decision is asymmetric:

- **False positive (Type I, α)**: Ship a change that does not actually improve the metric. You waste engineering resources, potentially degrade UX for a subset of users, and incur opportunity cost.
- **False negative (Type II, β)**: Fail to ship a genuinely good change. You leave revenue on the table and slow the product roadmap.
- **Premature stopping**: End an experiment early based on noise. The decision may flip once more data arrives.

The project explicitly frames three verdict states: `SHIP IT`, `DON'T SHIP`, and `NEEDS MORE DATA` (shown in `routers/overview.py`). The overview router applies the rule: if $p \geq 0.05$, the verdict is "needs more data" regardless of the observed direction of the effect.

Key Insight

Statistical significance alone does not license a ship decision. This project combines significance ($p < 0.05$), effect size (Cohen's h), Bayesian confirmation ($\mathbb{P}(B > A) > 0.95$), and segment-level heterogeneity checks before committing to a recommendation.

The real-world result from this experiment:

- Aggregate conversion: treatment 12.33% vs. control 12.04%, $p = 0.017$ – statistically significant but Cohen's $h = 0.009$ (small).
- Treatment *hurts* returning users: 11.82% vs. 12.28%.
- Treatment helps mobile new users: +8.2% lift.
- Revenue uplift exists even where conversion lift is marginal: treatment converters spend \$67 vs. \$61 (+9.6% AOV).
- Recommendation: **do not ship universally**; ship for mobile new users only.

4.2 Data Source and Synthetic Enrichment

Source files: `data-pipeline/01_download.py`, `data-pipeline/02_clean.py`, `data-pipeline/03_enrich`

4.2.1 Base Dataset

The raw data is the Udacity E-Commerce A/B Test dataset from Kaggle (~294K rows, January 2017). Each row represents one user exposure with columns: `user_id`, `group` (`control`/`treatment`), `landing_page` (`old_page`/`new_page`), `converted` (0/1), and `timestamp`.

Cleaning steps (02_clean.py)

The cleaning pipeline drops two categories of noise:

1. **Mismatched rows**: 3,893 rows where the group/page assignment is inconsistent (e.g., treatment group exposed to the old page). These indicate assignment errors.
2. **Duplicate user IDs**: Only the first exposure per user is kept to ensure one row per experimental unit.

4.2.2 Synthetic Enrichment

Raw data contains only a binary conversion outcome. To build an analytically rich dashboard with segment filters and heterogeneous treatment effects, the enrichment script (`data-pipeline/03_enrich.py`) adds synthetic columns, all with `seed=42` for full reproducibility.

Column	Distribution	Notes
`device_type`	Categorical	mobile 55%, desktop 35%, tablet 10%
`browser`	Categorical	Chrome 60%, Safari 20%, Firefox 12%, Edge 8%
`country`	Categorical	US 40%, UK 15%, Canada 10%, ...
`user_segment`	Categorical	new 70%, returning 30%
`traffic_source`	Categorical	organic 40%, paid 30%, social 20%, referral 10%
`revenue`	Log-normal	$\mu = \ln(45)$, $\sigma = 0.8$; converters only; treatment $\times 1.08$
`session_duration_sec`	Gamma	Converted: shape=3, scale=60; non-converted: shape=2, scale=45
`pages_viewed`	Poisson	$\lambda = 3.8$ (treatment), $\lambda = 3.5$ (control)

Table 4.1: Synthetic columns added by the enrichment pipeline.

Baked-in heterogeneous treatment effects

Two deliberate distortions are introduced to create analytically interesting scenarios:

1. **Mobile boost**: Among non-converted treatment users on mobile, 1.5% are flipped to converted. This creates a device-treatment interaction.
2. **Returning-user penalty (Simpson's Paradox ingredient)**: Among converted returning-segment treatment users, 8% are flipped back to non-converted. Because returning users are a minority (30%), this segment-level negative effect is masked in the aggregate.

Decision Justification

Why bake in Simpson's Paradox? The Udacity dataset has a small aggregate effect (≈ 0.3 pp lift). Without heterogeneous effects, all segments would tell the same story. By engineering a returning-user penalty, the dashboard demonstrates a real analyst skill: resisting the aggregate result and always checking segment-level breakdown before shipping.

4.3 Frequentist Framework

Source files: `backend/abtest_backend/stats_engine.py` (lines 19–125), `backend/abtest_backend/routers`

The frequentist module implements four functions: `z_test_proportions`, `chi_squared_test`, `wilson_confidence_interval`, and `cohens_h`. All are pure functions with no side

effects.

4.3.1 Two-Proportion Z-Test

The workhorse of binary A/B tests. Under $H_0 : p_A = p_B$, the test statistic is:

$$Z = \frac{\hat{p}_B - \hat{p}_A}{\text{SE}_{\text{pooled}}} \tag{4.1}$$

where the **pooled standard error** assumes the null hypothesis of equal proportions:

$$\text{SE}_{\text{pooled}} = \sqrt{\hat{p}_{\text{pool}}\left(1 - \hat{p}_{\text{pool}}\right)\left(\frac{1}{n_A} + \frac{1}{n_B}\right)} \tag{4.2}$$

$$\hat{p}_{\text{pool}} = \frac{c_A + c_B}{n_A + n_B} \tag{4.3}$$

Here c_A and c_B are conversion counts; n_A and n_B are group sizes. The two-sided p-value is:

$$p = 2\left(1 - \Phi\left(|Z|\right)\right) \tag{4.4}$$

where Φ is the standard normal CDF. This is exactly how `z_test_proportions` implements it, using `scipy.stats.norm.cdf`.

Listing 4.1: z_test_proportions – core computation (stats_engine.py lines 38–43)

```
p_pool = (conv_a + conv_b) / (n_a + n_b)
se_pool = np.sqrt(p_pool * (1 - p_pool) * (1 / n_a + 1 / n_b))
z_stat = diff / se_pool if se_pool > 0 else 0.0
p_value = 2 * (1 - sp_stats.norm.cdf(abs(z_stat)))
```

Pooled vs. unpooled standard error

The distinction matters for confidence intervals. The pooled SE is used for the test statistic because H_0 asserts $p_A = p_B$, so pooling is correct. However, the CI for the *difference* $(\hat{p}_B - \hat{p}_A)$ does not assume equality, so the **unpooled** SE is used:

$$\text{SE}_{\text{unpooled}} = \sqrt{\frac{\hat{p}_A(1 - \hat{p}_A)}{n_A} + \frac{\hat{p}_B(1 - \hat{p}_B)}{n_B}} \tag{4.5}$$

The 95% CI for the difference is then:

$$(\hat{p}_B - \hat{p}_A) \pm z_{0.975} \cdot \text{SE}_{\text{unpooled}} \tag{4.6}$$

4.3.2 Wilson Score Confidence Intervals

The standard Wald interval ($\hat{p} \pm z\sqrt{\hat{p}(1-\hat{p})/n}$) performs poorly near 0 and 1, and for small samples. The Wilson score interval corrects this by inverting the score test statistic. Its closed form is:

$$\text{CI}_{\text{Wilson}} = \frac{\hat{p} + \dfrac{z^2}{2n} \pm z\sqrt{\dfrac{\hat{p}(1-\hat{p})}{n} + \dfrac{z^2}{4n^2}}}{1 + \dfrac{z^2}{n}} \tag{4.7}$$

where $z = z_{\alpha/2}$ is the standard normal quantile. The implementation in `wilson_confidence_interva` factorises this as:

Listing 4.2: Wilson CI (stats_engine.py lines 95–100)

```
z = sp_stats.norm.ppf(1 - alpha / 2)
p_hat = successes / trials
denom = 1 + z**2 / trials
center = (p_hat + z**2 / (2 * trials)) / denom
margin = z * np.sqrt(
    (p_hat * (1 - p_hat) + z**2 / (4 * trials)) / trials
) / denom
```

The Wilson interval is always bounded within $[0, 1]$, unlike the Wald interval, making it the standard choice for proportion CIs in practice.

4.3.3 Chi-Squared Test

The chi-squared test of independence provides an alternative test for the 2×2 contingency table (group × converted). For a 2×2 table with no Yates correction:

$$\chi^2 = \sum_{i,j} \frac{(O_{ij} - E_{ij})^2}{E_{ij}} \tag{4.8}$$

where O_{ij} are observed counts and $E_{ij} = (\text{row total}) \times (\text{col total})/n$ are expected counts under independence. With one degree of freedom, $Z^2 \sim \chi^2_1$, so this test is exactly equivalent to the two-sided z-test.

The implementation adds **Cramér's V** as a normalised effect size:

$$V = \sqrt{\frac{\chi^2}{n}} \tag{4.9}$$

4.3.4 Cohen's h Effect Size

Cohen's h measures the practical significance of a difference between two proportions on the arcsine-transformed scale, which stabilises variance:

$$h = 2\arcsin(\sqrt{p_2}) - 2\arcsin(\sqrt{p_1}) \tag{4.10}$$

Interpretation thresholds: $|h| < 0.2$ is small, 0.2–0.5 is medium, > 0.5 is large. In this experiment, $h = 0.009$ – well below the small threshold – meaning the result, while statistically significant at $n \approx 290{,}000$, has negligible practical effect.

Listing 4.3: Cohen's h (stats_engine.py lines 113–114)

```
h = 2 * np.arcsin(np.sqrt(p2)) - 2 * np.arcsin(np.sqrt(p1))
```

Key Insight

The arcsine transformation $\varphi(p) = 2\arcsin(\sqrt{p})$ is a variance-stabilising transform for binomial proportions. The variance of $\varphi(\hat{p})$ is approximately $1/n$ regardless of the true p, making comparisons on this scale more uniform across the $[0, 1]$ range.

4.3.5 Sample Ratio Mismatch (SRM) Test

Before trusting any result, the experiment health must be verified. A **Sample Ratio Mismatch** occurs when the observed group split deviates significantly from the planned 50/50 allocation. This signals a randomisation bug: bots, cookie expiry, or differential dropout. The SRM test uses a chi-squared goodness-of-fit test against expected counts of $n/2$:

$$\chi^2_{\text{SRM}} = \frac{(n_A - n/2)^2}{n/2} + \frac{(n_B - n/2)^2}{n/2} \tag{4.11}$$

with one degree of freedom. A p-value below 0.01 flags the experiment as imbalanced (`is_balanced = False`).

Listing 4.4: SRM test (stats_engine.py lines 455–462)

```
expected = total / 2
chi2 = ((n_control - expected) ** 2 / expected
        + (n_treatment - expected) ** 2 / expected)
p_value = 1 - sp_stats.chi2.cdf(chi2, df=1)
return {
    "chi2": round(float(chi2), 6),
    "p_value": round(float(p_value), 6),
    "is_balanced": bool(p_value > 0.01),
}
```

Decision Justification

Why 0.01 not 0.05 for SRM? An SRM is a data quality failure, not a statistical finding. Using a stricter threshold reduces the chance of dismissing a valid SRM as random noise. If SRM is detected, the entire test result is suspect and should not be acted upon.

4.4 Bayesian Framework

Source files: `backend/abtest_backend/stats_engine.py` (lines 128–234), `backend/abtest_backend/router`

The Bayesian approach treats the true conversion rates θ_A and θ_B as random variables with prior distributions, updated to posterior distributions after observing the data.

4.4.1 Beta-Binomial Conjugacy

For binary outcomes, the natural likelihood is Binomial:

$$c \mid \theta, n \sim \text{Binomial}(n, \theta)$$

The conjugate prior for the Binomial likelihood is the Beta distribution:

$$\theta \sim \text{Beta}(\alpha_0, \beta_0)$$

Conjugacy means the posterior is also Beta, with analytically tractable parameters:

$$\theta \mid c, f \sim \text{Beta}(\alpha_0 + c,\ \beta_0 + f) \tag{4.12}$$

where c is observed conversions and $f = n - c$ is failures. This is the simplest possible Bayesian update: add successes to α, add failures to β. The proof follows from Bayes' theorem:

$$\begin{aligned} p(\theta \mid c, f) &\propto p(c \mid \theta)\, p(\theta) \\ &\propto \theta^c (1-\theta)^f \cdot \theta^{\alpha_0 - 1} (1-\theta)^{\beta_0 - 1} \\ &= \theta^{(\alpha_0 + c) - 1} (1-\theta)^{(\beta_0 + f) - 1} \\ &\propto \text{Beta}(\alpha_0 + c,\ \beta_0 + f) \end{aligned} \tag{4.13}$$

The bayesian router uses a **uniform (uninformative) prior**: $\alpha_0 = \beta_0 = 1$, equivalent to saying every conversion rate in $[0, 1]$ is equally plausible before seeing data. With $n_A \approx 145{,}000$ observations, the posterior is extremely tight and the prior choice is irrelevant.

4.4.2 $\mathbb{P}(B > A)$ via Monte Carlo

The key Bayesian decision metric is the probability that the treatment rate exceeds the control rate:

$$\mathbb{P}(\theta_B > \theta_A) = \int_0^1 \int_{\theta_A}^1 f(\theta_A)\, f(\theta_B)\, \mathrm{d}\theta_B\, \mathrm{d}\theta_A \tag{4.14}$$

This double integral does not have a simple closed form for general Beta parameters. The implementation estimates it via Monte Carlo simulation with 100,000 samples, seeded at 42 for reproducibility:

Listing 4.5: P(B > A) Monte Carlo (stats_engine.py lines 166–169)

```
rng = np.random.default_rng(42)
samples_a = rng.beta(post_a["alpha"], post_a["beta"], size=n_simulations)
samples_b = rng.beta(post_b["alpha"], post_b["beta"], size=n_simulations)
return round(float(np.mean(samples_b > samples_a)), 6)
```

The result for this experiment: $\mathbb{P}(B > A) = 0.992$, meaning there is a 99.2% posterior probability that treatment converts better than control.

4.4.3 Expected Loss

$\mathbb{P}(B > A)$ alone can be misleading because it ignores the *magnitude* of being wrong. The **expected loss** (also called expected regret) quantifies the cost of making the wrong choice:

$$\mathcal{L}_A = \mathbb{E}[\max(\theta_B - \theta_A, 0)] \qquad \text{(loss if you choose A when B is better)} \tag{4.15}$$

$$\mathcal{L}_B = \mathbb{E}[\max(\theta_A - \theta_B, 0)] \qquad \text{(loss if you choose B when A is better)} \tag{4.16}$$

These are estimated via Monte Carlo using the same posterior samples:

Listing 4.6: Expected loss (stats_engine.py lines 188–190)

```
loss_a = float(np.mean(np.maximum(samples_b - samples_a, 0)))
loss_b = float(np.mean(np.maximum(samples_a - samples_b, 0)))
```

The **decision rule**: ship when $\mathcal{L}_B < \epsilon$ for some small threshold ϵ (e.g., 0.001). If the expected loss from shipping is near zero, ship even if $\mathbb{P}(B > A)$ is below 0.95. In this experiment, $\mathcal{L}_B$ is near zero, confirming the frequentist result.

4.4.4 Credible Intervals vs. Confidence Intervals

The 95% **credible interval** for θ is the central 95% of the posterior Beta distribution:

$$\text{CrI}_{0.95}(\theta) = \left[F^{-1}_{\text{Beta}}(0.025;\, \alpha, \beta),\ F^{-1}_{\text{Beta}}(0.975;\, \alpha, \beta)\right] \tag{4.17}$$

where F^{-1}_{Beta} is the Beta quantile function, implemented via `sp_stats.beta.ppf`.

Key Insight

The correct interpretation of a 95% **credible interval** $[L, U]$: "Given the data, there is a 95% probability that the true conversion rate lies between L and U". This is the statement most practitioners want to make about a confidence interval but cannot: a 95% CI only guarantees that if the experiment were repeated many times, 95% of such intervals would contain the true value – a statement about the procedure, not about this particular interval.

4.5 Power Analysis

Source files: `backend/abtest_backend/stats_engine.py` (lines 237–337), `backend/abtest_backend/route`
`src/components/abtest/PowerCalculator.tsx`

Power analysis answers the pre-experiment question: *how many users do we need to collect?* It must be done before running the experiment – computing it post-hoc is invalid because the result is trivially correct for the observed effect.

4.5.1 Sample Size Formula

The required sample size per group for a two-proportion z-test is derived from requiring the non-centrality parameter of the test statistic to exceed the critical value at the desired power:

$$n = \frac{\left(z_{\alpha/2}\sqrt{2\,p_1(1-p_1)} + z_{\beta}\sqrt{p_1(1-p_1) + p_2(1-p_2)}\right)^2}{(p_2 - p_1)^2} \tag{4.18}$$

where:

- p_1 = baseline conversion rate (control)
- $p_2 = p_1 + \delta$ = expected treatment conversion rate (δ = MDE, minimum detectable effect)

- $z_{\alpha/2}$ = critical value for significance level α (1.96 for $\alpha = 0.05$, two-sided)
- z_β = critical value for power $1 - \beta$ (0.842 for 80% power)

Listing 4.7: required_sample_size (stats_engine.py lines 262–272)

```
p1 = baseline_rate
p2 = baseline_rate + mde
z_alpha = sp_stats.norm.ppf(1 - alpha / 2)
z_beta = sp_stats.norm.ppf(power)
numerator = (
    z_alpha * np.sqrt(2 * p1 * (1 - p1))
    + z_beta * np.sqrt(p1 * (1 - p1) + p2 * (1 - p2))
) ** 2
denominator = (p2 - p1) ** 2
return int(np.ceil(numerator / denominator))
```

Intuition for the formula

Equation (4.18) has a clean structure:

- The denominator $\delta^2 = (p_2 - p_1)^2$ means sample size grows as the *square* of the inverse of the effect: detecting a 0.5 pp effect requires 4× the sample needed for a 1 pp effect.
- Conversion rates near 0.5 have the highest variance ($p(1-p)$ is maximised at $p = 0.5$), requiring the most data.
- Increasing power from 80% to 90% replaces $z_\beta = 0.842$ with $z_\beta = 1.282$, increasing required n by roughly 25%.

4.5.2 Minimum Detectable Effect via Binary Search

Given a *fixed* sample size n, the inverse question is: what is the smallest effect δ the test can detect at the specified power? There is no closed-form inverse of equation (4.18), so the implementation uses binary search over $\delta \in (0.0001, 0.5)$:

Listing 4.8: minimum_detectable_effect (stats_engine.py lines 294–302)

```
lo, hi = 0.0001, 0.5
for _ in range(100):
    mid = (lo + hi) / 2
    needed = required_sample_size(baseline_rate, mid, alpha, power)
    if needed <= n:
        hi = mid
    else:
        lo = mid
return round(float(hi), 6)
```

After 100 bisection iterations, the interval width is $(0.5 - 0.0001)/2^{100} \approx 4 \times 10^{-31}$, providing effectively exact results.

4.5.3 Power Curve

For a fixed sample size n, statistical power as a function of effect size δ is:

$$\text{Power}(\delta) = \Phi\left(\frac{\delta}{\text{SE}} - z_{\alpha/2}\right) + \Phi\left(-\frac{\delta}{\text{SE}} - z_{\alpha/2}\right) \tag{4.19}$$

where $\text{SE} = \sqrt{p_1(1-p_1)/n + p_2(1-p_2)/n}$. The second term accounts for the lower tail of the two-sided test and is typically negligible. The frontend interactive sliders in `PowerCalculator.tsx` recompute this in real time for any combination of baseline rate, MDE, alpha, and power.

4.5.4 Runtime Estimation

The power router converts required sample size into calendar time:

$$\text{runtime} = \left\lceil \frac{\max(n_{\text{required}} - n_{\text{current}},\ 0)}{\bar{d}/2} \right\rceil + 1 \text{ days} \tag{4.20}$$

where $\bar{d}$ is the mean daily traffic (each group receives half). This gives a concrete answer to "when can we make a decision?", which is what product managers actually ask.

4.6 Sequential Testing

Source files: `backend/abtest_backend/stats_engine.py` (lines 340–464), `backend/abtest_backend/router`
`src/components/abtest/SequentialChart.tsx`

4.6.1 The Peeking Problem

The classical frequentist framework requires that the sample size be fixed *before* running the experiment. If analysts check p-values daily and stop when $p < 0.05$, the actual Type I error rate inflates dramatically above the nominal α.

If we check at K equally spaced looks and stop at the first crossing of $p < \alpha$ per look, the true experiment-wise error rate is approximately:

$$\alpha^* \approx 1 - (1 - \alpha)^K \tag{4.21}$$

For $K = 30$ daily looks at $\alpha = 0.05$: $\alpha^* \approx 0.79$. The p-value evolution chart in the dashboard makes this viscerally clear: the cumulative p-value frequently dips below 0.05 early in the experiment due to random fluctuation, then recovers.

4.6.2 O'Brien-Fleming Spending Function

Sequential testing solves the peeking problem by pre-specifying a spending function that allocates the total α budget across all planned interim analyses. The **O'Brien-Fleming** (OBF) approach uses a conservative spending function that allocates very little α to early looks and most to the final look.

The OBF boundary at look k of K total looks is:

$$z_k^* = \frac{z_{\alpha/2}}{\sqrt{k/K}} \tag{4.22}$$

where $z_{\alpha/2} = 1.96$ for $\alpha = 0.05$. The information fraction $k/K \in (0, 1]$ causes early boundaries to be very conservative (at $k = 1$, $K = 30$: $z_1^* = 1.96/\sqrt{1/30} \approx 10.73$) and the final boundary to be approximately equal to the fixed-sample critical value.

Listing 4.9: OBF boundaries (stats_engine.py lines 431–438)

```
for k in range(1, n_looks + 1):
    info_fraction = k / n_looks
    z_alpha = sp_stats.norm.ppf(1 - alpha / 2)
    boundary = z_alpha / np.sqrt(info_fraction)
    boundaries.append(round(float(boundary), 6))
```

The sequential endpoint identifies the first day where the cumulative z-statistic crosses the OBF boundary:

Listing 4.10: Optimal stopping detection (routers/sequential.py lines 62–70)

```
for i, day in enumerate(cum_stats):
    if i < len(boundaries) and abs(day["z_stat"]) >= boundaries[i]:
        optimal_stopping = {
            "date": day["date"],
            "day_number": i + 1,
            "z_stat": day["z_stat"],
            "boundary": boundaries[i],
        }
        break
```

4.6.3 Why O'Brien-Fleming Over Pocock?

Decision Justification

Two common group sequential spending functions are OBF and Pocock. Pocock uses a *constant* critical value across all looks ($z_k^* = c$ for all k), which sounds intuitive but pays a heavy price at the final look: the overall α constraint forces $c > z_{\alpha/2}$, making the final look *harder* to reach significance than a fixed-sample test.

OBF, by contrast, uses nearly all its α budget at the final look ($z_K^* \approx z_{\alpha/2}$). This means: (a) early stopping requires an overwhelming effect, which is appropriate since early data is noisy; (b) if the experiment runs to completion without early stopping, the final p-value threshold is almost identical to the standard test. OBF is therefore the conservative, defensible choice.

Look k (of $K = 30$)	OBF boundary z_k^*	Pocock boundary
1	10.73	2.67
5	4.80	2.67
10	3.39	2.67
20	2.40	2.67
30	1.96	2.67

Table 4.2: OBF vs. Pocock boundaries for $K = 30$ looks, $\alpha = 0.05$. OBF allocates nearly all α to the final look; Pocock is uniform.

4.7 Simpson's Paradox

Source files: `backend/abtest_backend/routers/segments.py`, `src/components/abtest/SimpsonParadox.`

4.7.1 Definition

Simpson's Paradox occurs when a trend or association observed in aggregate data *reverses* (or disappears) when the data is broken into subgroups. It arises because of a confounding variable that is unequally distributed between the groups being compared.

In this experiment:

- **Aggregate**: treatment 12.33% > control 12.04% ⇒ positive lift.
- **Returning users**: treatment 11.82% < control 12.28% ⇒ negative lift.
- **New users**: treatment 12.54% > control 12.00% ⇒ positive lift.

The positive aggregate result is driven by the larger new-user segment (70% of traffic), masking the negative returning-user effect. A product team acting on the aggregate result alone would ship a feature that demonstrably hurts 30% of users.

4.7.2 Detection Algorithm

The function `_simpsons_paradox` (segments router, lines 63–119) computes:

1. The aggregate treatment direction: agg_dir = "treatment" $\iff \hat{p}_B > \hat{p}_A$.
2. Per-segment directions for each value of `user_segment`.
3. Paradox flag: `detected = True` iff *all* segments have a direction *opposite* to the aggregate direction.

Listing 4.11: Simpson's Paradox detection (routers/segments.py lines 106–113)

```
all_opposite = all(
    s["direction"] != agg_direction for s in segment_directions
)
return {
    "detected": all_opposite and len(segment_directions) > 1,
    "aggregate_direction": agg_direction,
    "aggregate_lift": round(agg_lift, 4),
    "segments": segment_directions,
}
```

4.7.3 Causal Explanation

The paradox arises from unequal weighting of subgroups across the comparison. The formal condition: let C be a covariate with values $c_1, \ldots, c_J$. If treatment rate $p_B > p_A$ in aggregate but $p_{B|c_j} < p_{A|c_j}$ for all j, then C is a confounder and the aggregate comparison is misleading:

$$p_B = \sum_j w_{B,j}\, p_{B|c_j} \neq \sum_j w_{A,j}\, p_{A|c_j} = p_A \tag{4.23}$$

where $w_{B,j} \neq w_{A,j}$ are the stratum weights in each group. The weighted average is driven by the weights, not just the within-stratum rates.

Key Insight

The detection condition "all segments opposite" is strict and catches the strongest form of the paradox. In practice, partial reversals (some segments positive, some negative) are more common and equally actionable. A richer detector would flag any segment where the direction disagrees with the aggregate, regardless of whether all segments agree with each other.

4.8 Architecture

Source files: `backend/abtest_backend/main.py`, `backend/abtest_backend/data_loader.py`, `src/components/abtest/ABTestDashboard.tsx`

4.8.1 Design Philosophy: Pure Functions in a Single Engine

The entire statistical computation layer lives in a single file, `stats_engine.py`, with a deliberate constraint: all functions are **pure** (no side effects, no global state, no I/O). This makes them independently testable, trivially importable from routers or notebooks, and auditable – reviewers can verify the formulas without understanding the API layer.

The module is organised into four blocks separated by ASCII banner comments: Frequentist (lines 14–125), Bayesian (128–234), Power Analysis (237–337), and Sequential Testing (340–464).

4.8.2 API Layer

The FastAPI application mounts six routers, each corresponding to one dashboard tab:

Endpoint	Router	Key functions
`GET /api/v1/overview`	`overview.py`	`z_test_proportions`, `sample_ratio_mismatch_test`
`GET /api/v1/frequentist`	`frequentist.py`	`z_test_proportions`, `chi_squared_test`, `wilson_confid`
`GET /api/v1/bayesian`	`bayesian.py`	`beta_posterior`, `probability_b_beats_a`, `expected_los`
`GET /api/v1/segments`	`segments.py`	`z_test_proportions` (per segment)
`GET /api/v1/power`	`power.py`	`required_sample_size`, `minimum_detectable_effect`, `po`
`GET /api/v1/sequential`	`sequential.py`	`cumulative_stats_by_day`, `obrien_fleming_bounds`

Table 4.3: API endpoints and their statistical function dependencies.

All endpoints accept optional filter query parameters (`device_type`, `browser`, `country`, `user_segment`, `traffic_source`), applied via `data_loader.apply_filters()`. This enables live segment exploration from the frontend without any server restarts.

4.8.3 Data Loading

The data loader (`data_loader.py`) reads the enriched parquet at module import time, pre-computes filter option lists, and exposes an `apply_filters()` function. Loading once at startup keeps API latency low for the ≈ 290K-row dataset.

4.8.4 Frontend

The Next.js 14 dashboard (`src/components/abtest/ABTestDashboard.tsx`) uses `ABTestFilterContex` to share filter state across all tabs, and SWR hooks (`useOverview`, `useFrequentist`, etc.) for data fetching with automatic revalidation on filter change. Recharts renders all visualisations: area charts for posterior PDFs, composed charts for sequential z-statistics with OBF boundaries, and line charts for power curves.

4.9 Challenge and Interview Questions

Challenge Question

Challenge 1: The z-test uses pooled SE. Why not always use unpooled SE for both the test statistic and the confidence interval?

The pooled SE is correct *under the null hypothesis* because H_0 asserts $p_A = p_B = p$, so pooling gives the MLE of the common proportion. Using pooled SE for the test statistic maximises power under H_0.

For the CI, we are estimating the true difference, not testing that it is zero. Under the alternative, $p_A \neq p_B$, so we should not constrain them to be equal. Unpooled SE is the correct estimator of $\text{Var}(\hat{p}_B - \hat{p}_A)$ without imposing the null. Using pooled SE for the CI would under-cover the true parameter in the alternative.

Challenge Question

Challenge 2: The OBF boundary at look 1 of 30 is 10.73. Is this achievable in practice, and why does such a conservative boundary make sense?

A z-statistic of 10.73 corresponds to a p-value of $\approx 10^{-26}$. With the experiment's daily traffic (≈ 9,700 users/day), after day 1 $n \approx 4{,}850$ per group. For the observed lift of 0.29 pp, the z-statistic would be ≈ 0.5, nowhere near the boundary.

This is the feature, not a bug. To cross the day-1 boundary, the effect would need to exceed 10.73 standard errors – a treatment that catastrophically breaks the site would show this. OBF's logic is: only stop early if the evidence is so overwhelming that no reasonable amount of additional data could change the conclusion. In the absence of such a catastrophic signal, early stopping risks acting on noise.

Challenge Question

Challenge 3: Explain why $Z^2 = \chi^2$ for a 2×2 contingency table.

For a 2×2 table with groups A and B and binary outcome, the Pearson chi-squared statistic (no Yates correction) equals:

$$\chi^2 = n \cdot \frac{(ad - bc)^2}{(a+b)(c+d)(a+c)(b+d)}$$

where $a = c_A$, $b = n_A - c_A$, $c = c_B$, $d = n_B - c_B$. Expanding the z-test statistic $Z = (\hat{p}_B - \hat{p}_A)/\text{SE}_{\text{pooled}}$ and squaring produces exactly the same expression. This algebraic identity means the two-sided z-test and the chi-squared test are numerically identical – they differ only in presentation: the z-test gives directionality; the chi-squared does not.

Challenge Question

Challenge 4: Can the expected loss be negative? What does it mean for a loss to be very close to zero?

No. By construction, $\max(\theta_B - \theta_A, 0) \geq 0$ for every sample, so its expectation is also non-negative. Both $\mathcal{L}_A$ and $\mathcal{L}_B$ are zero only if the posteriors are degenerate point masses at identical values, which is impossible with real data.

In practice, one loss is very small (near-certain winner) and the other is much larger. The decision threshold $\epsilon = 0.001$ means: if shipping B costs on average less than 0.001 conversion rate points in the worst case, ship it. The absolute scale depends on the metric: for revenue, 0.001 per user × 1M users = \$1,000 potential loss per decision cycle.

Interview Question

Interview Q1: Your PM wants to check the p-value every morning and stop when it hits 0.05. How do you explain the problem?

Use the gambler analogy: if you flip a fair coin 100 times and stop the first time you see 10 heads in a row, you will incorrectly conclude the coin is biased. Daily peeking inflates the experiment-wise Type I error to $\approx 79\%$ for 30 looks at $\alpha = 0.05$.

Propose the sequential testing solution: pre-commit to K looks and use OBF boundaries. Show the PM the "Daily P-Value Evolution" chart – the p-value frequently crosses 0.05 during the experiment and comes back. If they had acted on the early crossing, they would have shipped a change whose true p-value was > 0.05.

Interview Question

Interview Q2: The aggregate A/B test shows $p = 0.017$ and you prepare to recommend shipping. A colleague points out that returning users have worse performance in treatment. What do you do?

This is exactly Simpson's Paradox. The correct response is: do not ship universally. Steps:

1. Verify the returning-user result is statistically significant: run the z-test within that segment.
2. Quantify the magnitude: is the returning-user harm larger or smaller than the new-user gain in absolute revenue terms?
3. Check revenue: even if conversion is hurt, does AOV uplift compensate?
4. Recommendation: targeted rollout (ship to new users only; hold for returning users). Run a separate returning-user test with a variant designed for that segment.

Never aggregate away heterogeneous effects in a ship/don't-ship decision.

Interview Question

Interview Q3: How do you verify the experiment ran correctly before trusting the result?

Run four checks:

1. **SRM test**: Does the observed 50/50 split hold (χ^2_{SRM} p-value > 0.01)? If not, there is an assignment bug and the result cannot be trusted.
2. **A/A test equivalence**: If an A/A test was run concurrently, it should show no significant difference ($p > 0.05$).
3. **Pre-experiment baseline**: Were control and treatment groups similar before the experiment started (same historical conversion rates)?
4. **Novelty effect**: Is the experiment long enough (typically ≥ 2 weeks) to dilute the novelty effect from users encountering a new design for the first time?

Chapter 5

Executive KPI Report

5.1 Business Context

Source files: `projects/04-executive-kpi-report/README.md`, `data-pipeline/01_generate_saas_data.p`

Executives at B2B SaaS companies live and die by a handful of metrics: is revenue growing, are customers staying, and is the unit economics engine working? The problem is that most companies produce these numbers manually – an analyst pulls SQL, pastes into Excel, formats a slide deck, and emails a PDF every month. The process takes two days, is prone to copy-paste errors, and the result is stale by the time it lands in inboxes.

This project builds an automated alternative. The fictional company **NovaCRM** is a mid-market B2B SaaS firm in the $5M–$12M ARR range. The system:

1. Generates 24 months of reproducible synthetic data with deliberate business events injected.
2. Computes the full SaaS metric suite: MRR waterfall, NRR, churn rates, unit economics, engagement, and financial health.
3. Runs anomaly detection across all KPIs to surface statistically unusual months.
4. Produces 6-period forecasts with confidence intervals using Holt-Winters exponential smoothing.
5. Exposes everything through a FastAPI backend consumed by a Next.js dashboard.
6. Generates a downloadable bilingual (English / Spanish) PDF report on demand.

Key Insight

The key insight motivating automated KPI reporting is that **the value is not in the numbers but in the signal extraction**. A static table of MRR values tells an executive nothing. A report that says "churn spiked 2.3 standard deviations above the 6-month baseline in July 2024, coinciding with the pricing change" is actionable. Automation makes signal extraction reproducible and timely.

5.2 Data Generation

Source files: `data-pipeline/01_generate_saas_data.py`

Because no real SaaS company would share their internal KPI data, the project uses a deterministic synthetic generator. The seed `np.random.default_rng(42)` ensures every run produces identical numbers, which is critical for a portfolio project where reviewers must be able to reproduce charts independently.

5.2.1 Company Parameters

NovaCRM starts in January 2024 with:

- MRR: $420,000 (ARR ≈ $5.0M)
- 310 total customers across three segments: **Starter** (55% of logos, 15% of MRR), **Professional** (30% logos, 40% MRR), and **Enterprise** (15% logos, 45% MRR).
- NPS baseline: 45
- DAU: 4,800 / MAU: 14,500

The revenue-to-customer ratio illustrates a classic SaaS pattern: Enterprise customers are few in number but disproportionately large in revenue, making their churn economically devastating compared with Starter churn.

5.2.2 Seasonality Function

Business acquisition follows a seasonal pattern encoded in `_seasonal_factor()`:

Listing 5.1: Seasonal multiplier for new business.

```
def _seasonal_factor(month_dt: pd.Timestamp) -> float:
    m = month_dt.month
    if m == 12:
        return 0.65              # holiday stall
    if m in (6, 7, 8):
        return 0.80 + RNG.uniform(-0.03, 0.03)
    if m in (1, 2, 3):
        return 1.10 + RNG.uniform(-0.03, 0.03)  # Q1 budget flush
```

```
9       return 1.0 + RNG.uniform(-0.05, 0.05)
```

December drops to 65% of baseline – decision-makers are on holiday and budget approvals stall. Q1 surges 10% above baseline because companies deploy freshly approved budgets. Summer runs 20% below baseline. This mirrors empirically observed B2B seasonality.

5.2.3 Injected Business Events

Two events are deliberately baked into the simulation:

Q3 2024 – Pricing change churn spike. July 2024 receives a `pricing_severity` of 1.0 (full impact), August 0.6, September 0.25. This adds up to 2% to the monthly revenue churn rate, up to 2% to logo churn, spikes support tickets by 25%, and drops NPS by up to 10 points.

Q2 2025 – Product launch growth bump. May 2025 gets `launch_severity` = 1.0. New MRR inflates by 40%, expansion MRR by 70%, CAC drops by up to $1,500 due to inbound demand, and NPS gains up to 8 points.

These events serve a dual purpose: they make the data narratively interesting (something actually happened), and they validate the anomaly detection system – a detector that misses a 2% churn spike is not useful.

5.2.4 MRR Waterfall Components

Each month, MRR evolves through four components:

$$\text{MRR}_t = \text{MRR}_{t-1} + \text{New}_t + \text{Expansion}_t - \text{Contraction}_t - \text{Churned}_t \tag{5.1}$$

The generation targets a long-run NRR of approximately 108%, achieved through:

- Expansion rate: $\approx 3.0\%$ of MRR/month (growing slowly as Enterprise share increases)
- Revenue churn rate: $\approx 2.0\%$ of MRR/month
- Contraction rate: $\approx 0.6\%$ of MRR/month
- Net monthly retention ratio: ≈ 1.007, which annualises to $1.007^{12} \approx 1.087$

Decision Justification

Why simulate rather than use a public dataset? Public SaaS datasets either lack the metric granularity needed (most share only revenue-level data), or they contain real customer information that creates privacy concerns. Simulation allows full control over the narrative arc: specific events at specific months, with known ground truth to validate the detection algorithms. The trade-off is that reviewers must trust the generation logic – which is why the seed and all parameters are fully documented.

5.3 SaaS Metrics Deep-Dive

Source files: `data-pipeline/02_compute_kpis.py`, `backend/kpi_backend/routers/overview.py`

5.3.1 Revenue Metrics

MRR and ARR. Monthly Recurring Revenue is the sum of all contracted monthly subscription fees. Annual Recurring Revenue is simply ARR = 12 × MRR. Both exclude one-time fees, professional services revenue, and variable usage charges. The generator produces MRR ranging from $420K at start to approximately $780K–$900K by December 2025.

MRR Waterfall. The waterfall decomposition in `analytics_engine.mrr_waterfall()` breaks MRR movement into five labelled components:

$$\underbrace{\text{Ending MRR}}_{\text{result}} = \underbrace{\text{Starting MRR}}_{\text{base}} + \underbrace{\text{New}}_{\text{new logos}} + \underbrace{\text{Expansion}}_{\text{upsell/seat growth}} - \underbrace{\text{Contraction}}_{\text{downgrades}} - \underbrace{\text{Churned}}_{\text{cancellations}} \tag{5.2}$$

This decomposition is essential for diagnosis. A flat MRR trend could hide a healthy expansion engine completely offset by churn – or vice versa. The waterfall reveals which lever needs attention.

Net Revenue Retention (NRR). NRR measures revenue retained and grown from the existing customer base, excluding new logos:

$$\text{NRR} = \frac{\text{MRR}_{\text{start}} + \text{Expansion} - \text{Contraction} - \text{Churned}}{\text{MRR}_{\text{start}}} \tag{5.3}$$

The generator computes a monthly retention ratio and annualises it: $\text{NRR}_{\text{annual}} = r_{\text{monthly}}^{12}$. NRR > 1.0 (or > 100%) means the existing customer base is growing on its

own, without any new logos. SaaS companies with NRR $\geq 120\%$ can grow ARR even with zero new customer acquisition.

Table 5.1: NRR benchmarks for B2B SaaS companies.

NRR	Interpretation	Traffic Light
$\geq 120\%$	Best-in-class (e.g. Snowflake, Datadog)	Green
110–120%	Healthy – expansion offsets churn well	Green
100–110%	Adequate – moderate expansion	Yellow
$< 100\%$	Contracting existing base	Red

NovaCRM targets NRR = 110% (`NRR_TARGET = 1.10`). The Enterprise segment alone runs at $\approx 112\%$ due to seat expansion and upsell; Starter customers run below 100% because they churn faster than they expand.

5.3.2 Customer Metrics

Logo Churn Rate. Logo churn counts the fraction of customers who cancelled in a period:

$$\text{Logo Churn} = \frac{\text{Customers Churned}_t}{\text{Customers at Start of Period}_t} \tag{5.4}$$

The generator uses a beginning-of-period denominator: `total_customers + churned_customers`. Segment baselines are: Starter 5.5%/month, Professional 3.0%/month, Enterprise 1.2%/month. These reflect the empirical reality that smaller, cheaper customers are easier to cancel.

Revenue Churn Rate. Revenue churn measures the fraction of MRR lost to cancellations, not counting downgrades:

$$\text{Revenue Churn} = \frac{\text{Churned MRR}_t}{\text{MRR}_{t-1}} \tag{5.5}$$

Revenue churn exceeding logo churn signals that larger (more valuable) accounts are leaving at a higher rate – a critical warning sign. The `generate_customer_commentary()` function in `commentary.py` explicitly flags this condition when Revenue Churn$-$Logo Churn > 0.01.

LTV:CAC Ratio. Customer Lifetime Value divided by Customer Acquisition Cost is the fundamental unit economics check:

$$\text{LTV} = \frac{\text{ARPU} \times 12}{\text{Logo Churn Rate}} \times \text{Gross Margin} \tag{5.6}$$

$$\text{LTV:CAC} = \frac{\text{LTV}}{\text{CAC}} \tag{5.7}$$

The simplified formula assumes constant ARPU and churn. Gross margin adjustment (the project uses 80%) accounts for the fact that not all revenue flows to the bottom line. Industry benchmarks: LTV:CAC $\geq 3\times$ is healthy; $\geq 5\times$ is excellent; $< 1\times$ means the business loses money on every customer it acquires.

Payback Period. The number of months to recover CAC from a customer's gross profit:

$$\text{Payback Months} = \frac{\text{CAC}}{\text{ARPU}} \tag{5.8}$$

NovaCRM targets payback under 12 months. During the product launch (Q2 2025), inbound demand reduces CAC by up to \$1,500 and payback shortens accordingly.

5.3.3 The Rule of 40

The Rule of 40 is a heuristic balance between growth and profitability. SaaS companies that score ≥ 40 are considered healthy; below 40 indicates over-investment in growth without corresponding efficiency, or stagnation:

$$\text{Rule of 40} = \text{Revenue Growth Rate (annualised, \%)} + \text{Gross Margin (\%)} \geq 40 \tag{5.9}$$

The generator computes it as:

Listing 5.2: Rule of 40 computation in 01_generate_saas_data.py.

```
annualized_growth = (mrr / mrr_prev - 1) * 12
rule_of_40 = annualized_growth + gross_margin   # both as ratios
rule_of_40_pct = rule_of_40 * 100               # as percentage points
```

A company growing MRR at 2% per month ($\approx$ 27% annualised) with 78% gross margin scores $27 + 78 = 105$, well above 40 – the margin subsidises modest growth. A company growing 50% but at 0% margin still scores 50, which passes the test.

5.4 Anomaly Detection

Source files: backend/kpi_backend/analytics_engine.py, backend/kpi_backend/routers/anomalies
notebooks/03_anomaly_detection.ipynb

5.4.1 Z-Score Method

The z-score measures how many standard deviations a value lies from the global mean of the series:

$$z_i = \frac{x_i - \bar{x}}{\sigma} \tag{5.10}$$

The implementation in `detect_anomalies_zscore()` scans any series and flags values where $|z_i| \geq \theta$, with $\theta = 2.0$ as the default threshold:

Listing 5.3: Z-score detection in analytics_engine.py.

```
def detect_anomalies_zscore(series, index_labels=None, threshold=2.0):
    if series.empty or series.std() == 0:
        return []
    mean = series.mean()
    std = series.std()
    zscores = (series - mean) / std
    anomalies = []
    for i, z in enumerate(zscores):
        if abs(z) >= threshold:
            anomalies.append({
                "month": str(index_labels.iloc[i]),
                "value": round(float(series.iloc[i]), 4),
                "zscore": round(float(z), 4),
                "severity": anomaly_severity(z),
            })
    return anomalies
```

5.4.2 IQR Method

The IQR (Interquartile Range) method defines fences using the data's quartiles:

$$\text{IQR} = Q_3 - Q_1 \tag{5.11}$$

$$\text{Lower fence} = Q_1 - k \cdot \text{IQR} \tag{5.12}$$

$$\text{Upper fence} = Q_3 + k \cdot \text{IQR} \tag{5.13}$$

with $k = 1.5$ (standard Tukey fences). The IQR method is robust to outliers because quartiles resist extreme values – a single catastrophic month does not inflate Q_1 or Q_3 the way it inflates $\bar{x}$ and σ. The implementation computes an approximate z-score for detected outliers to maintain a consistent severity classification across both methods.

5.4.3 Dual Method Strategy

The `/api/v1/anomalies` endpoint accepts a `method` parameter accepting `zscore`, `iqr`, or `both`. When both methods run, the router deduplicates by (metric, month) key, keeping the higher severity classification:

Listing 5.4: Deduplication logic in anomalies router.

```
seen = set()
unique_anomalies = []
for a in all_anomalies:
    key = (a["metric"], a["month"])
    if key not in seen:
        seen.add(key)
        unique_anomalies.append(a)
    else:
        for existing in unique_anomalies:
            if existing["metric"] == a["metric"] and \
               existing["month"] == a["month"]:
                sev_order = {"critical": 3, "warning": 2, "info": 1}
                if sev_order.get(a["severity"], 0) > \
                   sev_order.get(existing["severity"], 0):
                    existing.update(a)
                break
```

The rationale for running both methods is agreement analysis: when both z-score and IQR flag the same (metric, month) pair, confidence in the anomaly is higher. The notebook analysis showed approximately 80% overlap at default thresholds, with the union catching known events that either method alone sometimes missed at tight thresholds.

5.4.4 Severity Classification

The `anomaly_severity()` function maps z-score magnitude to a three-tier system:

Table 5.2: Anomaly severity thresholds.

Severity	Condition	Interpretation
`info`	$1.0 \leq \|z\| < 2.0$	Notable deviation; monitor
`warning`	$2.0 \leq \|z\| < 3.0$	Unusual; investigate
`critical`	$\|z\| \geq 3.0$	Extreme; escalate immediately

The July 2024 churn spike (pricing change) produces a z-score of approximately +2.3 for logo churn rate, correctly classified as `warning`. The support ticket spike in the same period may reach $|z| > 3$, triggering a `critical` alert.

5.4.5 Metrics Scanned

The anomaly router scans twelve metrics: `mrr`, `arr`, `nrr`, `logo_churn_rate`, `revenue_churn_rate`, `nps`, `ltv_cac_ratio`, `rule_of_40`, `gross_margin`, `dau_mau_ratio`, `cac`, and `payback_months`. The broader scan catches positive anomalies (growth accelerations) as well as negative ones – the Q2 2025 product launch registers as a positive anomaly in new MRR and expansion MRR.

Key Insight

Z-score uses the global 24-month mean and standard deviation. The KPI pipeline (`data-pipeline/02_compute_kpis.py`) also pre-computes *rolling* 6-month z-scores (`zscore_{metric}` columns). Rolling z-scores adapt to recent trends: a value that was anomalous in month 3 may appear normal by month 18 if the series has shifted. The API uses global z-scores for absolute benchmarking; rolling z-scores are available in the pre-computed parquets for trend-adjusted alerting.

5.5 Forecasting with Holt-Winters

Source files: `backend/kpi_backend/analytics_engine.py`, `backend/kpi_backend/routers/forecast.py`, `notebooks/04_forecasting.ipynb`

5.5.1 Model Selection

With 24 months of data, the viable forecasting options are limited. ARIMA with seasonal components needs at least 2–3 full seasonal cycles. Prophet works but adds a large dependency for marginal accuracy gain on short series. The choice is **Holt-Winters exponential smoothing with additive trend and no seasonal component**.

The seasonal component is omitted because:

1. Only 24 months = 2 seasonal cycles, insufficient for reliable seasonal parameter estimation.
2. MRR is a cumulative stock variable – its month-to-month movements are small relative to its level, so seasonality in increments is weak.

The model equation for h-step-ahead forecast:

$$\hat{y}_{t+h} = l_t + h \cdot b_t \tag{5.14}$$

where the level l_t and trend b_t are updated at each period:

$$l_t = \alpha \cdot y_t + (1 - \alpha)(l_{t-1} + b_{t-1}) \tag{5.15}$$

$$b_t = \beta(l_t - l_{t-1}) + (1 - \beta)b_{t-1} \tag{5.16}$$

Parameters α (level smoothing) and β (trend smoothing) are optimised by maximum likelihood via `statsmodels` when available. The implementation in `exponential_smoothing_forecas` falls back to simple exponential smoothing (no trend, $l_t = \alpha y_t + (1 - \alpha)l_{t-1}$) when statsmodels is absent or when the series has fewer than 3 observations.

5.5.2 Confidence Interval Construction

Prediction intervals are approximated from the residual standard error (RSE) with a horizon-dependent widening factor:

$$w_i = \text{RSE} \times 1.96 \times \sqrt{1 + 0.1 \cdot i} \tag{5.17}$$

$$\text{CI}_i^{\pm} = \hat{y}_{t+i} \pm w_i \tag{5.18}$$

where $i \in \{0, 1, \ldots, h-1\}$ is the step index. The $\sqrt{1 + 0.1i}$ factor widens the interval as the horizon increases, reflecting greater uncertainty further into the future.

Listing 5.5: CI construction in analytics_engine.py.

```
residuals = values - fitted
rse = float(np.std(residuals))
ci_lower, ci_upper = [], []
for i in range(periods):
    width = rse * 1.96 * np.sqrt(1 + i * 0.1)
    ci_lower.append(round(float(fcast[i] - width), 2))
    ci_upper.append(round(float(fcast[i] + width), 2))
```

5.5.3 Fallback Strategy

If `statsmodels` raises any exception (convergence failure, degenerate series, too-short input), the engine falls back to manual simple exponential smoothing with the supplied α. The fallback produces flat (no-trend) forecasts, which is conservative but never crashes. This defensive pattern is appropriate for production APIs where a broken forecast is worse than a simple one.

5.5.4 Forecast Accuracy (Cross-Validated)

Notebook 04 runs rolling-origin cross-validation with a minimum of 12 training months and maximum 3-step-ahead forecasts. Typical cross-validated MAPE values are:

Table 5.3: Holt-Winters cross-validated MAPE by metric and horizon.

Metric	+1 Month MAPE	+3 Month MAPE
MRR	$\approx$ 1–2%	$\approx$ 3–5%
Logo Churn Rate	$\approx$ 5–15%	$\approx$ 10–25%
NPS	$\approx$ 3–8%	$\approx$ 5–12%

MRR is the most forecastable because its strong trend dominates over noise. Logo churn is the hardest: the pricing change spike creates large residuals that inflate RSE and widen confidence intervals. The production API forecasts six metrics: `mrr`, `logo_churn_rate`, `nps`, `arr`, `nrr`, and `cac`, all using `exponential_smoothing_forecast()` with `periods=6`.

5.6 Health Score Composite

Source files: `data-pipeline/02_compute_kpis.py`, `backend/kpi_backend/analytics_engine.py`

The health score condenses the six most diagnostic SaaS metrics into a single 0–100 number displayed as an animated SVG ring gauge in the dashboard. Two implementations exist: a vectorised pipeline version in `02_compute_kpis.py` and a row-level runtime version in `analytics_engine.compute_health_score()`.

5.6.1 Six-Metric Weighted Composite

$$H = \sum_{k=1}^{6} w_k \cdot s_k \in [0, 100] \tag{5.19}$$

where $\sum w_k = 1$ and each component score s_k is a linear scaling of the raw metric value between a "bad" threshold (score = 0) and a "good" threshold (score = 100):

$$s(\text{metric}) = \frac{\text{metric} - \text{bad}}{\text{good} - \text{bad}} \times 100, \quad \text{clamped to } [0, 100] \tag{5.20}$$

For lower-is-better metrics (churn), the scaling is inverted. Table 5.4 shows the component definitions.

Table 5.4: Health score components, weights, and scaling thresholds.

Component	Metric	Weight	Good	Bad
MRR Growth Score	MoM MRR growth	20%	$\geq 5\%$/mo	$\leq 0\%$/mo
NRR Score	Net Revenue Retention	20%	$\geq 115\%$	$\leq 95\%$
Churn Score	Logo churn (inverted)	15%	$\leq 2\%$/mo	$\geq 5\%$/mo
NPS Score	NPS	15%	≥ 55	≤ 25
Rule of 40 Score	Rule of 40 (%)	15%	≥ 40	≤ 15
LTV:CAC Score	LTV:CAC ratio	15%	$\geq 4.0\times$	$\leq 1.5\times$
Total		100%		

5.6.2 Traffic Light Classification

The health score maps to a three-level traffic light used for colour coding throughout the UI and PDF:

- **Green**: $H \geq 75$ – company is healthy.
- **Yellow**: $50 \leq H < 75$ – attention required.
- **Red**: $H < 50$ – urgent action needed.

The per-metric traffic lights (for KPI cards) use a target comparison rather than fixed thresholds. For higher-is-better metrics: green if value/target ≥ 1.0, yellow if ≥ 0.90, red otherwise. For lower-is-better metrics (churn), the ratio is inverted – the `traffic_light_status()` function in `analytics_engine.py` handles both directions with a `higher_is_better` flag.

Decision Justification

Why 20% weight on both MRR growth and NRR? SaaS companies fail in two distinct ways: they stop acquiring new customers (MRR growth collapses) or they lose existing ones faster than they can replace them (NRR falls below 100%). Weighting both equally ensures neither failure mode is masked by the other. Churn, NPS, Rule of 40, and LTV:CAC each take 15% as secondary signals that corroborate the primary revenue story.

5.7 PDF Report Generation

Source files: `backend/kpi_backend/report_generator.py`, `backend/kpi_backend/commentary.py`, `backend/kpi_backend/routers/report.py`

5.7.1 fpdf2 and Kaleido

The report is generated as a PDF byte stream using `fpdf2`, a pure-Python PDF library that requires no external binaries. Plotly charts are rasterised to PNG via `kaleido`

(Plotly's headless renderer) and embedded as images.

The `_render_chart_png()` helper writes a Plotly figure to a temporary file at 700×350 pixels at 2× scale (1400×700 effective resolution). Temporary files are tracked in a list and deleted in the `finally` block of `generate_report()` regardless of whether generation succeeds or fails, preventing disk accumulation.

5.7.2 PDF Structure

The `_KPIReport` class subclasses `FPDF` to inject a consistent header (report title + date on every page) and footer (bilingual page number). The `generate_report()` function accepts a `sections` list so the user can request a partial report:

Table 5.5: PDF report sections and their default inclusion.

Section ID	Content	Default
`cover`	Title page with period info	Yes
`executive_summary`	3–5 sentence narrative + health score	Yes
`kpi_dashboard`	Colour-coded KPI table (7 columns)	Yes
`revenue_analysis`	Revenue commentary + waterfall chart	Yes
`customer_health`	Customer narrative (NPS, churn, tickets)	Yes
`anomaly_alerts`	Anomaly narrative + flagged items table	Yes
`forecast`	Per-metric forecast tables with CI	Yes
`recommendations`	Data-driven action items	Yes

The ReportPanel component in the Next.js dashboard lets users select a subset of sections via checkboxes, choose the output language, then fetch `/api/kpi/api/v1/report/generate`. The browser receives a blob and creates a temporary download link using `window.URL.createObjectURL(`

5.7.3 Bilingual NLG Commentary

The `commentary.py` module provides four generators: `generate_executive_summary()`, `generate_revenue_commentary()`, `generate_customer_commentary()`, and `generate_anomaly_narr` All accept `lang='en'` or `lang='es'`.

Commentary is built from f-string templates with conditional branches based on metric thresholds. The executive summary selects from three tone descriptors based on health score, then composes sentences for MRR direction, churn status, and NPS standing:

Listing 5.6: Bilingual tone selection in commentary.py.

```
if health >= 75:
    tone_en = "strong"
    tone_es = "solida"
elif health >= 50:
```

```
    tone_en = "moderate"
    tone_es = "moderada"
else:
    tone_en = "concerning"
    tone_es = "preocupante"
```

The anomaly narrative generator lists up to three critical anomalies by (metric, month) and reports warning counts as a secondary line. For no anomalies, it returns a clean bill-of-health message in the appropriate language.

Key Insight

Template-based NLG is the right choice for business metrics commentary. LLM-generated commentary would be more fluent but less controllable – the template guarantees that specific thresholds ("churn exceeded 5%") always map to specific language ("flagging potential retention issues"). For a report that executives sign off on and share with investors, predictability matters more than prose quality.

5.8 System Architecture

Source files: `backend/kpi_backend/main.py`, `backend/kpi_backend/data_loader.py`, `src/hooks/useKP`

5.8.1 Data Pipeline

The data pipeline is deliberately simple and linear. There are no orchestration tools, no databases, and no message queues. The flow is:

1. `data-pipeline/01_generate_saas_data.py` produces `monthly_metrics.parquet` and `segment_metrics.parquet` (24 rows × 30 columns each).
2. `data-pipeline/02_compute_kpis.py` reads both raw parquets, computes all derived columns (MoM/YoY changes, rolling 3m/6m averages, rolling z-scores, traffic lights, health scores) and writes `monthly_kpis.parquet` and `segment_kpis.parquet` (≈100+ columns).

The backend `data_loader.py` loads all four parquets at module import time into module-level DataFrames. Every API request reads from in-memory DataFrames rather than disk, giving sub-millisecond data access at the cost of a few MB of memory for 24 rows.

5.8.2 FastAPI Backend Routes

The `main.py` application mounts six routers under the `/api/v1` prefix:

Table 5.6: API endpoints.

Path	File	Returns
GET /api/v1/overview	routers/overview.py	Health score + KPI cards + commentary
GET /api/v1/revenue	routers/revenue.py	Waterfall, ARR trend, NRR, segments
GET /api/v1/customers	routers/customers.py	Churn trend, NPS, Lorenz curve
GET /api/v1/forecast	routers/forecast.py	Per-metric ES forecasts + CI
GET /api/v1/anomalies	routers/anomalies.py	Flagged anomaly list + summary
GET /api/v1/report/generate	routers/report.py	PDF byte stream

All GET endpoints accept optional query parameters: `segment` (filter by Starter/Professional/Enterprise), `start_month`/`end_month` (YYYY-MM range), and `lang` (en/es). The `apply_filters()` utility in `data_loader.py` handles all filtering uniformly against pandas Timestamp comparisons.

5.8.3 Next.js Frontend Architecture

The frontend uses Next.js with the App Router. The dashboard lives at `src/app/kpi/page.tsx` and renders `<KPIDashboard/>`, which nests two context providers: `KPIFilterProvider` (segment + date range state) and `LanguageProvider` (en/es toggle).

API calls are centralised in `src/hooks/useKPIAPI.ts` using SWR (stale-while-revalidate):

Listing 5.7: SWR hooks for KPI API in useKPIAPI.ts.

```
export function useOverview(qs: string) {
  return useSWR<OverviewResponse>(
    `/api/v1/overview${qs}`, kpiFetcher, swrConfig)
}
export function useAnomalies(qs: string) {
  return useSWR<AnomalyResponse>(
    `/api/v1/anomalies${qs}`, kpiFetcher, swrConfig)
}
```

The `KPIFilterContext` exposes a `queryString` computed property that all panels pass to their SWR hooks, ensuring all panels stay synchronised when filters change. The `LanguageContext` wraps the entire dashboard with a `t(key)` translation function backed by `src/lib/translations.ts`, supporting bilingual UI display without a runtime i18n library dependency.

The eight navigation tabs (About, Overview, Revenue, Customers, Forecast, Anomalies, Report, Technical Process) map directly to eight panel components, all lazy-loaded on first activation.

5.9 Challenge and Interview Questions

Challenge Question

NRR arithmetic. A SaaS company starts January with MRR of \$500,000. During the month, new logos add \$40,000, existing customers expand by \$25,000, some customers downgrade (contraction) by \$8,000, and churned customers account for \$18,000 in lost MRR. What is the ending MRR? What is the monthly NRR? What is the annualised NRR?

Solution: Ending MRR $= 500{,}000 + 40{,}000 + 25{,}000 - 8{,}000 - 18{,}000 = \$539{,}000$. Monthly NRR $= (500{,}000 + 25{,}000 - 8{,}000 - 18{,}000)/500{,}000 = 499{,}000/500{,}000 = 0.998$ (99.8%). Annualised NRR $= 0.998^{12} \approx 97.6\%$. Even though the company grew total MRR by 7.8% (thanks to new logos), the existing base is contracting – NRR $< 100\%$ is a warning sign that the company would shrink without new acquisition.

Challenge Question

Health score computation. Given the weight table (Table 5.4), compute the health score for a company with: MoM MRR growth = 3%, NRR = 105%, logo churn = 3%, NPS = 42, Rule of 40 = 35, LTV:CAC = 2.8×. Which component contributes the least?

Hint: Scale each metric linearly between its bad and good thresholds, multiply by its weight, and sum. The MRR growth score is $(0.03-0.00)/(0.05-0.00)\times 100 = 60$, contributing $60\times 0.20 = 12$ points. NRR score: $(1.05-0.95)/(1.15-0.95)\times 100 = 50$, contributing 10 points. Churn score (inverted): $(0.05 - 0.03)/(0.05 - 0.02) \times 100 = 66.7$, contributing 10.0 points. NPS: $(42 - 25)/(55 - 25) \times 100 = 56.7$, contributing 8.5 points. Rule of 40: $(35 - 15)/(40 - 15) \times 100 = 80$, contributing 12 points. LTV:CAC: $(2.8 - 1.5)/(4.0 - 1.5) \times 100 = 52$, contributing 7.8 points. Total $H \approx 60.3$, status: yellow. NPS contributes least.

Interview Question

Why use both z-score and IQR? "In your anomaly detection system you run both methods. What does each one contribute and when would you drop one of them?"

Good answer: Z-score is fast to interpret (the number directly communicates severity) and easy to threshold, but it assumes approximate normality and is sensitive to outliers – a single extreme value inflates σ and can mask other anomalies. IQR is distribution-free and robust, but gives less granular severity information and may miss asymmetric anomalies. Running both and taking the union gives higher recall; deduplicating and keeping the higher severity prevents alert fatigue. Drop IQR if the series is known to be approximately normal and you need consistent z-score-based severity levels in reports. Drop z-score if the series has heavy tails (e.g. support ticket volumes during incident windows) where quartile-based fences are more stable.

Interview Question

Why not ARIMA or Prophet for forecasting? "You chose Holt-Winters with no seasonal component. Wouldn't ARIMA or Prophet give better forecasts?"

Good answer: With only 24 data points, ARIMA with seasonal components requires estimating too many parameters for reliable cross-validation. Prophet is designed for series with hundreds of observations and strong seasonality patterns. Holt-Winters with additive trend captures the dominant signal (upward trend) with just two parameters (α, β) that statsmodels optimises via MLE. Cross-validated MAPE of 1–2% at 1-month horizon is sufficient for board-level planning; the confidence interval construction is transparent and explainable to non-technical stakeholders. Adding complexity for marginal accuracy gain on 24 data points is a bad trade-off.

Challenge Question

Rule of 40 stress test. NovaCRM reports annualised MRR growth of 18% and gross margin of 78%, giving Rule of 40 = 96. A competitor reports Rule of 40 = 42 with 2% growth and 40% gross margin. Which company would you rather invest in, and why does the Rule of 40 fail to distinguish them?

Hint: The Rule of 40 is a heuristic that collapses two dimensions into one number. A high-margin, slow-growth company and a low-margin, high-growth company can produce identical scores. The relevant question is whether the growth is durable and whether the margin can expand – these require LTV:CAC, NRR, and churn analysis, not just the Rule of 40. In this case NovaCRM's 78% gross margin suggests a scalable SaaS business; the competitor's 40% margin may indicate high infrastructure or customer success costs that will constrain profitability at scale.

Challenge Question

Confidence interval interpretation. The Holt-Winters forecast for MRR at horizon $h = 3$ returns a point forecast of \$850,000 with CI (\$810,000, \$890,000). An executive asks: "Does this mean there is a 95% chance MRR will be between \$810K and \$890K in three months?" How do you answer correctly?

Answer: No. A 95% CI computed from historical residuals is a frequentist statement: if the same model were used repeatedly on similar data, 95% of such intervals would contain the true value. It is not a probability statement about this specific interval. More practically: the CI does not account for structural breaks (like a pricing change), model misspecification, or the small-sample nature of 24 observations. The interval should be interpreted as a plausible range under business-as-usual conditions, not as a probabilistic guarantee. For a board presentation, framing it as "our model projects \$850K, with a reasonable range of \$810K–\$890K absent any significant business changes" is more accurate than quoting the 95% probability.

Interview Question

Bilingual PDF architecture. "How would you scale this system to support ten languages instead of two?"

Good answer: The current template-based NLG in `commentary.py` has one `if lang == 'es'` branch per function. Scaling to ten languages requires a different pattern: externalise all string templates into a translation dictionary keyed by `lang` (like the frontend's `translations.ts`), and parameterise the commentary functions to select the right template. String interpolation with named variables – e.g. `templates[lang]['mrr_growth'].format(pct=..., value=...)` – scales to any number of languages without code duplication. For the PDF, the font choice must be verified for character support – fpdf2's built-in Helvetica only covers Latin characters, so Cyrillic or CJK languages would require loading a custom TTF font via `pdf.add_font()`.

Challenge Question

Lorenz curve and revenue concentration. The `customer_concentration_lorenz()` function in `analytics_engine.py` sorts customer segments by revenue-per-customer and computes a Gini coefficient via trapezoidal integration. Given three segments with revenues \$100K, \$180K, \$220K and customer counts 330, 45, 15 respectively, which segment appears first in the Lorenz sort order and what does the resulting Gini coefficient imply?

Solution: Rev/customer: Starter (\$100K/330 $\approx$ \$303), Professional (\$180K/45 = \$4,000), Enterprise (\$220K/15 $\approx$ \$14,667). Lorenz sort (ascending): Starter, Professional, Enterprise. The 15 Enterprise customers represent 3.8% of the 390-customer base but contribute 44% of revenue. The curve bows sharply below the 45-degree equality line, yielding a high Gini coefficient (likely > 0.50). This high concentration means Enterprise churn is catastrophically risky – losing one Enterprise account can equal losing 50 Starter accounts in revenue terms.

Chapter 6

Financial Portfolio Tracker

Source files: `projects/05-financial-portfolio-tracker/backend/portfolio_backend/portfolio_en`
`backend/portfolio_backend/data_loader.py`, `backend/portfolio_backend/routers/{overview,`
`performance, risk, montecarlo, frontier, correlation}.py`, `src/components/portfolio/Portfol`

6.1 Business Context

The central question driving this project is: *How is a diversified, multi-asset portfolio performing relative to its benchmark, what is the underlying risk-return profile, and which allocations are sub-optimal?*

This question is not academic. An investor holding ETFs across US equities, international equities, emerging markets, fixed income, real estate, and commodities needs a unified analytical view that goes beyond a brokerage's total-return figure. The specific decisions supported are:

- **Rebalancing trigger**: Has drift in weights pushed the portfolio away from its target allocation?
- **Risk-adjusted evaluation**: Is the portfolio compensating sufficiently for its volatility relative to the risk-free rate?
- **Forward-looking uncertainty**: Given historical return distribution, what is the probability of reaching a target value over the next year?
- **Frontier optimality**: Is the current allocation on the efficient frontier, or does mean-variance theory suggest a Pareto-superior combination?

The default portfolio used throughout this project consists of six Vanguard and SPDR ETFs selected to span major asset classes:

Ticker	Name	Category	Weight
VOO	Vanguard S&P 500 ETF	US Equity	30%
VXUS	Vanguard Total Intl Stock ETF	International Equity	20%
VWO	Vanguard FTSE Emerging Markets	Emerging Markets	10%
BND	Vanguard Total Bond Market ETF	Fixed Income	20%
VNQ	Vanguard Real Estate ETF	Real Estate	10%
GLD	SPDR Gold Shares	Commodities	10%
SPY	SPDR S&P 500 ETF	Benchmark	—

Table 6.1: Default portfolio configuration (`config.py`, lines 5–12). The benchmark SPY is fetched alongside portfolio assets and used exclusively for relative metrics.

Key Insight

The portfolio is intentionally a classic "lazy portfolio" construction – low-cost passive ETFs diversified across asset classes. This makes it a realistic reference portfolio while keeping the analytical focus on the *methodology* rather than stock selection. The actuarial parallel is clear: an insurer holding a fixed-income portfolio faces the same risk-return trade-off questions at an asset-liability management level.

6.2 Data Acquisition and Caching

Source files: `backend/portfolio_backend/data_loader.py`, `backend/portfolio_backend/config.py`

6.2.1 yfinance vs. Alpha Vantage

Market data is fetched via `yfinance`, Yahoo Finance's unofficial Python wrapper. The original README mentions both yfinance and Alpha Vantage; the implementation settled on yfinance for three reasons:

Decision Justification

Why yfinance over Alpha Vantage?

1. **No API key**: yfinance requires no registration, making the project immediately reproducible for anyone cloning the repo.
2. **Adjusted closes**: `yf.download(..., auto_adjust=True)` returns split- and dividend-adjusted closing prices, which is the correct input for return calculations. Alpha Vantage's free tier requires manual dividend adjustments.
3. **Rate limits**: Alpha Vantage's free tier allows 25 calls per day. A 6-asset portfolio with a benchmark requires 7 simultaneous downloads – easily exhausted during development.

The trade-off is reliability: Yahoo Finance is an unofficial API and can break on schema changes. The cache layer (§6.2.2) absorbs transient failures gracefully.

6.2.2 The Parquet Cache Layer

The data loader implements a file-based cache that persists price DataFrames as Parquet files. The core logic lives in `data_loader.py`.

Listing 6.1: Cache freshness check – `data_loader.py` lines 33–43

```
1 def _cache_is_fresh(tickers: list[str], period: str) -> bool:
2     meta = _meta_path(tickers, period)
3     if not meta.exists():
4         return False
5     ts = dt.datetime.fromisoformat(meta.read_text().strip())
6     age = dt.datetime.now() - ts
7     return age.total_seconds() < CACHE_TTL_HOURS * 3600
```

`CACHE_TTL_HOURS = 4` means a fresh request hitting the API triggers one download, and all subsequent requests within four hours are served from `data/cache/<tickers>_<period>.parq`. This design matters in production: Cloud Run instances are stateless, but the cache directory survives for the duration of a container's life. During development, it eliminates repeated API calls when iterating on analysis code.

The cache key is derived by sorting the tickers alphabetically and appending the period string (e.g., `BND_GLD_SPY_VNQ_VOO_VWO_VXUS_5y.parquet`). Sorting ensures that two calls with the same tickers in different order hit the same cache file.

6.2.3 Return Type: Simple vs. Arithmetic

The data loader computes daily returns using `pct_change()` – simple arithmetic returns:

$$R_t = \frac{P_t - P_{t-1}}{P_{t-1}} \tag{6.1}$$

A comment in `data_loader.py:147` is explicit: *"Daily simple returns (arithmetic, not log – standard for portfolio math."*

This is a deliberate choice. While log returns $r_t = \ln(P_t/P_{t-1})$ have nicer statistical properties (time-additive, approximately normal), **portfolio aggregation requires arithmetic returns**:

$$R_{p,t} = \sum_{i=1}^{N} w_i \, R_{i,t} \tag{6.2}$$

This linear weighting identity holds exactly for arithmetic returns. For log returns, the equivalent expression requires a covariance correction term and does not simplify to a dot product. Since the fundamental operation in `data_loader.py` line 161 is `returns.multiply(weights_series).sum(axis=1)`, arithmetic returns are the mathematically correct choice.

6.3 Return Calculations

Source files: `portfolio_engine.py` lines 20–117

6.3.1 Cumulative Returns and the Wealth Index

Starting from a series of daily arithmetic returns $R_1, R_2, \ldots, R_T$, cumulative returns are computed via the compound product:

$$W_T = \prod_{t=1}^{T}(1 + R_t) - 1 \tag{6.3}$$

In code, this is `(1 + returns).cumprod() - 1` (`portfolio_engine.py:22`). The intermediate series $\{W_t\}$ is the *wealth index*: a dollar invested at $t = 0$ grows to $1 + W_T$ by day T.

6.3.2 Annualized Return (CAGR)

The Compound Annual Growth Rate converts the total holding-period return into an equivalent constant annual rate:

$$\text{CAGR} = \left(\prod_{t=1}^{T}(1 + R_t)\right)^{1/n_{\text{years}}} - 1 \tag{6.4}$$

where $n_{\text{years}} = T/252$ and T is the number of trading days. The implementation (`portfolio_engine.py:31-38`) guards against edge cases: empty returns, non-positive total product (an asset with a 100% loss), and zero holding period.

Interview Question

Q: Why divide by 252 and not 365?
Financial returns are conventionally annualized using trading days (252 per year in US markets) rather than calendar days. This convention preserves consistency: daily volatility is scaled by $\sqrt{252}$, and CAGR divides by $T/252$. If you used 365, a portfolio that was flat on weekends would appear to have a lower CAGR than one measured only on trading days. The 252-day year is a market standard, not a mathematically derived constant.

6.3.3 Rolling Returns

Rolling window returns answer the question: "What was the total return over the past k trading days, measured at every point in time?"

$$R_t^{(k)} = \prod_{s=t-k+1}^{t} (1 + R_s) - 1 \tag{6.5}$$

Implemented as `(1 + returns).rolling(window).apply(np.prod, raw=True) - 1` (`portfolio_engine.py:27`). The performance endpoint exposes 30-day, 90-day, and 252-day windows, letting the viewer identify seasonality and momentum patterns that cumulative return charts obscure.

6.3.4 Calendar Returns Matrix

The calendar return matrix (rows = years, columns = months 1–12, plus a YTD column) is a standard in portfolio reporting. Each cell is:

$$R_{\text{year,month}} = \prod_{t \in (\text{year,month})} (1 + R_t) - 1 \tag{6.6}$$

The implementation uses a two-level `groupby` on `[idx.year, idx.month]` followed by `.unstack(level="month")` (`portfolio_engine.py:104-117`). This produces a DataFrame that the frontend renders as a heatmap, immediately identifying which months and years drove performance.

6.4 Risk Metrics

Source files: `portfolio_engine.py` lines 120–228, `routers/risk.py`

6.4.1 Annualized Volatility

Daily return volatility is the sample standard deviation scaled to annual frequency:

$$\sigma_{\text{ann}} = \hat{\sigma}_{\text{daily}} \times \sqrt{252} \tag{6.7}$$

The $\sqrt{252}$ scaling follows from the assumption that daily returns are independent and identically distributed. If $\text{Var}(R_t) = \sigma^2$, then for a 252-day period the variance is $252\sigma^2$, giving standard deviation $\sigma\sqrt{252}$.

Challenge Question

When does the $\sqrt{T}$ scaling break down?
The $\sqrt{T}$ rule assumes i.i.d. returns. Empirically, financial returns exhibit *volatility clustering* (large moves follow large moves – GARCH dynamics) and *fat tails* (excess kurtosis > 3). Both violate the i.i.d. Gaussian assumption. GARCH models capture the first phenomenon; extreme value theory addresses the second. For a portfolio analytics tool aimed at retail investors, the i.i.d. simplification is acceptable. For actuarial solvency capital calculations (e.g., Solvency II internal models), it is not.

6.4.2 Sharpe Ratio

The Sharpe ratio [**?**] measures excess return per unit of total volatility:

$$\text{Sharpe} = \frac{\bar{R}_p - r_f}{\sigma_p} \tag{6.8}$$

where $\bar{R}_p$ is the annualized portfolio return, r_f is the risk-free rate (set to 4.5% in `config.py`, approximately the US 3-month T-bill rate at the time of implementation), and σ_p is the annualized volatility.

A Sharpe ratio above 1.0 is generally considered good; above 2.0 is excellent and often indicates overfitting in backtests. The ratio is dimensionless, enabling comparison across portfolios with different return scales.

6.4.3 Sortino Ratio

The Sortino ratio modifies the Sharpe denominator to use only *downside* volatility – the root mean squared deviation of excess returns that are negative:

$$\sigma_{\text{down}} = \sqrt{\frac{1}{n} \sum_{t:\, R_t < r_f/252} \left(R_t - \frac{r_f}{252}\right)^2} \times \sqrt{252} \tag{6.9}$$

$$\text{Sortino} = \frac{\bar{R}_p - r_f}{\sigma_{\text{down}}} \tag{6.10}$$

The implementation (`portfolio_engine.py:133-143`) first subtracts the daily risk-free rate from each observation, then keeps only the negative residuals for the RMS calculation. This is the *semi-deviation* approach. The Sortino ratio is always $\geq$ the Sharpe ratio when the return is positive, because upside variability is excluded from the denominator.

Key Insight

Sortino is more appropriate than Sharpe when the return distribution is *positively skewed* (more large positive days than large negative days). For a long-only equity portfolio, which experiences rare large crashes but frequent small gains, Sortino better captures what investors actually care about: the risk of loss, not symmetric volatility.

6.4.4 Calmar Ratio

The Calmar ratio relates annualized return to the magnitude of the worst peak-to-trough decline:

$$\text{Calmar} = \frac{\text{CAGR}}{|\text{Max Drawdown}|} \tag{6.11}$$

(`portfolio_engine.py:146-151`). The Calmar is popular in hedge fund evaluation because drawdown is a more intuitive risk concept than volatility. A fund with 15% CAGR and 30% max drawdown has Calmar = 0.5; a fund with 10% CAGR and 10% max drawdown has Calmar = 1.0 and is arguably better risk-adjusted.

6.4.5 Maximum Drawdown

Maximum drawdown measures the largest peak-to-trough decline in the portfolio's wealth index:

$$\text{MDD} = \min_t \left(\frac{W_t}{\max_{s \leq t} W_s} - 1 \right) \tag{6.12}$$

The implementation (`portfolio_engine.py:48-82`) constructs the wealth index, computes the running maximum, and identifies the trough and recovery dates. A result of MDD = -0.35 means the portfolio fell 35% from its peak before recovering. The function returns four values: `max_dd`, `peak_date`, `trough_date`, and `recovery_date` (or `None` if recovery had not occurred by the end of the data window).

6.4.6 Beta and Jensen's Alpha

Beta

Portfolio beta measures the sensitivity of portfolio returns to benchmark returns:

$$\beta = \frac{\text{Cov}(R_p, R_m)}{\text{Var}(R_m)} \tag{6.13}$$

This is the OLS slope coefficient from regressing portfolio returns on benchmark returns. The implementation (`portfolio_engine.py:154-163`) uses `np.cov` to compute the 2×2 covariance matrix and extracts the off-diagonal element divided by the benchmark variance. $\beta < 1$ indicates the portfolio amplifies benchmark moves by less than 1-for-1; $\beta > 1$ means it is more aggressive. For a diversified multi-asset portfolio including bonds and gold, $\beta < 1$ relative to SPY is expected.

Jensen's Alpha

Alpha measures the risk-adjusted return above what the CAPM would predict:

$$\alpha = \bar{R}_p - \left[r_f + \beta \cdot (\bar{R}_m - r_f)\right] \tag{6.14}$$

(`portfolio_engine.py:166-175`). A positive alpha indicates the portfolio outperformed the benchmark after adjusting for its market exposure. Note that alpha is computed on annualized returns, so it represents the annualized excess return attributable to the portfolio's construction rather than to market beta.

Interview Question

Q: What are the assumptions embedded in CAPM, and why might alpha be misleading?
CAPM assumes: (1) investors hold mean-variance efficient portfolios; (2) all investors share the same expectations; (3) a single factor (the market) explains all systematic risk. In practice, the Fama-French three-factor and five-factor models show that size, value, profitability, and investment style explain a significant portion of apparent alpha. A portfolio overweight small-cap value stocks may show positive CAPM alpha not because the manager is skilled, but because it loads on the value and size risk premia. For a passive ETF portfolio like the one here, CAPM alpha is a reasonable first-order benchmark comparison.

6.4.7 Tracking Error and Information Ratio

Tracking error is the annualized standard deviation of the portfolio's daily excess return over the benchmark:

$$\text{TE} = \text{std}(R_p - R_m) \times \sqrt{252} \tag{6.15}$$

The information ratio then measures active return per unit of tracking risk:

$$\text{IR} = \frac{\bar{R}_p - \bar{R}_m}{\text{TE}} \tag{6.16}$$

(`portfolio_engine.py:178-196`). A higher IR indicates that active bets (deviations from the benchmark) are generating proportionally more return. For a passive multi-asset portfolio versus SPY, a negative IR is common: the portfolio deliberately includes asset classes (bonds, gold) that underperform in bull markets, trailing SPY in those periods while offering better drawdown protection.

6.5 Value at Risk

Source files: `portfolio_engine.py` lines 199–242, `routers/risk.py` lines 27–39

Value at Risk at confidence level α is defined as the loss threshold that is not exceeded with probability α:

$$\text{VaR}_\alpha = -\inf\Big\{x : \mathbb{P}(R \le x) > 1 - \alpha\Big\} \tag{6.17}$$

The risk router computes VaR using three distinct methods at both 95% and 99% confidence levels.

6.5.1 Parametric (Gaussian) VaR

Parametric VaR assumes daily returns follow a normal distribution:

$$\text{VaR}_\alpha^{\text{param}} = -\Big(\hat{\mu} + z_{1-\alpha} \cdot \hat{\sigma}\Big) \tag{6.18}$$

where $z_{1-\alpha}$ is the $(1-\alpha)$-quantile of the standard normal. For $\alpha = 0.95$, $z_{0.05} \approx -1.645$, so:

$$\text{VaR}_{0.95} = -(\hat{\mu} - 1.645\hat{\sigma})$$

The implementation uses `scipy.stats.norm.ppf(1 - confidence)` (`portfolio_engine.py:208`). The result is positive by convention – VaR represents a loss magnitude. Parametric VaR is analytically fast but underestimates tail risk when the empirical distribution has fat tails (excess kurtosis > 0).

6.5.2 Historical VaR

Historical VaR makes no distributional assumption:

$$\text{VaR}_{\alpha}^{\text{hist}} = -\text{Percentile}_{(1-\alpha)\times 100}(R_1, \ldots, R_T) \tag{6.19}$$

(`portfolio_engine.py:212-216`). The $(1-\alpha)$-th percentile of the return sample is the empirical quantile. This approach automatically captures fat tails, skewness, and any non-normality present in the historical record. Its weakness is that it implicitly assumes the future will resemble the past – a sample period excluding a major crash (e.g., 2008) will understate true tail risk.

6.5.3 Monte Carlo VaR

The Monte Carlo variant (`portfolio_engine.py:230-242`) simulates 10,000 one-day returns by drawing from $\mathcal{N}(\hat{\mu}, \hat{\sigma}^2)$ and then takes the empirical percentile:

Listing 6.2: Monte Carlo VaR – `portfolio_engine.py` lines 237–242

```
rng = np.random.default_rng(42)
simulated = rng.normal(mu, sigma, n_sims)
return float(-np.percentile(simulated, (1 - confidence) * 100))
```

For a Gaussian input distribution, Monte Carlo VaR converges to parametric VaR as $n_{\text{sims}} \to \infty$. The seed `42` ensures reproducibility across API calls.

6.5.4 CVaR as a Coherent Risk Measure

CVaR – also called Expected Shortfall (ES) – is the conditional expectation of losses beyond the VaR threshold:

$$\text{CVaR}_{\alpha} = \mathbb{E}\Big[-R \mid R \leq -\text{VaR}_{\alpha}\Big] \tag{6.20}$$

(`portfolio_engine.py:219-227`). In words: given that you are in the 5% worst outcomes, what is your expected loss on average? CVaR is always $\geq$ VaR at the same confidence level.

Challenge Question

Why is CVaR preferred over VaR in modern risk management?
VaR is *not a coherent risk measure*: it fails the subadditivity axiom. A coherent risk measure ρ satisfies:

$$\rho(X+Y) \leq \rho(X) + \rho(Y)$$

meaning that combining two portfolios cannot increase risk beyond the sum of their individual risks (diversification should be beneficial). VaR can violate subadditivity: consider two bonds each with a 4% default probability. At 95% confidence, each bond has VaR $= 0$ (no loss in 95% of scenarios). A portfolio of both bonds held 50/50 can have a positive VaR if their joint default probability exceeds 5%, violating the diversification principle.
CVaR satisfies subadditivity and is therefore *coherent* under the Artzner et al. (1999) axioms [**?**]. The Basel III/IV banking regulatory framework has moved from VaR to Expected Shortfall precisely for this reason.

6.6 Monte Carlo GBM Simulation

Source files: `portfolio_engine.py` lines 266–344, `routers/montecarlo.py`

6.6.1 Geometric Brownian Motion

The Monte Carlo simulation models portfolio value evolution as Geometric Brownian Motion (GBM). The SDE is:

$$dS_t = \mu S_t\, dt + \sigma S_t\, dW_t \tag{6.21}$$

where μ is the drift (expected instantaneous return per unit time), σ is the instantaneous volatility, and W_t is a standard Wiener process (Brownian motion). The multiplicative structure of GBM ensures $S_t > 0$ for all $t > 0$ almost surely, which is appropriate for asset prices.

6.6.2 The Ito Drift Correction

To discretize GBM, we apply Ito's lemma to $f(S_t) = \ln S_t$. By Ito's formula for a twice-differentiable function f:

$$df(S_t) = f'(S_t)\, dS_t + \frac{1}{2} f''(S_t)\, (dS_t)^2 \tag{6.22}$$

Substituting $f(S) = \ln S$, $f'(S) = 1/S$, $f''(S) = -1/S^2$, and using $(dW_t)^2 = dt$:

$$\begin{aligned} d(\ln S_t) &= \frac{1}{S_t}\left(\mu S_t\, dt + \sigma S_t\, dW_t\right) + \frac{1}{2}\left(-\frac{1}{S_t^2}\right)\sigma^2 S_t^2\, dt \\ &= \mu\, dt + \sigma\, dW_t - \frac{\sigma^2}{2}\, dt \\ &= \left(\mu - \frac{\sigma^2}{2}\right) dt + \sigma\, dW_t \end{aligned} \tag{6.23}$$

Integrating over one time step $\Delta t = 1$ (one trading day):

$$\ln S_{t+1} - \ln S_t = \left(\mu - \frac{\sigma^2}{2}\right) + \sigma\, Z, \quad Z \sim \mathcal{N}(0,1) \tag{6.24}$$

Exponentiating both sides:

$$S_{t+1} = S_t \cdot \exp\left[\left(\mu - \frac{\sigma^2}{2}\right) + \sigma Z\right] \tag{6.25}$$

The term $\mu - \sigma^2/2$ is the *log-drift* or *Ito correction*. It arises because the arithmetic mean and geometric mean of a lognormal distribution differ by $\sigma^2/2$. The arithmetic expected return satisfies $\mathbb{E}[S_{t+1}/S_t] = e^{\mu}$, while the *median* path satisfies $\ln(S_{t+1}/S_t) = \mu - \sigma^2/2$.

Key Insight

The Ito correction is exact, not approximate. Using μ instead of $\mu - \sigma^2/2$ as the log-drift is a common implementation error that produces paths with an upward bias: the arithmetic average of simulated paths would exceed the empirical mean return by $\sigma^2/2$. For $\sigma = 15\%$ (a typical equity portfolio), this bias equals $0.015^2/2 \cdot 252 \approx 1.1\%$ per year – meaningful over multi-year horizons.
The relationship between arithmetic mean (μ) and geometric mean ($\mu - \sigma^2/2$) is an instance of Jensen's inequality: $\mathbb{E}[\ln X] \leq \ln \mathbb{E}[X]$ for any random variable X.

6.6.3 Implementation: 1,000 Paths with Percentile Envelopes

The implementation (`portfolio_engine.py:292-337`) estimates $\hat{\mu}$ and $\hat{\sigma}$ from the portfolio return series, then generates $n_{\text{sims}} = 1{,}000$ paths over $T = 252$ trading days.

Listing 6.3: GBM vectorized simulation – `portfolio_engine.py` lines 304–319

```
mu = float(returns.mean())
sigma = float(returns.std())
drift = mu - 0.5 * sigma ** 2      # Ito correction

```

```
rng = np.random.default_rng(42)
random_shocks = rng.normal(0, 1, (n_simulations, days))

log_returns = drift + sigma * random_shocks
log_paths = np.cumsum(log_returns, axis=1)
# Prepend zero (starting point at t=0)
log_paths = np.hstack([np.zeros((n_simulations, 1)), log_paths])
paths = initial_value * np.exp(log_paths)
```

The vectorized approach generates an $(n_{\text{sims}} \times T)$ matrix of standard normals in a single call, then computes cumulative log-returns via `np.cumsum` along the time axis. Prepending a column of zeros sets $S_0 = \text{initial\_value}$ for every path. The final `np.exp` converts log-space paths to price paths.

Percentile envelopes are then extracted across the simulation dimension:

Listing 6.4: Percentile extraction – `portfolio_engine.py` lines 325–327

```
for p in [5, 25, 50, 75, 95]:
    percentiles[p] = np.percentile(paths, p, axis=0).tolist()
```

The 50th-percentile path is the median outcome; the 5th–95th band is the 90% confidence envelope for the portfolio's future value. The Monte Carlo endpoint (`routers/montecarlo.py`) additionally computes:

- `prob_profit`: $\hat{\mathbb{P}}(S_T > S_0)$ – fraction of paths that finish above the starting value.
- `prob_target`: $\hat{\mathbb{P}}(S_T \geq \texttt{target})$ – fraction reaching a user-specified dollar target.
- `var_95`/`var_99`: loss at the 5th and 1st percentile of the simulated final distribution.

Interview Question

Q: What does the Monte Carlo simulation assume that may not hold?

1. **Constant parameters**: $\hat{\mu}$ and $\hat{\sigma}$ are estimated from historical returns and held fixed throughout the simulation. In reality, both drift and volatility are time-varying (regime changes, economic cycles, GARCH dynamics).
2. **Gaussian shocks**: Financial returns have fat tails (excess kurtosis > 0). Normal shocks underestimate the probability of extreme outcomes.
3. **Single-series simulation**: The portfolio is modeled as a single time series. It does not separately simulate each asset and aggregate, so it cannot model tail co-movement (all assets crashing together during a crisis).
4. **Independent increments**: GBM has no memory. Serial correlation and volatility clustering are ignored.

These limitations are acceptable for building intuition about forward uncertainty. For actuarial capital modeling (e.g., Solvency II internal models), a correlated multi-asset GBM with time-varying parameters, or a fat-tailed alternative such as a variance-gamma or jump-diffusion process, would be required.

6.7 Efficient Frontier

Source files: `portfolio_engine.py` lines 350–532, `routers/frontier.py`

6.7.1 Mean-Variance Framework

Modern Portfolio Theory [**?**] posits that rational investors prefer portfolios that maximize expected return for a given level of risk, where risk is measured by variance. For a portfolio of N assets with weight vector $\boldsymbol{w} = (w_1, \ldots, w_N)^\top$:

$$\mu_p = \boldsymbol{w}^\top \boldsymbol{\mu} \tag{6.26}$$

$$\sigma_p^2 = \boldsymbol{w}^\top \boldsymbol{\Sigma} \boldsymbol{w} \tag{6.27}$$

where $\boldsymbol{\mu}$ is the vector of expected asset returns and $\boldsymbol{\Sigma}$ is the $N \times N$ covariance matrix. Both are estimated from historical data using their sample counterparts, annualized by multiplying by $T_{\text{ann}} = 252$ (returns scaled linearly, covariance scaled by 252).

6.7.2 Random Portfolio Cloud

The first step in visualizing the frontier generates 5,000 random portfolios by sampling from the probability simplex:

Listing 6.5: Random weight generation – `portfolio_engine.py` lines 376–386

```
rng = np.random.default_rng(42)
for i in range(n_portfolios):
    w = rng.random(n_assets)
    w = w / w.sum()          # project onto simplex
    port_ret = float(np.dot(w, mean_returns))
    port_vol = float(np.sqrt(np.dot(w.T, np.dot(cov_matrix, w))))
    results_sharpe[i] = (port_ret - rf_rate) / port_vol
```

Generating uniform random vectors and renormalizing does not yield a uniform distribution on the simplex, but it densely populates the feasible region and makes the frontier boundary visible as the upper edge of the cloud in the (σ, μ) scatter plot.

6.7.3 Minimum-Variance Portfolio

The minimum-variance portfolio solves:

$$\min_{\boldsymbol{w}} \quad \sqrt{\boldsymbol{w}^\top \boldsymbol{\Sigma} \boldsymbol{w}} \tag{6.28}$$

$$\text{s.t.} \quad \sum_{i=1}^{N} w_i = 1 \tag{6.29}$$

$$w_i \geq 0 \quad \forall i \tag{6.30}$$

(`portfolio_engine.py:407-438`). The long-only constraint (6.30) restricts the solution to positive weights – no short selling. The `SLSQP` (Sequential Least Squares Programming) solver from `scipy.optimize` handles this nonlinear constrained problem. The initial point $\boldsymbol{w}_0 = (1/N, \ldots, 1/N)$ is a feasible interior point, aiding convergence.

6.7.4 Maximum-Sharpe Portfolio (Tangency Portfolio)

The tangency portfolio maximizes the Sharpe ratio:

$$\max_{\boldsymbol{w}} \quad \frac{\boldsymbol{w}^\top \boldsymbol{\mu} - r_f}{\sqrt{\boldsymbol{w}^\top \boldsymbol{\Sigma} \boldsymbol{w}}} \tag{6.31}$$

$$\text{s.t.} \quad \sum_{i=1}^{N} w_i = 1, \quad w_i \geq 0 \tag{6.32}$$

The implementation minimizes the *negative* Sharpe ratio (`portfolio_engine.py:454-459`), re-using the SLSQP solver. This is the portfolio where the Capital Market Line is tangent to the efficient frontier: it achieves the highest excess return per unit of volatility possible given the available assets and constraints.

Decision Justification

Why SLSQP over closed-form solutions?

Without the long-only constraint, the minimum-variance portfolio has a closed-form solution:

$$\boldsymbol{w}^* = \frac{\boldsymbol{\Sigma}^{-1}\mathbf{1}}{\mathbf{1}^\top\boldsymbol{\Sigma}^{-1}\mathbf{1}}$$

and the unconstrained maximum-Sharpe portfolio is:

$$\boldsymbol{w}^* = \frac{\boldsymbol{\Sigma}^{-1}(\boldsymbol{\mu} - r_f\mathbf{1})}{\mathbf{1}^\top\boldsymbol{\Sigma}^{-1}(\boldsymbol{\mu} - r_f\mathbf{1})}$$

With the long-only constraint $w_i \geq 0$, these closed forms are no longer valid – the KKT conditions require careful handling of active bound constraints. SLSQP handles the bounded constrained problem correctly. The trade-off is numerical sensitivity to the conditioning of $\boldsymbol{\Sigma}$; with $N = 6$ assets and $T \approx 1260$ daily observations, this is not a concern in practice.

6.7.5 Frontier Curve via Parametric Optimization

The full efficient frontier is traced by solving a parametric optimization: for each target return μ^* in $[\mu_{\text{MV}},\ \mu_{\max}]$, find the minimum-variance portfolio achieving exactly μ^*:

$$\begin{aligned} \min_{\boldsymbol{w}} \quad & \sqrt{\boldsymbol{w}^\top\boldsymbol{\Sigma}\boldsymbol{w}} \\ \text{s.t.} \quad & \boldsymbol{w}^\top\boldsymbol{\mu} = \mu^*, \quad \mathbf{1}^\top\boldsymbol{w} = 1, \quad \boldsymbol{w} \geq \mathbf{0} \end{aligned} \tag{6.33}$$

The implementation (`portfolio_engine.py:481-532`) runs this optimization at 50 equally spaced target returns. Solver failures at infeasible targets (which can occur under the long-only constraint) are silently dropped, yielding only the achievable portion of the frontier.

6.8 Correlation Analysis

Source files: `portfolio_engine.py` lines 250–259, `routers/correlation.py`

6.8.1 Pearson Correlation Matrix

The Pearson correlation between assets i and j is:

$$\rho_{ij} = \frac{\text{Cov}(R_i, R_j)}{\sigma_i\sigma_j} \tag{6.34}$$

The implementation delegates to `pandas.DataFrame.corr()` (`portfolio_engine.py:252`), which computes pairwise Pearson correlations on daily return columns. The correlation matrix is symmetric with $\rho_{ii} = 1$.

Portfolio variance can be written explicitly in terms of pairwise correlations:

$$\sigma_p^2 = \sum_{i=1}^{N} \sum_{j=1}^{N} w_i w_j \sigma_i \sigma_j \rho_{ij} \tag{6.35}$$

This form makes the diversification benefit explicit: when $\rho_{ij} < 1$, the cross terms are smaller than they would be in a fully correlated portfolio ($\rho_{ij} = 1$), reducing σ_p^2 below the weighted sum of individual variances.

6.8.2 Rolling 60-Day Correlation

Static correlation hides an important phenomenon: correlations are not constant over time. During market stress, correlations tend to spike toward 1 – the exact moment when diversification is most needed, it tends to fail most dramatically.

The rolling correlation uses a 60-day window (`portfolio_engine.py:255-259`):

$$\hat{\rho}_{ij,t}^{(60)} = \mathrm{Corr}\Big(R_{i,\,t-59:t},\ R_{j,\,t-59:t}\Big) \tag{6.36}$$

The correlation endpoint computes the rolling correlation of each individual asset versus the weighted portfolio return, visualizing how each asset's co-movement with the overall portfolio evolves over time.

6.8.3 Diversification Ratio

The correlation endpoint also computes the diversification ratio (`routers/correlation.py:52-60`):

$$\mathrm{DR} = \frac{\sum_{i=1}^{N} w_i \sigma_i}{\sigma_p} \tag{6.37}$$

$\mathrm{DR} \geq 1$ always. $\mathrm{DR} = 1$ means the portfolio behaves as if all assets are perfectly correlated. $\mathrm{DR} > 1$ indicates genuine diversification benefit: a DR of 1.2 means the portfolio's realized volatility is 20% less than a hypothetical single-asset portfolio with the same weighted-average volatility.

6.9 System Architecture

6.9.1 Backend: Pure Functions and Modular Routers

The backend follows a strict separation between computation and I/O:

- `portfolio_engine.py`: Pure functions accepting `pandas.Series/DataFrame` and returning serializable Python types. No file I/O, no HTTP calls, no side effects.
- `data_loader.py`: All I/O – yfinance calls, Parquet cache reads and writes. Returns a standardized `dict` consumed by routers.
- `routers/*.py`: Each file handles exactly one analytical concern and orchestrates `data_loader` followed by `portfolio_engine` calls.

The FastAPI application (`main.py`) mounts all six routers under `/api/v1` and runs on port 2055:

Endpoint	Router	Key outputs
`GET /api/v1/overview`	`overview.py`	KPIs, allocation, benchmark comparison
`GET /api/v1/performance`	`performance.py`	Cumulative returns, drawdown, calendar
`GET /api/v1/risk`	`risk.py`	VaR, CVaR, Sharpe, Sortino, Calmar, beta
`GET /api/v1/montecarlo`	`montecarlo.py`	Percentile paths, profit probability
`GET /api/v1/frontier`	`frontier.py`	Random cloud, frontier curve, optimal points
`GET /api/v1/correlation`	`correlation.py`	Correlation matrix, rolling corr, DR

Table 6.2: API surface. All endpoints accept a `period` query parameter (`1y`, `2y`, `3y`, `5y`, `10y`, `max`).

6.9.2 Frontend: Next.js 8-Tab Dashboard

The Next.js frontend (`src/components/portfolio/PortfolioDashboard.tsx`) defines eight tabs in the `TABS` array (lines 14–23) and renders each as a separate panel component:

1. **About** – Project description and methodology overview.
2. **Overview** – KPI cards (portfolio value, CAGR, Sharpe, max drawdown), allocation breakdown, benchmark comparison.
3. **Performance** – Cumulative return chart (portfolio vs. SPY vs. individual assets), rolling returns, drawdown chart, calendar return heatmap.
4. **Risk** – VaR comparison table (parametric/historical/MC at 95%/99%), CVaR, return distribution histogram, rolling volatility, benchmark-relative metrics.
5. **Correlation** – Heatmap of asset correlation matrix, rolling 60-day correlation time series per asset, diversification ratio KPI.
6. **Monte Carlo** – Fan chart of percentile envelopes (5th/25th/50th/75th/95th), probability of profit, probability of reaching a user-specified target, final-value statistics table.
7. **Frontier** – Scatter plot of 5,000 random portfolios colored by Sharpe ratio, efficient frontier curve, highlighted minimum-variance and maximum-Sharpe points, current portfolio position.
8. **Methodology** – Inline documentation of formulas and data sources.

A global `PortfolioContext` (`src/context/PortfolioContext.tsx`) stores the selected time period and propagates it to all panel-level API hooks (`src/hooks/usePortfolioAPI.ts`), so changing the period selector in the header triggers refetches across all tabs simultaneously.

6.10 Challenge Questions

Challenge Question

Log returns vs. arithmetic returns: quantify the approximation error.
Log returns $r_t = \ln(1+R_t) \approx R_t$ for small R_t. Assuming daily portfolio returns $R_t \sim \mathcal{N}(\mu_d, \sigma_d^2)$ with $\mu_d = 0.04/252$ and $\sigma_d = 0.15/\sqrt{252}$, compute the expected 5-year terminal value under (a) arithmetic compounding and (b) log-return compounding. At what annualized volatility level does the difference in 5-year terminal value exceed 50 basis points of return?

Challenge Question

Eigendecomposition of the covariance matrix and minimum-variance weights.
The portfolio variance $\boldsymbol{w}^\top \boldsymbol{\Sigma} \boldsymbol{w}$ can be written as $\sum_k \lambda_k (\boldsymbol{w}^\top \boldsymbol{v}_k)^2$ where $(\lambda_k, \boldsymbol{v}_k)$ are the eigenvalue-eigenvector pairs of $\boldsymbol{\Sigma}$. Without the long-only constraint, show algebraically that the minimum-variance portfolio maximally loads on the eigenvectors with the *smallest* eigenvalues. Interpret the first eigenvector (associated with the largest eigenvalue) for the six-asset portfolio – what economic factor does it likely represent?

Challenge Question

Estimation error and the Ledoit-Wolf shrinkage estimator.
With $N = 6$ assets and $T = 1260$ daily observations, how many free parameters are being estimated in $\boldsymbol{\Sigma}$? Compute the asymptotic standard error of a sample correlation coefficient $\hat{\rho}_{ij}$ under the null $\rho_{ij} = 0$. How does the sample covariance matrix's condition number affect the numerical stability of the SLSQP optimization? Research the Ledoit-Wolf linear shrinkage estimator $\hat{\boldsymbol{\Sigma}}_{\text{LW}} = (1-\alpha)\hat{\boldsymbol{\Sigma}} + \alpha \mu_{\text{trace}} \mathbf{I}$ and explain the bias-variance trade-off it exploits.

Interview Question

Q: A colleague presents a backtest showing Sharpe ratio of 3.2. What questions do you ask first?

Key concerns: (1) *In-sample overfitting* – was the strategy fitted on the same data used to evaluate it? (2) *Survivorship bias* – does the universe include only assets that survived to today? (3) *Look-ahead bias* – does any signal use data that would not have been available at the time of the trade? (4) *Transaction costs* – does the backtest account for bid-ask spread and market impact? (5) *Sample period* – does it cover a full market cycle including a bear market? A Sharpe of 3.2 on a passive ETF portfolio over any realistic sample period without leverage is almost certainly indicative of an error.

Interview Question

Q: Why is the efficient frontier a hyperbola in (σ, μ) space?

Without the long-only constraint, the set of minimum-variance portfolios for each target return μ^* traces a parabola in (σ^2, μ) space, arising from the quadratic form $\boldsymbol{w}^\top \boldsymbol{\Sigma} \boldsymbol{w}$. In (σ, μ) space (taking the square root of variance), this parabola becomes a hyperbola. Only the upper branch above the global minimum-variance point is *efficient* in the Markowitz sense. The long-only constraint makes the frontier piecewise and truncates it, but the hyperbolic shape from the unconstrained case provides the intuition for why the frontier curves outward.

Challenge Question

Prove that CVaR is subadditive; construct an explicit VaR non-subadditivity counterexample.

Consider two zero-coupon bonds, each with face value \$100 and independent default probability $p = 0.04$. Construct the joint loss distribution for a portfolio of \$50 in each bond. Show analytically that $\text{VaR}_{0.95}(\text{Bond } 1 + \text{Bond } 2) > \text{VaR}_{0.95}(\text{Bond } 1) + \text{VaR}_{0.95}(\text{Bond } 2)$, violating subadditivity. Then compute $\text{CVaR}_{0.95}$ for each bond individually and for the combined portfolio, and verify the subadditivity inequality $\text{CVaR}_{0.95}(X + Y) \leq \text{CVaR}_{0.95}(X) + \text{CVaR}_{0.95}(Y)$ holds.

Interview Question

Q: The simulation estimates $\hat{\mu}$ from historical returns. What is estimation risk, and how would you address it?

Estimation risk refers to the statistical uncertainty in the estimated parameters themselves. The sample mean has standard error $\hat{\sigma}/\sqrt{T}$; for a five-year daily series with $T = 1260$ and annualized volatility $\hat{\sigma} = 15\%$, the 95% confidence interval for the annualized drift is approximately $\hat{\mu} \pm 1.96 \times 15\%/\sqrt{5} \approx \hat{\mu} \pm 13\%$. In other words, the estimated annual drift is essentially indistinguishable from zero at conventional significance levels. Approaches to address estimation risk: (1) Bayesian shrinkage toward a prior (Black-Litterman model combines market equilibrium with analyst views); (2) setting $\mu = 0$ in the drift and relying solely on $\hat{\sigma}$ for uncertainty (parameter-free simulation); (3) bootstrapping historical return blocks rather than parametric GBM; (4) robust optimization that explicitly accounts for confidence ellipsoids around $\hat{\boldsymbol{\mu}}$ and $\hat{\boldsymbol{\Sigma}}$.

Chapter 7

Operational Efficiency: NYC 311

Key Insight

This chapter covers a full-stack operational analytics dashboard built on 3.5 million NYC 311 service requests from calendar year 2024. The project demonstrates bottleneck identification, SLA compliance analysis with formal hypothesis testing, Pareto prioritization, process mining via Sankey flow diagrams, and custom D3.js visualizations that cannot be replicated with standard charting libraries.

7.1 Business Context

Source files: `projects/06-operational-efficiency/README.md`

NYC 311 is the city's non-emergency service request line, handling everything from noise complaints and pothole reports to building violations and street light outages. In 2024, the system processed approximately 3.5 million service requests across more than 30 city agencies. The core business question driving this analysis is:

> *Where are the bottlenecks in NYC's 311 service request pipeline, which agencies consistently miss SLA targets, and what resource allocation changes could improve resolution times—enabling city operations managers to prioritize improvement initiatives?*

The intended audience is city operations managers and budget analysts who need to justify staffing changes, agency funding decisions, and process improvement programs. The output is not a one-time report but an interactive dashboard with five filter dimensions (agency, complaint type, borough, channel, month) so that any manager can slice to their area of responsibility.

7.1.1 Analyst Deliverables

The project produces four artifacts:

1. A **Next.js dashboard** with seven tabs (overview, bottleneck, departments, geographic, trends, Pareto, and embedded notebooks).
2. A **FastAPI backend** (port 2056) exposing six analytical endpoints backed by eleven pure functions in `ops_engine.py`.
3. Four **Jupyter notebooks** documenting the full analytical narrative from ingestion through process mining.
4. A **three-stage ETL pipeline** that downloads, cleans, and enriches raw data from the Socrata API.

7.2 Data Acquisition: Socrata SODA API

Source files: `projects/06-operational-efficiency/data-pipeline/01_download.py`, `projects/06-operational-efficiency/data-pipeline/02_clean.py`, `projects/06-operational-effi`

7.2.1 Socrata Pagination Strategy

The NYC Open Data portal exposes the 311 dataset (identifier `erm2-nwe9`) via the Socrata Open Data API (SODA). The dataset exceeds what can be fetched in a single request, requiring explicit pagination.

Listing 7.1: Pagination loop in `01_download.py`

```
BASE_URL = "https://data.cityofnewyork.us/resource/erm2-nwe9.json"
PAGE_LIMIT = 50_000

def build_params(offset: int) -> dict:
    return {
        "$select": ", ".join(COLUMNS),
        "$where": "created_date >= '2024-01-01T00:00:00' "
                  "AND created_date < '2025-01-01T00:00:00'",
        "$order": "unique_key",
        "$limit": PAGE_LIMIT,
        "$offset": offset,
    }
```

The loop fetches pages of 50,000 rows ordered by `unique_key` to guarantee stable pagination. Each page is fetched with retry logic (up to three attempts, with exponential back-off of `RETRY_DELAY_SECONDS * attempt`) before raising a `RuntimeError`. Termination occurs when a page returns fewer than `PAGE_LIMIT` rows. The complete download takes approximately ten minutes and produces a raw CSV.

Sixteen columns are selected at query time rather than downloading all available fields—a deliberate choice to reduce transfer size and avoid downstream memory pressure when loading into pandas.

7.2.2 Data Cleaning (`02_clean.py`)

The cleaning stage applies six deterministic transformations in sequence:

1. **Date parsing**: `created_date`, `closed_date`, and `due_date` are parsed with `errors='coerce'` (invalid dates become `NaT`) and timezone information is stripped to produce timezone-naive UTC timestamps.
2. **Null-date removal**: Rows where `created_date` is null are dropped—these cannot be placed on any timeline and are useless for any time-based analysis.
3. **Borough standardization**: Whitespace stripped, title-cased, and the value `'Unspecified'` mapped to `null`.
4. **Agency code normalization**: Stripped and uppercased.
5. **Status normalization**: Stripped and title-cased.
6. **Temporal anomaly correction**: Any row where `closed_date < created_date` has `closed_date` set to `NaT`. These are data entry errors that would produce negative resolution times if left uncorrected.
7. **Deduplication**: Duplicate `unique_key` values are dropped, keeping the first occurrence.

Output is written to `nyc311_clean.parquet` using PyArrow for efficient columnar storage.

7.2.3 Feature Engineering (`03_enrich.py`)

The enrichment stage derives ten columns that power all downstream analytics:

Table 7.1: Engineered columns added by `03_enrich.py`

Column	Description
`resolution_hours`	`(closed_date - created_date)` in hours
`resolution_days`	`resolution_hours / 24`; null if not closed
`is_overdue`	`True` if `closed_date > due_date`; `null` if either is missing
`response_category`	Bucketed: $<$ 1 day, 1–3 days, 3–7 days, 7–30 days, $>$ 30 days
`created_month`	`YYYY-MM` string for monthly grouping
`created_dow`	Spanish day name (Lunes, Martes, . . . , Domingo)
`created_hour`	Integer 0–23
`complaint_category`	Top 20 complaint types by frequency; remainder mapped to `'Otros'`
`process_stage`	Status $\rightarrow$ {Abierto, En Progreso, Cerrado, Pendiente, Otro}
`sla_status`	`Cumple` / `Incumple` / `Sin SLA`

The `complaint_category` bucketing is intentional: limiting to the top 20 types prevents the Sankey and Pareto visualizations from becoming unreadable while concentrating analytical attention on the complaint types that collectively generate the majority of volume.

7.3 SLA Compliance Analysis

Source files: `projects/06-operational-efficiency/backend/ops_backend/ops_engine.py`, `projects/06-operational-efficiency/src/components/ops/SLAVerdictCard.tsx`

7.3.1 SLA Definition and Classification

Service Level Agreements in the 311 dataset are encoded as the `due_date` field—the date by which the agency is expected to close the request. Not every complaint type has a defined SLA; many rows have a null `due_date`.

The enrichment step classifies every row into one of three states:

$$\texttt{sla\_status} = \begin{cases} \texttt{Sin SLA} & \text{if } \texttt{due\_date} = \texttt{null} \\ \texttt{Cumple} & \text{if } \texttt{due\_date} \neq \texttt{null} \ \wedge \ \texttt{closed\_date} \leq \texttt{due\_date} \\ \texttt{Incumple} & \text{if } \texttt{due\_date} \neq \texttt{null} \ \wedge \ \texttt{closed\_date} > \texttt{due\_date} \end{cases} \tag{7.1}$$

Open requests past their due date are also classified as `Incumple`:

Listing 7.2: Open-but-past-due classification in `03_enrich.py`

```
open_past_due = (
    has_due
    & df["closed_date"].isna()
    & (pd.Timestamp.now() > df["due_date"])
)
df.loc[open_past_due, "sla_status"] = "Incumple"
```

7.3.2 Compliance Rate Formula

The compliance rate is computed only over rows that have a defined SLA (i.e., `Sin SLA` rows are excluded from the denominator):

$$C = \frac{\#\{\text{rows where sla\_status} = \texttt{Cumple}\}}{\#\{\text{rows where sla\_status} \in \{\texttt{Cumple}, \texttt{Incumple}\}\}} \times 100 \tag{7.2}$$

In `ops_engine.py`, this is the `_sla_rate()` helper used throughout the codebase:

Listing 7.3: `_sla_rate()` in `ops_engine.py`

```python
def _sla_rate(group: pd.DataFrame) -> float | None:
    with_sla = group[group["sla_status"] != "Sin SLA"]
    if len(with_sla) == 0:
        return None
    return round(float((with_sla["sla_status"] == "Cumple").mean()), 4)
```

7.3.3 Verdict System

The `sla_verdict()` function maps a numeric compliance rate to a categorical verdict displayed prominently in the dashboard:

$$\text{Verdict} = \begin{cases} \texttt{CUMPLE} & C \geq 85\% \\ \texttt{EN RIESGO} & 70\% \leq C < 85\% \\ \texttt{INCUMPLE} & C < 70\% \end{cases} \tag{7.3}$$

The `SLAVerdictCard` React component renders the verdict with color-coded styles—green for CUMPLE, amber for EN RIESGO, red for INCUMPLE—using a drop shadow glow effect proportional to the severity.

7.3.4 Statistical Testing: Comparing Agency Compliance Rates

When comparing two agencies' compliance rates, we are comparing two proportions from independent samples. The appropriate test is a two-proportion z-test:

$$Z = \frac{\hat{p}_1 - \hat{p}_2}{\sqrt{\hat{p}_{\text{pool}}\,(1 - \hat{p}_{\text{pool}})\left(\dfrac{1}{n_1} + \dfrac{1}{n_2}\right)}} \tag{7.4}$$

where $\hat{p}_{\text{pool}} = \dfrac{x_1 + x_2}{n_1 + n_2}$ is the pooled proportion, x_i is the count of compliant requests for agency i, and n_i is the total requests with a defined SLA.

7.3.5 Bonferroni Correction for Multiple Comparisons

When simultaneously comparing k agencies, running each test at $\alpha = 0.05$ inflates the family-wise error rate. The Bonferroni correction adjusts the per-test threshold:

$$\alpha_{\text{adj}} = \frac{\alpha}{m} \tag{7.5}$$

where m is the number of pairwise comparisons. For 10 agencies, $m = \binom{10}{2} = 45$, yielding $\alpha_{\text{adj}} = 0.05/45 \approx 0.0011$. This is conservative—the Holm-Bonferroni step-down

procedure is a less conservative alternative that maintains the same family-wise error rate guarantee.

Challenge Question

Why does Bonferroni become increasingly conservative as the number of comparisons grows? Under what conditions would you prefer a false discovery rate (FDR) approach such as Benjamini-Hochberg over Bonferroni for an operational SLA comparison?

7.4 Bottleneck Detection

Source files: `projects/06-operational-efficiency/backend/ops_backend/ops_engine.py`, `projects/06-operational-efficiency/backend/ops_backend/routers/bottleneck.py`

The `detect_bottlenecks()` function identifies the ten agency–complaint-type combinations with the highest average resolution time:

Listing 7.4: Bottleneck aggregation in `ops_engine.py`

```
agg = (
    work.groupby(["agency_name",
    "complaint_category"])["resolution_days"]
    .agg(["mean", "median", "count"])
    .reset_index()
)
agg.columns = [
    "agency_name", "complaint_category",
    "avg_days", "median_days", "count"
]
agg = agg.sort_values("avg_days", ascending=False).head(10)
```

Each bottleneck record is enriched with the SLA compliance rate for that specific agency–complaint-type subgroup, enabling managers to distinguish between *slow but compliant* cases (SLA targets set generously) and *slow and non-compliant* cases (genuine operational failures).

Key Insight

Sorting by mean resolution time rather than median can misrepresent bottlenecks when resolution time distributions are heavily right-skewed—a single 180-day outlier can dominate the mean. In practice, both statistics are returned to the frontend; the median is the more robust indicator of typical performance.

Resolution time percentile statistics (P90, P95) are reported in the analytical notebooks but are not surfaced in the API to keep response payloads lightweight. They can be added via `numpy.percentile()` calls within the same aggregation block.

7.5 Pareto Analysis

Source files: `projects/06-operational-efficiency/backend/ops_backend/ops_engine.py`, `projects/06-operational-efficiency/src/components/d3/ParetoChart.tsx`

7.5.1 Pareto Calculation

The `pareto_analysis()` function builds the cumulative frequency distribution over complaint categories:

Listing 7.5: Pareto accumulation in `ops_engine.py`

```
vc = df["complaint_category"].value_counts()
cum_pct = 0.0
for cat, cnt in vc.items():
    pct = float(cnt) / total
    cum_pct += pct
    items.append({
        "complaint_type": str(cat),
        "count": int(cnt),
        "pct": round(pct * 100, 1),
        "cumulative_pct": round(cum_pct * 100, 1),
    })
```

Each item carries its individual percentage and the running cumulative total. The 80% threshold—the Pareto cutpoint—is rendered as a dashed amber reference line in the D3 visualization.

7.5.2 Priority Matrix

Alongside the Pareto chart, `priority_matrix()` produces a scatter dataset placing each complaint type on two axes: volume (x) and average resolution time (y). This quadrant view reveals four actionable categories:

- **High volume, fast resolution**: Already optimized; maintain current process.
- **High volume, slow resolution**: Priority 1 for process improvement—small efficiency gains affect the most requests.
- **Low volume, slow resolution**: Niche problems; investigate but deprioritize versus high-volume quadrant.

- **Low volume, fast resolution**: Low urgency.

Only complaint types with more than 100 records are included to avoid noise from rare categories dominating the scatter.

7.6 Process Mining: Sankey Flow

Source files: `projects/06-operational-efficiency/backend/ops_backend/ops_engine.py`, `projects/06-operational-efficiency/src/components/d3/SankeyFlow.tsx`

7.6.1 Four-Layer Flow Construction

The Sankey diagram models the lifecycle of a 311 request as a directed flow through four node layers:

$$\text{complaint\_category} \longrightarrow \text{agency\_name} \longrightarrow \text{process\_stage} \longrightarrow \text{response\_category}$$

The `build_sankey_data()` function constructs nodes and links using a namespace-prefixed dictionary to prevent name collisions across layers (e.g., a node named "Cerrado" in process_stage is distinct from any hypothetical agency named the same):

Listing 7.6: Namespace-prefixed node registry in `ops_engine.py`

```
def _get_or_add(name: str, prefix: str) -> int:
    key = f"{prefix}::{name}"
    if key not in node_index:
        node_index[key] = len(node_names)
        node_names.append({"id": len(node_names), "name": name})
    return node_index[key]
```

To keep the diagram readable, the function filters to the top 10 complaint types and top 10 agencies by volume before building links.

7.6.2 What the Sankey Reveals

The Sankey is the most analytically rich visualization in the dashboard because it simultaneously encodes:

- **Volume concentration**: Wide bands at the complaint layer identify the Pareto-dominant complaint types.
- **Agency routing**: How complaint types are distributed across agencies—some complaints are handled by a single agency while others are split.

- **Process completion**: The split between "Cerrado" and "Abierto"/"En Progreso" at the stage layer shows the overall closure rate.
- **Resolution speed**: The flow into "<1 dia" vs ">30 dias" response categories immediately shows which paths are fast and which are bottlenecked.

The bottleneck node (highest throughput) is visually highlighted with a red drop shadow (`drop-shadow(0 0 8px rgba(239,68,68,0.6))`), drawing the manager's eye directly to the most critical stage.

7.7 Temporal Patterns

Source files: `projects/06-operational-efficiency/backend/ops_backend/ops_engine.py`, `projects/06-operational-efficiency/src/components/d3/SeasonalityChart.tsx`

7.7.1 Day-of-Week / Hour-of-Day Heatmap

The `dow_hour_heatmap()` function produces a 7×24 pivot matrix:

Listing 7.7: DoW x hour pivot in `ops_engine.py`

```
day_order = [
    "Lunes", "Martes", "Miercoles",
    "Jueves", "Viernes", "Sabado", "Domingo"
]
pivot = df.pivot_table(
    index="created_dow",
    columns="created_hour",
    aggfunc="size",
    fill_value=0,
)
pivot = pivot.reindex(index=day_order, columns=hours, fill_value=0)
```

The reindex step is critical: without it, days with zero requests for some hour would be absent from the matrix and the pivot would have misaligned columns.

The `SeasonalityChart` D3 component renders this matrix as a color-encoded grid using an interpolator from `#0F0F0F` (dark, low volume) to `#D4A15E` (amber, high volume). Hour labels are shown only every third hour to avoid crowding.

7.7.2 Staffing Implications

The temporal heatmap supports concrete operational decisions:

- Peaks on weekday mornings (typically 9–11am) suggest that call center staffing should be front-loaded early in the working day.

- Elevated weekend volume for certain complaint types (e.g., noise complaints) argues for adjusted staffing on Saturdays and Sundays for those specific agencies.
- Monthly trend analysis via `monthly_time_series()` can detect seasonal effects—for example, heating complaint surges in winter months—enabling agencies to pre-position resources.

Interview Question

You observe that the day-of-week heatmap shows Monday mornings as the single highest-volume cell. A city operations director asks whether this represents genuine increased demand or a backlog of weekend requests being entered on Monday. How would you design a follow-up analysis to distinguish between the two hypotheses?

7.8 Custom D3 Visualizations

Source files: `projects/06-operational-efficiency/src/components/d3/SankeyFlow.tsx`, `projects/06-operational-efficiency/src/components/d3/GaugeChart.tsx`, `projects/06-operation` `projects/06-operational-efficiency/src/components/d3/ChoroplethMap.tsx`, `projects/06-operat` `projects/06-operational-efficiency/src/components/d3/SeasonalityChart.tsx`

7.8.1 Why D3 over Recharts

Standard React charting libraries such as Recharts or Chart.js cover bar charts, line charts, and scatter plots well. They cannot produce:

- **Sankey/alluvial diagrams**: Require the `d3-sankey` layout algorithm to compute node positions and link bezier paths.
- **Choropleth maps**: Require geographic path rendering from GeoJSON or custom SVG path data, plus a color scale keyed to a continuous variable.
- **Animated arc gauges**: Require `d3.arc()` with transition interpolation over the end angle.

All six D3 components follow the same pattern: a `useRef` for the SVG element, a `useEffect` that clears and redraws on data change, and a cleanup function that calls `svg.selectAll('*').remove()`.

Decision Justification

D3 was chosen over Recharts for three specific visualization types (Sankey, choropleth, gauge) while simpler visualizations (monthly line chart, bottleneck table) use standard React state and Tailwind. This hybrid approach avoids the overhead of writing D3 for every chart while still enabling visualizations that would be impossible in Recharts.

7.8.2 Gauge Chart

The gauge renders a half-donut arc from $-\pi/2$ to $+\pi/2$, with the fill arc animated over 800ms using `d3.easeCubicOut`:

Listing 7.8: Arc animation in `GaugeChart.tsx`

```
g.append('path')
  .attr('d', arcFill.endAngle(startAngle)({}) ?? '')
  .attr('fill', fillColor)
  .transition()
  .duration(800)
  .ease(d3.easeCubicOut)
  .attrTween('d', function () {
    const interpolate = d3.interpolate(startAngle, fillAngle)
    return (t: number) => arcFill.endAngle(interpolate(t))({}) ?? ''
  })
```

Fill color maps to the verdict thresholds: green ($\geq$85%), amber ($\geq$70%), red ($<$70%).

7.8.3 Choropleth Map

The choropleth uses hand-coded SVG paths approximating the five NYC borough outlines on a 500×500 viewBox. A sequential color scale maps request volume (or SLA compliance rate) from near-black (`#1A1510`) to amber (`#D4A15E`):

Listing 7.9: Color scale in `ChoroplethMap.tsx`

```
const colorScale = d3.scaleSequential(
  (t: number) => d3.interpolateRgb('#1A1510', '#D4A15E')(t)
).domain([minVal * 0.5, maxVal])
```

The `minVal * 0.5` lower bound prevents low-volume boroughs from appearing completely black. Borough shapes support click-through filtering: clicking a borough propagates the selection to the `OpsFilterContext` and re-fetches all dashboard panels for that borough.

7.8.4 Pareto Chart

The Pareto chart uses dual y-axes—a left linear scale for count (bars in amber/dark-amber) and a right 0–100 scale for cumulative percentage (line in blue). The 80% reference line is rendered as a dashed amber line:

Listing 7.10: 80% reference line in `ParetoChart.tsx`

```
g.append('line')
```

```
    .attr('x1', 0).attr('x2', innerW)
    .attr('y1', yRight(80)).attr('y2', yRight(80))
    .attr('stroke', '#F2C94C')
    .attr('stroke-dasharray', '6,4')
    .attr('opacity', 0.7)
```

Bars at or before the 80% cumulative threshold are rendered in `#D4A15E` (highlighted amber); bars after the threshold use `#6B4E2A` (dark brown), visually separating the "vital few" from the "trivial many."

7.9 System Architecture

Source files: `projects/06-operational-efficiency/backend/ops_backend/main.py`, `projects/06-ope`
`projects/06-operational-efficiency/backend/ops_backend/ops_engine.py`, `projects/06-operatio`

7.9.1 Backend: Pure Functions Catalog

`ops_engine.py` contains eleven functions, all pure (no side effects, no global state). Each receives a filtered `pd.DataFrame` and returns a JSON-serializable dict or list:

7.9.2 Five-Dimension Filter System

All six API endpoints accept the same five query parameters, each of which is applied as an exact-match filter in `data_loader.apply_filters()`:

On the frontend, `OpsFilterContext` holds the active filter state and derives a `queryString` that is appended to every SWR data-fetching call. When the user changes a filter, React re-renders all panels simultaneously because each panel calls `useOpsFilters()` and includes `queryString` in its SWR key.

7.9.3 API Route Map

The data is loaded once at module import time in `data_loader.py`, selecting only the 15 required columns from the enriched parquet. This keeps the in-memory footprint minimal and makes all endpoint handlers stateless reads of the preloaded DataFrame.

7.9.4 Frontend Tab Structure

The dashboard implements seven tabs through a `TabNav` component:

1. **Overview**: KPI row, SLA verdict card, gauge charts, borough and channel distributions.

Table 7.2: Function catalog for `ops_engine.py`

Function	Returns	Purpose
`_safe()`	scalar	Converts `NaN`/`inf` to `None` for JSON
`_sla_rate()`	float	SLA compliance rate, excluding "Sin SLA" rows
`compute_overview()`	dict	Top-level KPIs (totals, rates, distributions)
`sla_verdict()`	str	Maps compliance % to CUMPLE/EN RIESGO/INCUMPLE
`build_sankey_data()`	dict	Four-layer flow: complaint→agency→stage→response
`detect_bottlenecks()`	list	Top 10 agency–complaint combos by avg resolution time
`agency_ranking()`	list	Agencies ranked by volume with resolution stats
`agency_complaint_heatmap()`	dict	10×15 agency×complaint pivot matrix
`borough_summary()`	list	Per-borough statistics with top complaint
`monthly_time_series()`	list	Monthly aggregation of volume and SLA compliance
`dow_hour_heatmap()`	dict	7×24 request count matrix
`pareto_analysis()`	dict	Ranked complaint types with cumulative percentages
`priority_matrix()`	list	Complaint types by volume and resolution time

2. **Bottleneck**: Sankey flow diagram, resolution bottleneck table.
3. **Departments**: Agency ranking table, agency×complaint heatmap.
4. **Geographic**: Choropleth map with borough click-through filtering.
5. **Trends**: Monthly line chart, day-of-week/hour seasonality heatmap.
6. **Pareto**: Pareto chart, priority scatter matrix.
7. **Proceso Tecnico**: Four embedded Jupyter notebooks rendered as full-page HTML iframes.

7.10 Challenge and Interview Questions

Challenge Question

The `detect_bottlenecks()` function ranks by average resolution time. Construct a composite bottleneck score that incorporates volume, average resolution time, and SLA non-compliance rate. How would you normalize and weight these three factors to produce a single ranking metric that an operations director could act on?

Table 7.3: Filter dimensions and their target columns

Parameter	Column	Type
`agency`	`agency_name`	Exact string
`complaint_type`	`complaint_category`	Exact string (bucketed)
`borough`	`borough`	Exact string
`channel`	`open_data_channel_type`	Exact string
`year_month`	`created_month`	`YYYY-MM` string

Table 7.4: API endpoints served by `main.py`

Method	Path	Engine Functions Called
GET	`/api/v1/overview`	`compute_overview`, `sla_verdict`
GET	`/api/v1/bottleneck`	`build_sankey_data`, `detect_bottlenecks`
GET	`/api/v1/departments`	`agency_ranking`, `agency_complaint_heatmap`
GET	`/api/v1/geographic`	`borough_summary`
GET	`/api/v1/trends`	`monthly_time_series`, `dow_hour_heatmap`
GET	`/api/v1/pareto`	`pareto_analysis`, `priority_matrix`
GET	`/api/v1/filters`	(precomputed filter lists from `data_loader`)

Challenge Question

The `complaint_category` column caps categories at 20 by frequency and collapses the rest into `'Otros'`. Propose an alternative bucketing strategy that preserves analytical granularity for the Sankey without making the diagram unreadable. What is the trade-off between using frequency-based top-N versus agency-based filtering?

Interview Question

An interviewer shows you the SLA compliance formula in Equation 7.2 and asks: "What is the implicit assumption about `Sin SLA` rows, and when does that assumption break down?" Walk through your reasoning about how excluding those rows from the denominator can make compliance look artificially high or low, depending on which agencies or complaint types have SLAs defined.

Interview Question

You want to determine whether Borough A has a statistically significantly lower SLA compliance rate than Borough B. Walk through the hypothesis test setup using Equation 7.4: state H_0 and H_1, specify the test statistic, describe when you would apply the Bonferroni correction from Equation 7.5, and interpret a p-value of 0.03 under both the uncorrected and corrected thresholds.

Challenge Question

The choropleth in `ChoroplethMap.tsx` uses hand-coded SVG path strings instead of a GeoJSON-based projection. What are the limitations of this approach for a production application? Describe how you would replace the static paths with a proper D3 geographic projection (e.g., `d3.geoMercator()`) using the official NYC borough boundary GeoJSON.

Interview Question

The `apply_filters()` function in `data_loader.py` applies all filters as exact-match string comparisons on a preloaded in-memory DataFrame. The data team tells you the dataset has grown to 15 million rows. What are the performance implications of this architecture, and what changes—short of migrating to a database—would you make to maintain acceptable API response times?

7.11 Summary

Table 7.5: Project 06 at a glance

Attribute	Detail
Dataset	NYC 311 Service Requests 2024, ≈3.5M rows, Socrata API (`erm2-nwe9`)
ETL	3-stage pipeline: download → clean → enrich; output: Parquet via PyArrow
Backend	FastAPI, 7 endpoints, 11 pure functions, 5-dimension filter system
Frontend	Next.js 14, D3.js, 6 custom visualizations, 7 dashboard tabs
Key analyses	SLA compliance (z-test, Bonferroni), Pareto 80/20, Sankey process mining
Unique strength	D3 visualizations (Sankey, choropleth, gauge) that Recharts cannot produce

This project demonstrates that operational analysis at scale requires more than descriptive statistics: the combination of formal hypothesis testing (z-tests with multiple comparison correction), process-level visualization (Sankey flow), prioritization frameworks (Pareto + priority matrix), and temporal pattern analysis (DoW/hour heatmap) produces a dashboard where a city manager can move from a high-level compliance verdict down to the specific agency–complaint-type combination responsible for the worst outcomes—all within a single, filtered session.

Chapter 8

Shared Infrastructure & Deployment

All seven portfolio projects share a common infrastructure layer: a single consolidated FastAPI process serving multiple sub-applications, Docker images built from the repository root, a GitHub Actions CI/CD pipeline that uses Workload Identity Federation to deploy to Google Cloud Run, and a Next.js reverse-proxy pattern that keeps backend URLs off the client bundle. This chapter documents each layer in detail, explaining the design rationale behind every decision.

8.1 Consolidated FastAPI Architecture

Source files: backend/main.py, backend/dev.sh, backend/requirements.txt

Rather than deploying five independent FastAPI services to Cloud Run, all API backends are mounted into a single root application using FastAPI's `app.mount()` mechanism. The entry point lives at `backend/main.py`.

Listing 8.1: Consolidated API entry point – backend/main.py

```
app = FastAPI(title="DA Portfolio API", version="1.0.0")

app.add_middleware(CORSMiddleware, allow_origins=[
    "http://localhost:3000",  "http://localhost:3050",
    "http://localhost:3051",  "http://localhost:3053",
    "http://localhost:3055",  "http://localhost:3056",
], allow_methods=["GET"], allow_headers=["*"])

app.mount("/insurance", insurance_app)
app.mount("/olist",     olist_app)
app.mount("/abtest",    abtest_app)
```

```
12 app.mount("/kpi",       kpi_app)
13 app.mount("/portfolio", portfolio_app)
14 app.mount("/ops",       ops_app)
```

Key Insight

That origin list is localhost-only, and deliberately so. It once carried `"https://*.vercel.app"`, which is worth dwelling on for two reasons. First, it never worked: Starlette compares `allow_origins` by exact string, and patterns require `allow_origin_regex` — so the entry matched nothing even while the Vercel deployments were live. Second, it is no longer needed at all. The dashboards reach this service through a same-origin proxy running at the edge (Section 8.6), which is a server-side fetch: no browser origin is involved, so CORS never enters the picture in production. The remaining entries exist purely so each backend can be driven standalone during development.

Each sub-application is a fully independent FastAPI instance with its own routers, data loaders, and `/health` endpoint. The root app provides only the mount table and a shared CORS middleware layer. Each sub-app's Python package is injected via `sys.path.insert()` at startup, pointing to the project-specific backend directory.

Listing 8.2: sys.path manipulation for sub-app discovery (lines 10–15)

```
1 PROJECTS = os.path.join(os.path.dirname(os.path.abspath(__file__)),
      "..", "projects")
2 sys.path.insert(0, os.path.join(PROJECTS,
      "01-insurance-claims-dashboard", "backend"))
3 sys.path.insert(0, os.path.join(PROJECTS, "00-demo-aestehtics",
      "backend"))
4 sys.path.insert(0, os.path.join(PROJECTS, "03-ab-test-analysis",
      "backend"))
5 sys.path.insert(0, os.path.join(PROJECTS, "04-executive-kpi-report",
      "backend"))
6 sys.path.insert(0, os.path.join(PROJECTS, "06-operational-efficiency",
      "backend"))
```

Key Insight

The `app.mount()` pattern is not the same as `include_router()`. A mounted sub-app is a complete ASGI application: it handles its own middleware, exception handlers, and OpenAPI schema. When the root app receives a request to `/insurance/api/v1/loss-triangle`, FastAPI strips the `/insurance` prefix and dispatches the remainder to `insurance_app` as if it were receiving `/api/v1/loss-triangle` directly. This means each sub-app works identically in standalone mode and in the consolidated deployment.

A single health endpoint at the root level reports all mounted services:

Listing 8.3: Root health endpoint

```
@app.get("/health")
def health():
    return {"status": "ok",
            "services": ["insurance", "olist", "abtest", "kpi", "ops"]}
```

The development server is started from the repository root via `backend/dev.sh`:

Listing 8.4: backend/dev.sh

```
#!/bin/bash
cd "$(dirname "$0")/.."
python3 -m uvicorn backend.main:app \
    --host 0.0.0.0 --port 8080 --reload
```

Decision Justification

Why one process instead of five? Cloud Run charges per instance-hour. Five separate services would each incur cold-start latency and idle cost. A single consolidated service shares one warm instance, reduces total memory overhead from duplicate interpreter initialisation, and simplifies the CI/CD pipeline to a single deploy step. The trade-off is that a bug in one sub-app's data loader crashes the entire process on startup – acceptable for a portfolio context where each project is individually tested before merging.

8.2 Docker Multi-Service Architecture

Source files: backend/Dockerfile, projects/02-ecommerce-cohort-analysis/Dockerfile

Both Dockerfiles use the *repository root* as the build context. This is intentional: it allows a single `COPY` instruction to pull code and processed data from multiple project

subdirectories, which would be impossible if the build context were scoped to the project folder alone.

Listing 8.5: Consolidated API Dockerfile – backend/Dockerfile

```
FROM python:3.11-slim
WORKDIR /workspace

COPY backend/requirements.txt ./backend/
RUN pip install --no-cache-dir -r backend/requirements.txt

COPY backend/main.py ./backend/

# Insurance backend + data
COPY projects/01-insurance-claims-dashboard/backend/insurance_backend/ \
     ./projects/01-insurance-claims-dashboard/backend/insurance_backend/
COPY projects/01-insurance-claims-dashboard/data/processed/ \
     ./projects/01-insurance-claims-dashboard/data/processed/

# ... (repeated for olist, abtest, kpi, ops)
EXPOSE 8080
CMD ["uvicorn", "backend.main:app", "--host", "0.0.0.0", "--port",
    "8080"]
```

The Streamlit Dockerfile follows the same repo-root build context pattern but is far simpler because it serves only one project:

Listing 8.6: Streamlit Dockerfile – projects/02-ecommerce-cohort-analysis/Dockerfile

```
# Build from repo root: docker build -f projects/02-.../Dockerfile .
FROM python:3.11-slim
WORKDIR /app

COPY projects/02-ecommerce-cohort-analysis/requirements.txt ./
RUN pip install --no-cache-dir -r requirements.txt

COPY projects/02-ecommerce-cohort-analysis/streamlit/ ./streamlit/
COPY projects/02-ecommerce-cohort-analysis/data/processed/
    ./data/processed/

EXPOSE 8501
CMD ["streamlit", "run", "streamlit/app.py",
     "--server.port=8501", "--server.address=0.0.0.0",
     "--server.headless=true"]
```

Key Insight

The layer ordering in both Dockerfiles is deliberate. Dependencies (`pip install`) come before source code `COPY` instructions. Docker caches each layer independently: if only Python source files change but `requirements.txt` is unchanged, the expensive pip install layer is served from cache, reducing rebuild time from several minutes to seconds. Processed data files are copied *last*, after source code, because data changes trigger a full rebuild regardless – there is no benefit to caching incomplete images.

Docker build commands reference the Dockerfile explicitly while using the repository root as context:

Listing 8.7: Build invocation pattern (from CI workflow)

```
docker build \
  -f backend/Dockerfile \
  -t
    us-central1-docker.pkg.dev/$PROJECT_ID/da-portfolio-api/da-portfolio-api:$SHA
    \
  -t
    us-central1-docker.pkg.dev/$PROJECT_ID/da-portfolio-api/da-portfolio-api:latest
    \
  .
```

Both the `:latest` and the `:$GITHUB_SHA` tags are pushed. Cloud Run deploys the SHA-tagged image (immutable, traceable to a specific commit) while `:latest` is available for local pulls during development.

8.3 CI/CD with GitHub Actions

Source files: .github/workflows/deploy-api.yml, .github/workflows/deploy-hub.yml, .github/workflows/deplo
cohort-redirect.yml

Two independent workflows handle deployment, each activated only when relevant files change.

8.3.1 Path Filters for Selective Deploys

The API workflow triggers on any push to `main` that touches backend code from any of the five mounted projects:

Listing 8.8: Path filter in deploy-api.yml

```
on:
```

```
  push:
    branches: [main]
    paths:
      - "backend/**"
      - "projects/00-demo-aestehtics/backend/**"
      - "projects/01-insurance-claims-dashboard/backend/**"
      - "projects/03-ab-test-analysis/backend/**"
      - "projects/04-executive-kpi-report/backend/**"
      - "projects/06-operational-efficiency/backend/**"
  workflow_dispatch:
```

A second workflow once deployed the Streamlit cohort dashboard on the same principle, filtering on `projects/02-ecommerce-cohort-analysis/streamlit/**`. It has been replaced by `deploy-cohort-redirect.yml`, which deploys an nginx image to the same Cloud Run service — the service now answers HTTP 308 to `/cohorts/` for every path rather than serving a dashboard:

Listing 8.9: Path filter in deploy-cohort-redirect.yml

```
on:
  push:
    branches: [main, dev]
    paths:
      - "ops/cohort-redirect/**"
      - ".github/workflows/deploy-cohort-redirect.yml"
```

Key Insight

The service was kept rather than deleted, and the distinction is a deployment decision, not sentiment. Its `run.app` hostname had been linked from the portfolio site, from two language versions of a blog post and from the repository's own README for months. A deleted Cloud Run service answers 404, which destroys those links; a 308 forwards them and hands the search ranking to the canonical path. Paths are deliberately *not* preserved across the redirect: Streamlit's own routes have no counterpart in the rebuild, so carrying a path over would turn a working old link into a 404 on the new site.

All workflows support `workflow_dispatch`, allowing manual re-deploys without a code change (useful after updating data files in GCS).

8.3.2 Workload Identity Federation

Neither workflow stores a Google Cloud service account key in GitHub Secrets. Instead, authentication uses Workload Identity Federation (WIF): GitHub's OIDC token exchange mechanism that grants short-lived GCP credentials scoped to a specific repository.

Listing 8.10: WIF authentication step

```
- name: Authenticate to Google Cloud
  uses: google-github-actions/auth@v2
  with:
    workload_identity_provider: ${{ secrets.GCP_WIF_PROVIDER }}
    service_account: ${{ secrets.GCP_SERVICE_ACCOUNT }}
```

The three GitHub Secrets used are:

Secret	Value
GCP_PROJECT_ID	project-ad7a5be2-a1c7-4510-82d
GCP_WIF_PROVIDER	projects/451451662791/.../github-provider
GCP_SERVICE_ACCOUNT	github-deployer@project-ad7a5be2-...iam.gserviceaccount.com

Decision Justification

Why WIF instead of a service account key? Long-lived JSON key files are a security liability: they can leak, expire without notice, and require manual rotation. WIF issues credentials that expire within the workflow run (typically under one hour) and are cryptographically bound to the specific GitHub repository and branch. The WIF pool (`github-pool`) and provider (`github-provider`) were configured once in GCP and require no maintenance.

8.3.3 GCS Data Download Step

Processed parquet files are not committed to git – they live in `gs://da-portfolio-data-assets`. The CI workflow downloads them during the build step, before `docker build`, so the existing `COPY` instructions in the Dockerfile work without modification:

Listing 8.11: GCS data download step in deploy-api.yml

```
- name: Download data from GCS
  run: |
    mkdir -p projects/00-demo-aestehtics/backend/data
    mkdir -p projects/01-insurance-claims-dashboard/data/processed
    mkdir -p projects/03-ab-test-analysis/data/processed
```

```
6      mkdir -p projects/04-executive-kpi-report/data/processed
7      mkdir -p projects/06-operational-efficiency/data/processed
8      gcloud storage cp "$DATA_BUCKET/olist-backend/*"
      projects/00-demo-aestehtics/backend/data/
9      gcloud storage cp "$DATA_BUCKET/insurance-processed/*"
      projects/01-.../data/processed/
10     gcloud storage cp "$DATA_BUCKET/abtest-processed/*"
      projects/03-.../data/processed/
11     gcloud storage cp "$DATA_BUCKET/kpi-processed/*"
      projects/04-.../data/processed/
12     gcloud storage cp "$DATA_BUCKET/ops-processed/*"
      projects/06-.../data/processed/
```

This approach solves a common portfolio dilemma: large binary files should not be in git (git is not a data store), but the Docker build needs them at image build time. By staging data in GCS and downloading it into the runner's working directory, the Dockerfile sees the files as if they had always been local.

8.4 Cloud Run Deployment

Source files: .github/workflows/deploy-api.yml, .github/workflows/deploy-hub.yml, .github/workflows/deploy-cohort-redirect.yml

Both services deploy to Cloud Run in `us-central1` under the Artifact Registry repository `da-portfolio-api`.

8.4.1 Memory Sizing: The OOM Incident History

The API service memory allocation has a documented history of adjustments driven by real out-of-memory (OOM) events visible in the git log:

Commit	Memory	Reason
`5020609`	512 Mi	Initial allocation
`c80caaa`	1 Gi	OOM crash observed
`870690d`	2 Gi	Further OOM on peak load
`8a5021a`	1 Gi (revert)	Fixed via column pruning
`2fce040`	2 Gi (restored)	Actual usage 1,056 MiB post-pruning

The root cause of the OOM was the `nyc311_enriched.parquet` file loading all 26 columns at import time, consuming approximately 4,417 MB. The fix in commit `8a5021a` applies column pruning in the data loader:

Listing 8.12: Column pruning fix in ops_backend/data_loader.py

```
REQUIRED_COLUMNS = [
    "agency_name", "borough", "complaint_category", "complaint_type",
    "created_month", "created_dow", "created_hour",
    "open_data_channel_type", "resolution_days", "resolution_hours",
    "status", "sla_status", "is_overdue",
    "response_category", "process_stage",
]
df = pd.read_parquet(PARQUET_PATH, columns=REQUIRED_COLUMNS)
```

Pruning from 26 to 15 columns dropped the NYC dataset from 4,417 MB to approximately 350–400 MB. Total footprint across all five loaded datasets settled at 1,056 MiB, justifying the final 2 Gi allocation (which provides roughly 1 GiB of headroom for request processing).

8.4.2 Scaling Configuration

Both Cloud Run services use the same scaling parameters:

Listing 8.13: Cloud Run flags in deploy-api.yml

```
flags: |
  --memory=2Gi
  --cpu=1
  --min-instances=0
  --max-instances=2
  --allow-unauthenticated
  --port=8080
```

- **min-instances=0**: The service scales to zero when idle. This eliminates idle instance charges but introduces cold start latency (typically 3–8 seconds for a Python/pandas image of this size).
- **max-instances=2**: Caps horizontal scaling. A portfolio API does not need elastic scaling; two instances handle any realistic concurrent load while preventing runaway cost.
- **allow-unauthenticated**: Required for a public portfolio. Cloud Run's default is to reject unauthenticated requests.

The Streamlit service (`da-cohort-streamlit`) uses the same scaling parameters but with `-memory=512Mi` and `-port=8501`.

Key Insight

Cold starts are the most visible UX issue with min-instances=0. Because all five sub-app data loaders run at import time (module-level `pd.read_parquet()` calls), the first request to any endpoint after a cold start bears the full cost of loading all datasets into memory. A `/health` ping from the frontend on page load is a common mitigation pattern – waking the container before the user navigates to a data-heavy page.

8.5 GCS Data Management

Processed parquet files for all projects are stored in a single GCS bucket:

`gs://da-portfolio-data-assets`

The bucket is structured by project prefix:

GCS prefix	Local destination (CI)	Consumed by
`olist-backend/`	`projects/00-demo-aestehtics/backend/data/`	API (olist)
`insurance-processed/`	`projects/01-.../data/processed/`	API (insurance)
`abtest-processed/`	`projects/03-.../data/processed/`	API (abtest)
`kpi-processed/`	`projects/04-.../data/processed/`	API (kpi)
`ops-processed/`	`projects/06-.../data/processed/`	API (ops)
`cohort-processed/`	`projects/02-.../data/processed/`	Streamlit

To update a dataset after running a notebook pipeline locally:

Listing 8.14: Uploading updated data to GCS

```
gcloud storage cp data/processed/orders_enriched.parquet \
    gs://da-portfolio-data-assets/cohort-processed/
```

After the upload, a `workflow_dispatch` trigger on the relevant workflow re-deploys the service with the new data baked into the Docker image. There is no live data mount – the parquet files are embedded in the image at build time, making each deployed revision fully self-contained and reproducible.

Decision Justification

Embedded data vs. runtime mount: An alternative design would mount GCS as a FUSE filesystem (via Cloud Storage FUSE) and read parquets at request time. This would allow data updates without rebuilding the image. The current design was chosen for simplicity and cold-start predictability: a FUSE mount adds latency on the first read and requires additional IAM configuration. For a portfolio with infrequent data refreshes, the rebuild-on-update pattern is operationally simpler.

8.6 Next.js Proxy Pattern

Source files: projects/*/next.config.js, functions/api/[[path]].ts

Every frontend in the portfolio reaches its FastAPI backend through a relative `/api/<prefix>` path. The browser never calls the Cloud Run URL directly. *Which* component performs that proxying differs between development and production, and conflating the two is the single easiest mistake to make here.

Development	Production
`Browser → /api/<prefix>/<path>`	`Browser → /api/<prefix>/<path>`
`→ Next.js server (rewrites)`	`→ Pages Function (edge)`
`→ FastAPI backend URL/<path>`	`→ Cloud Run/<prefix>/<path>`

Key Insight

In production there is no Next.js server. Every dashboard is compiled with `output: 'export'`, which emits static files and no runtime — so `rewrites()` never executes. It is a development-only convenience. The production proxy is `functions/api/[[path]].ts`, a Cloudflare Pages Function running at the edge, which is why the `rewrites()` blocks below can look authoritative and still describe nothing that happens to a real user.

The development proxy is configured in each project's `next.config.js` using Next.js's `rewrites()` async function:

Listing 8.15: next.config.js – Insurance project (projects/01-.../next.config.js)

```
1 const nextConfig = {
2   async rewrites() {
3     // Server-side env var: no NEXT_PUBLIC_ prefix
4     const backend = process.env.INSURANCE_BACKEND_URL ||
      'http://localhost:2051'
5     return [
```

```
      {
        source: '/api/insurance/:path*',
        destination: `${backend}/:path*`,
      },
    ]
  },
}
module.exports = nextConfig
```

The same pattern appears across all projects, each with its own source prefix and environment variable:

Project	Source pattern	Env var	Dev default
00 (Airbnb)	`/api/olist/:path*`	`OLIST_BACKEND_URL`	`localhost:2050`
01 (Insurance)	`/api/insurance/:path*`	`INSURANCE_BACKEND_URL`	`localhost:2051`
03 (A/B Test)	`/api/abtest/:path*`	`ABTEST_BACKEND_URL`	`localhost:2053`
04 (KPI Report)	`/api/kpi/:path*`	`KPI_BACKEND_URL`	`localhost:2052`
05 (Portfolio)	`/api/portfolio/:path*`	`PORTFOLIO_BACKEND_URL`	`localhost:2055`
06 (Operations)	`/api/ops/:path*`	`OPS_BACKEND_URL`	`localhost:2056`

Key Insight

The absence of a `NEXT_PUBLIC_` prefix on the backend URL environment variable is deliberate and security-relevant. Variables without this prefix are *never* embedded in the JavaScript bundle sent to the browser, so a user inspecting the client-side bundle cannot discover the Cloud Run service URL. This matters because Cloud Run URLs follow a predictable pattern (`https://SERVICE-HASH-uc.a.run.app`), and an exposed origin could be called directly, bypassing the proxy that fronts it. The principle survived the migration even though the enforcing component changed: in development the boundary is the Next.js server, in production it is the Pages Function, and in both cases the origin stays out of the bundle.

Setting that variable switches a project from its standalone backend (`localhost:2051`) to the consolidated service (`localhost:8080/insurance`) with no code change. In production it is not read at all: the origin lives in the Pages Function instead.

Listing 8.16:].ts]Production proxy – functions/api/[[path]].ts

```
const DEFAULT_ORIGIN = 'https://da-portfolio-api-HASH-uc.a.run.app'
const SERVICES = new
    Set(['insurance','olist','abtest','kpi','portfolio','ops'])

```

```
const [service, ...rest] = segments
if (!service || !SERVICES.has(service)) return Response.json({...},
    {status: 404})

const origin = env.API_ORIGIN ?? DEFAULT_ORIGIN
const upstream = await fetch(new
    URL(`${origin}/${service}/${rest.join('/')}`))
```

Two properties of that Function are load-bearing. The service segment is checked against a fixed set rather than forwarded blindly, so the route cannot be used as an open proxy to arbitrary paths on the origin. And a failed upstream call is caught and returned as HTTP 504 rather than propagating: Cloud Run runs at `min-instances=0`, so the first request after an idle period can fail purely from a cold start, and the dashboards' cold-start banner polls until it recovers.

Key Insight

The counterpart rule on the frontend is to *never* set `NEXT_PUBLIC_*_API_URL` in a production build. Doing so bakes an absolute Cloud Run URL into the client bundle and bypasses the proxy entirely — undoing both the origin-hiding above and the service allowlist. `scripts/build-hub.mjs` strips those variables from the build environment for exactly this reason, so the mistake cannot be made by accident.

8.7 The Consolidated Frontend Hub

Source files: scripts/build-hub.mjs, functions/_middleware.ts, functions/ingest/[[path]].ts, .github/workflows/deploy-hub.yml

The backend consolidation described in Section 8.1 has an exact counterpart on the frontend, and it arrived later. The seven dashboards began as seven separate deployments on seven hostnames — six on Vercel plus a Streamlit app on a raw Cloud Run URL. They are now **one** Cloudflare Pages project, `data-analyst-hub`, serving `data-analyst.gonor.me`, each dashboard under its own path.

8.7.1 Merging Seven Builds Into One Tree

Each application builds independently with `output: 'export'` and already namespaces its own routes, so no `basePath` is required. `scripts/build-hub.mjs` then merges the seven `out/` trees into a single `dist/`.

The merge is not a blind copy. It **fails the build** when two applications emit the same path with different content — the one failure mode that a path-based merge introduces and that no individual project's tests can see. Project 00 owns the root

Table 8.1: Path allocation within the merged hub

Path	Project	Backend
/ /airbnb /olist	00 demo-aesthetics	static JSON + /olist
/insurance	01 insurance	/insurance
/cohorts	02 ecommerce	none (static JSON)
/abtest	03 ab-test	/abtest
/kpi	04 kpi-report	/kpi
/portfolio	05 portfolio	/portfolio
/operations	06 operations	/ops

(favicon.svg, 404.html), so it is merged last and a documented rule resolves those collisions in its favour.

Key Insight

Seven applications now share one origin, which turns a previously harmless habit into a bug: any asset placed at public/ root by one dashboard can collide with another's. Assets must live under the application's own slug (public/kpi/...). This is the class of error that only appears after consolidation, which is why the merge script enforces it rather than the convention relying on memory.

One workflow, deploy-hub.yml, replaced six per-frontend workflows. That is a consequence of the merge rather than tidiness: because the applications share one dist/, building only the project that changed would publish a tree missing the other six.

8.7.2 Edge Functions

Three Pages Functions run in front of the static tree:

Table 8.2: Pages Functions in the hub

Route	Purpose
/api/<svc>/*	Proxy to Cloud Run, service allowlist (Section 8.6)
/ingest/*	Same-origin analytics proxy
/health	Liveness of the Pages layer itself, no outbound call
every request	_middleware.ts: canonical-host redirect

The /ingest proxy exists because analytics loaded from a third-party host is dropped by ad blockers — both the events and the lazily-loaded session recorder. Serving it same-origin removes that failure mode.

_middleware.ts addresses a subtler problem. Creating a Pages project mints <project>.pages.dev; the custom domain is a CNAME added *on top*, never a replace-

ment. Both hostnames keep serving the same build, so the site is publicly reachable at two URLs, search engines index whichever they find first, and analytics splits between them. Measured elsewhere in this portfolio at 44% of pageviews landing on the unbranded host. The middleware issues a 301 from the bare apex to the custom domain, preserving path and query.

Key Insight

The hostname comparison is an exact match, never `endsWith('.pages.dev')`. Preview deployments live at `dev.<project>.pages.dev` and `<hash>.<project>.pages.dev`; a suffix check would redirect those to production and make every preview impossible to test — defeating the reason previews exist. Root middleware also runs ahead of *every* asset on the origin, so a fault in it takes the whole site down rather than just the redirect, which is why the deploy gate drives all three hostname cases explicitly.

8.7.3 Retiring a Hostname Without Losing It

Consolidation leaves behind hostnames that people and search engines already know. The portfolio treats them uniformly: *redirect, never delete.* The six Vercel deployments answer HTTP 308 to their new paths, the Pages apex answers 301, and the Streamlit Cloud Run service answers 308 (Section 8.3).

The exception that proves the rule is worth recording. Three additional subdomains had been created for individual dashboards during an earlier, abandoned attempt at one-subdomain-per-project. None ever received a DNS record, so none ever resolved, and analytics confirmed zero pageviews across their entire lifetime. Those were *deleted* rather than redirected: with no traffic, no inbound links and no DNS to redirect from, there was nothing for a redirect to preserve. The rule protects reachable history, not every artifact that was ever configured.

8.8 Port Convention

Port assignments follow a consistent scheme across all projects to prevent conflicts in local development. The 20XX range is reserved for FastAPI backends; the 30XX range for Next.js frontends.

Project	Frontend (Next.js)	Backend (FastAPI)	Consolidated
00 – Airbnb demo	3050	2050	`:8080/olist`
01 – Insurance	3051	2051	`:8080/insurance`
02 – Cohort (web)	3052†	—	none (static JSON)
03 – A/B Test	3053	2053	`:8080/abtest`
04 – Executive KPI	3052	2052	`:8080/kpi`
05 – Portfolio tracker	3055	2055	`:8080/portfolio`
06 – Operations	3056	2056	`:8080/ops`
Consolidated API	—	8080	Cloud Run prod
Merged hub (wrangler)	4173	—	Cloudflare Pages prod

In the consolidated mode, Next.js frontends set their backend URL environment variable to `http://localhost:8080/<prefix>` and all API traffic goes through the single uvicorn process on port 8080.

† Projects 02 and 04 both declare `next dev -p 3052` in their `package.json`. The two development servers therefore cannot run at the same time — the second to start fails to bind. This has no effect on production, where both are static exports merged into one tree and ports do not exist, which is precisely why the collision went unnoticed.

Key Insight

The convention was one port per project, and it silently stopped holding. The merged hub removed the pressure that would have surfaced it: since production no longer runs seven servers, nothing except a developer running two dashboards at once will ever notice. Conventions that are not enforced by a build step decay quietly — the same reason `scripts/build-hub.mjs` fails the build on a path collision rather than documenting the rule and hoping.

8.9 Challenge Questions

Challenge Question

Sub-app routing internals. When a request arrives at the root FastAPI app for `GET /insurance/api/v1/loss-triangle`, trace the exact path through the ASGI middleware stack. At what point does the `/insurance` prefix get stripped? What would happen if a sub-app registered its own `CORSMiddleware` – would it conflict with the root-level middleware?

Challenge Question

Layer cache invalidation. The Dockerfile copies `requirements.txt` and runs `pip install` before copying source code. Explain precisely when this layer cache is invalidated. If a developer adds a new route to `insurance_backend/routers/triangles.py` without changing `requirements.txt`, how many layers does Docker rebuild? If they simultaneously add a new dependency to `requirements.txt`, how does the rebuild sequence change?

Challenge Question

Column pruning impact. The `nyc311_enriched.parquet` file dropped from 4,417 MB to approximately 350 MB after pruning from 26 to 15 columns. Assuming Parquet's columnar storage with Snappy compression and uniform column size, estimate the expected memory reduction factor. Why might the actual reduction (factor of ≈ 12.6) exceed the simple column-count ratio (factor of ≈ 1.73)?

Interview Question

Interviewer: "You said data files are baked into the Docker image. What happens when the data scientists run a new notebook and produce updated parquets? How long does a data refresh take end to end, and what are the failure modes?"

What to cover: GCS upload, workflow_dispatch or path-trigger commit, CI checkout + GCS download + docker build + push + Cloud Run deploy sequence, approximate wall-clock time (typically 4–8 minutes), failure modes at each step (GCS permission denied, docker layer cache miss, Cloud Run health check timeout), and the rollback path (Cloud Run keeps the previous revision and can roll back via gcloud run services update-traffic).

Interview Question

Interviewer: "Why is the backend URL not a `NEXT_PUBLIC_` variable? What is the actual security risk if it were exposed client-side?"

What to cover: NEXT_PUBLIC_ variables are embedded in the static JS bundle at build time and visible to anyone using browser devtools. The Cloud Run URL pattern is guessable but still best kept private. Exposing it allows direct API calls that bypass the proxy in front of it — in production a Cloudflare Pages Function that enforces a service allowlist, converts upstream failures into a 504, and applies edge caching. A strong answer also notes that the build pipeline strips these variables rather than trusting the developer to omit them. For a public portfolio the risk is primarily unwanted traffic and cost rather than data breach, but the principle of least exposure applies.

Interview Question

Interviewer: "Your Cloud Run service has min-instances=0. A recruiter visits your portfolio at 9am and the first page load takes 8 seconds. How would you fix this without setting min-instances=1?"

What to cover: Health-ping warmup component firing `GET /health` on the landing page (wakes the container before the user navigates to a data page), Cloud Run's CPU always-on option (keeps CPU allocated between requests, reduces subsequent cold starts), lighter startup (lazy data loading on first request rather than module-level), and the cost trade-off of min-instances=1 (approximately $7–10/month for a 2Gi instance in us-central1).

Formula Quick Reference

All formulas consolidated from Chapters 2–7, grouped by domain. Each entry includes the equation, variable definitions, and a pointer to the full derivation.

.1 Actuarial & Insurance

All formulas from Chapter 2.

Name	Formula
Age-to-age factor	$\hat{f}_k = \frac{\sum_i C_{i,k+1}}{\sum_i C_{i,k}}$ (volume-weighted, Eq. 2.4)
Cumulative dev. factor	$\widehat{\text{CDF}}_k = \prod_{j=k}^{K-1} \hat{f}_j$ (Eq. 2.5)
Ultimate loss	$\hat{U}_i = C_{i,k} \times \widehat{\text{CDF}}_k$ (Eq. 2.6)
CL IBNR	$\widehat{\text{IBNR}}_i = \max(0,\ \hat{U}_i - C_{i,k})$ (Eq. 2.7)
Unreported fraction	$q_k = 1 - 1/\widehat{\text{CDF}}_k$ (Eq. 2.8)
BF IBNR	$\widehat{\text{IBNR}}_i^{\text{BF}} = \text{Prem}_i \times \text{ELR} \times q_k$ (Eq. 2.10)
BF ultimate	$\hat{U}_i^{\text{BF}} = C_{i,k} + \text{Prem}_i \times \text{ELR} \times q_k$ (Eq. 2.11)
Loss ratio	LR = Incurred Losses/Earned Premium (Eq. 2.12)
Combined ratio	CR = LR + ER (Eq. 2.13)
Pure premium	PP = Frequency × Severity (Eq. 2.15)
Frequency	Claim Count/Exposure (Eq. 2.16)
Severity	Total Losses/Claim Count (Eq. 2.17)

Variables: $C_{i,k}$ = cumulative paid losses for accident year i at dev lag k; K = max lag; ELR = expected loss ratio (a priori); ER = expense ratio.

.2 Statistical Testing

All formulas from Chapter 4.

Name	Formula
Z-test statistic	$Z = (\hat{p}_B - \hat{p}_A)/\text{SE}_{\text{pooled}}$ (Eq. 4.1)
Pooled SE	$\text{SE}_{\text{pool}} = \sqrt{\hat{p}_{\text{pool}}(1-\hat{p}_{\text{pool}})(1/n_A + 1/n_B)}$ (Eq. 4.2)
Pooled proportion	$\hat{p}_{\text{pool}} = (c_A + c_B)/(n_A + n_B)$ (Eq. 4.3)
Unpooled SE	$\text{SE}_{\text{unpool}} = \sqrt{\hat{p}_A(1-\hat{p}_A)/n_A + \hat{p}_B(1-\hat{p}_B)/n_B}$ (Eq. 4.5)
Wilson CI	$\dfrac{\hat{p} + z^2/2n \pm z\sqrt{\hat{p}(1-\hat{p})/n + z^2/4n^2}}{1 + z^2/n}$ (Eq. 4.7)
Chi-squared	$\chi^2 = \sum_{i,j}(O_{ij} - E_{ij})^2/E_{ij}$
Cramér's V	$V = \sqrt{\chi^2/n}$
Cohen's h	$h = 2\arcsin(\sqrt{p_2}) - 2\arcsin(\sqrt{p_1})$ (Eq. 4.10)
Sample size	$n = \dfrac{(z_{\alpha/2}\sqrt{2p_1(1-p_1)} + z_\beta\sqrt{p_1(1-p_1)+p_2(1-p_2)})^2}{(p_2-p_1)^2}$ (Eq. 4.18)
Power curve	$\text{Power}(\delta) = \Phi(\delta/\text{SE} - z_{\alpha/2}) + \Phi(-\delta/\text{SE} - z_{\alpha/2})$ (Eq. 4.19)
OBF boundary	$z_k^* = z_{\alpha/2}/\sqrt{k/K}$ (Eq. 4.22)
Peeking inflation	$\alpha^* \approx 1 - (1-\alpha)^K$
Bonferroni	$\alpha_{\text{adj}} = \alpha/m$ (Eq. 7.5)

Variables: c = conversions; n = group size; δ = MDE; K = total looks; m = number of comparisons.

.3 Bayesian Inference

From Chapter 4.

Name	Formula
Beta posterior	$\theta \mid c, f \sim \text{Beta}(\alpha_0 + c,\ \beta_0 + f)$ (Eq. 4.12)
$P(B > A)$	$\int_0^1 \int_{\theta_A}^1 f(\theta_A) f(\theta_B)\, d\theta_B\, d\theta_A$ (Monte Carlo, Eq. 4.14)
Expected loss	$\mathcal{L}_A = \mathbb{E}[\max(\theta_B - \theta_A, 0)]$ (Eq. 4.15)
Credible interval	$[F^{-1}_{\text{Beta}}(0.025),\ F^{-1}_{\text{Beta}}(0.975)]$ (Eq. 4.17)

Variables: c = successes; $f = n - c$ = failures; α_0, β_0 = prior params (= 1 for uniform).

.4 Survival & Customer Analytics

From Chapter 3.

Name	Formula
Kaplan-Meier	$\hat{S}(t) = \prod_{t_i \le t} \left(1 - \frac{d_i}{n_i}\right)$ (Eq. 3.2)
Retention rate	$\text{Ret}_{c,k} = \lvert\{u \in C_c : \text{active in month } k\}\rvert / \lvert C_c \rvert$ (Eq. 3.1)
Logistic regression	$\log(p/(1-p)) = \beta_0 + \beta_1 x_1 + \cdots + \beta_k x_k$ (Eq. 3.3)
Odds ratio	$\widehat{\text{OR}}_i = \exp(\hat{\beta}_i)$ (Eq. 3.4)
Gini coefficient	$G = 1 - 2\int_0^1 L(p)\,dp$ (Eq. 3.5)

Variables: d_i = events at time t_i; n_i = at-risk at t_i; C_c = cohort c; $L(p)$ = Lorenz curve.

.5 SaaS Metrics

From Chapter 5.

Name	Formula
MRR waterfall	$\text{MRR}_t = \text{MRR}_{t-1} + \text{New} + \text{Exp} - \text{Contr} - \text{Churn}$
NRR	$\frac{\text{MRR}_{\text{start}} + \text{Exp} - \text{Contr} - \text{Churn}}{\text{MRR}_{\text{start}}}$
Logo churn	$\text{Churned Customers}_t / \text{Customers at Start}_t$
Revenue churn	$\text{Churned MRR}_t / \text{MRR}_{t-1}$
LTV	$(\text{ARPU} \times 12/\text{Churn Rate}) \times \text{Gross Margin}$
Rule of 40	$\text{Rev Growth Rate (\%)} + \text{Gross Margin (\%)} \ge 40$
Health score	$H = \sum_{k=1}^{6} w_k \cdot s_k, \quad s_k = \text{clamp}_{[0,100]}\left(\frac{\text{metric}-\text{bad}}{\text{good}-\text{bad}} \times 100\right)$

.6 Forecasting & Anomaly Detection

From Chapter 5.

Name	Formula
Holt-Winters level	$l_t = \alpha \cdot y_t + (1-\alpha)(l_{t-1} + b_{t-1})$
Holt-Winters trend	$b_t = \beta(l_t - l_{t-1}) + (1-\beta)b_{t-1}$
Forecast	$\hat{y}_{t+h} = l_t + h \cdot b_t$
Forecast CI	$\hat{y}_{t+i} \pm \text{RSE} \times 1.96 \times \sqrt{1 + 0.1i}$
Z-score	$z_i = (x_i - \bar{x})/\sigma$
IQR fences	$Q_1 - 1.5 \cdot \text{IQR}, \quad Q_3 + 1.5 \cdot \text{IQR}, \quad \text{IQR} = Q_3 - Q_1$

.7 Financial & Portfolio

All formulas from Chapter 6.

Name	Formula
Simple return	$R_t = (P_t - P_{t-1})/P_{t-1}$ (Eq. 6.1)
Portfolio return	$R_{p,t} = \sum_i w_i R_{i,t}$ (Eq. 6.2)
Cumulative return	$W_T = \prod_{t=1}^{T}(1 + R_t) - 1$ (Eq. 6.3)
CAGR	$(\prod(1 + R_t))^{252/T} - 1$ (Eq. 6.4)
Ann. volatility	$\sigma_{\text{ann}} = \hat{\sigma}_{\text{daily}} \times \sqrt{252}$ (Eq. 6.7)
Sharpe ratio	$(\bar{R}_p - r_f)/\sigma_p$ (Eq. 6.8)
Sortino ratio	$(\bar{R}_p - r_f)/\sigma_{\text{down}}$ (Eq. 6.10)
Calmar ratio	$\text{CAGR}/\lvert\text{MDD}\rvert$ (Eq. 6.11)
Max drawdown	$\min_t(W_t/\max_{s\le t} W_s - 1)$ (Eq. 6.12)
Beta	$\text{Cov}(R_p, R_m)/\text{Var}(R_m)$ (Eq. 6.13)
Jensen's alpha	$\bar{R}_p - [r_f + \beta(\bar{R}_m - r_f)]$ (Eq. 6.14)
Tracking error	$\text{std}(R_p - R_m) \times \sqrt{252}$ (Eq. 6.15)
Information ratio	$(\bar{R}_p - \bar{R}_m)/\text{TE}$ (Eq. 6.16)
Parametric VaR	$-(\hat{\mu} + z_{1-\alpha} \cdot \hat{\sigma})$ (Eq. 6.18)
Historical VaR	$-\text{Percentile}_{(1-\alpha)\times 100}$ (Eq. 6.19)
CVaR	$\mathbb{E}[-R \mid R \le -\text{VaR}_\alpha]$ (Eq. 6.20)
GBM SDE	$dS = \mu S\,dt + \sigma S\,dW$ (Eq. 6.21)
GBM discrete	$S_{t+1} = S_t \exp[(\mu - \sigma^2/2) + \sigma Z]$ (Eq. 6.25)
Portfolio variance	$\sigma_p^2 = \boldsymbol{w}^\top \boldsymbol{\Sigma}\, \boldsymbol{w}$ (Eq. 6.27)
Pearson corr.	$\rho_{ij} = \text{Cov}(R_i, R_j)/(\sigma_i \sigma_j)$ (Eq. 6.34)

Key constants: $r_f = 4.5\%$; trading days = 252; Monte Carlo seed = 42.

.8 Operational

From Chapter 7.

Name	**Formula**
SLA compliance	$C = \#\text{Cumple}/(\#\text{Cumple} + \#\text{Incumple}) \times 100$ (Eq. 7.2)
SLA verdict	CUMPLE $\geq 85\%$; EN RIESGO $\geq 70\%$; INCUMPLE $< 70\%$ (Eq. 7.3)
Two-prop z-test	Same as A/B test z-test (Eq. 7.4)
Bonferroni	$\alpha_{\text{adj}} = \alpha/m, \quad m = \binom{k}{2}$ (Eq. 7.5)

Glossary

Key terms organized by domain, with chapter references. Terms marked with an asterisk (*) appear in multiple chapters.

Actuarial & Insurance

Accident Year

The year in which an insured loss event occurred, regardless of when it was reported or paid. Used as the row dimension in loss triangles. *(Ch. 1)*

Age-to-Age Factor (ATA)

The ratio of cumulative losses at successive development lags, used to project future development. Also called a link ratio. *(Ch. 1)*

Bornhuetter-Ferguson (BF) Method

A reserve estimation method that blends observed losses with an a priori expected loss ratio, more stable than Chain-Ladder for immature accident years. *(Ch. 1)*

CAS Loss Reserve Database

Public dataset maintained by the Casualty Actuarial Society containing Schedule P loss triangles across six lines of business. *(Ch. 1)*

Chain-Ladder Method

The standard actuarial reserving technique that uses historical development patterns (ATA factors) to project ultimate losses. *(Ch. 1)*

Combined Ratio

Loss ratio plus expense ratio. Below 100% indicates underwriting profit. The gold-standard P&C profitability metric. *(Ch. 1)*

Cumulative Development Factor (CDF)

The product of all remaining ATA factors from a given lag to ultimate; multiplied by current paid losses to project ultimate losses. *(Ch. 1)*

Development Lag

The number of years (or periods) between the accident year and the evaluation date. Column dimension in loss triangles. *(Ch. 1)*

Expected Loss Ratio (ELR)

An a priori assumption about the ratio of losses to premiums, used in the BF

method. Typically derived from pricing or industry benchmarks. *(Ch. 1)*

Frequency

Number of claims per unit of exposure. One of the two components of pure premium. *(Ch. 1)*

IBNR

Incurred But Not Reported – the estimated liability for claims that have occurred but not yet been reported to the insurer. *(Ch. 1)*

Line of Business (LOB)

A category of insurance coverage (e.g., Commercial Auto, Workers' Compensation, Medical Malpractice). *(Ch. 1)*

Loss Ratio

Incurred losses divided by earned premiums. The primary measure of underwriting profitability. *(Ch. 1)*

Loss Triangle

A matrix of cumulative losses indexed by accident year (rows) and development lag (columns), with the lower-right portion unknown. *(Ch. 1)*

Pure Premium

Frequency times severity. The expected loss cost per unit of exposure before loading for expenses. *(Ch. 1)*

Severity

Average cost per claim. The second component of pure premium. *(Ch. 1)*

Tail Factor

A multiplicative factor applied beyond the last observed development lag to account for residual development to ultimate. *(Ch. 1)*

Statistical Testing

Alpha (α)*

The significance level; the maximum acceptable probability of a Type I error (false positive). Commonly 0.05. *(Ch. 3, 6)*

Bonferroni Correction

Divides the per-test significance level by the number of comparisons to control the family-wise error rate. Conservative. *(Ch. 3, 6)*

Cohen's h

An effect size measure for two proportions using the arcsine variance-stabilizing transform. Small < 0.2, medium 0.2–0.5, large > 0.5. *(Ch. 3)*

Confidence Interval (CI)

A range of values constructed so that, across repeated sampling, a specified fraction (e.g., 95%) of such intervals contain the true parameter. *(Ch. 3)*

Cramér's V

A normalized effect size for chi-squared tests, bounded in $[0, 1]$. *(Ch. 3)*

Credible Interval

Bayesian analog of a CI. A 95% credible interval means there is a 95% posterior probability the parameter lies within the interval. *(Ch. 3)*

Expected Loss (Bayesian)

The expected cost of making the wrong decision; $\mathcal{L}_B = \mathbb{E}[\max(\theta_A - \theta_B, 0)]$. Used for Bayesian decision-making. *(Ch. 3)*

MDE

Minimum Detectable Effect – the smallest treatment effect a test can reliably detect at the specified power and significance level. *(Ch. 3)*

O'Brien-Fleming (OBF) Boundaries

A group sequential testing spending function that uses very conservative early boundaries and nearly standard final boundaries. *(Ch. 3)*

Peeking Problem

The inflation of Type I error rate caused by checking p-values at multiple time points during an experiment without correction. *(Ch. 3)*

Power ($1 - \beta$)

The probability of correctly rejecting the null when a true effect exists. Typically set at 80% or 90%. *(Ch. 3)*

P(B > A)

Bayesian posterior probability that the treatment conversion rate exceeds control. Estimated via Monte Carlo. *(Ch. 3)*

Sample Ratio Mismatch (SRM)

A diagnostic test checking whether the observed group split matches the planned allocation. Failure signals randomization bugs. *(Ch. 3)*

Sequential Testing

A framework that allows interim analyses during an experiment while controlling the overall Type I error rate. *(Ch. 3)*

Simpson's Paradox

A phenomenon where an aggregate trend reverses when data is disaggregated by a confounding variable. *(Ch. 3)*

Type I Error

Rejecting the null hypothesis when it is true (false positive). *(Ch. 3)*

Type II Error

Failing to reject the null when it is false (false negative). *(Ch. 3)*

Wilson Score Interval

A confidence interval for proportions that outperforms the Wald interval near 0 and 1, always bounded within $[0, 1]$. *(Ch. 2, 3)*

Z-test (Two-Proportion)*

A hypothesis test comparing two independent proportions using the normal approximation to the binomial. *(Ch. 3, 6)*

Financial & Portfolio

Alpha (Jensen's)

The risk-adjusted excess return above the CAPM prediction. Positive alpha indicates outperformance relative to market beta exposure. *(Ch. 5)*

Beta

The sensitivity of portfolio returns to benchmark returns; the OLS slope of portfolio on benchmark. $\beta < 1$ means less volatile than the market. *(Ch. 5)*

CAGR

Compound Annual Growth Rate. Converts total holding-period return into an equivalent constant annual rate. Annualized using 252 trading days. *(Ch. 5)*

Calmar Ratio

CAGR divided by the absolute value of max drawdown. Popular in hedge fund evaluation. *(Ch. 5)*

CAPM

Capital Asset Pricing Model. Predicts expected return as $r_f + \beta(R_m - r_f)$. Single-factor model with strong simplifying assumptions. *(Ch. 5)*

CVaR / Expected Shortfall

The expected loss given that losses exceed VaR. A coherent risk measure satisfying subadditivity, unlike VaR. *(Ch. 5)*

Efficient Frontier

The set of portfolios achieving maximum return for each level of risk (or minimum risk for each return level) under given constraints. *(Ch. 5)*

GBM

Geometric Brownian Motion. An SDE $dS = \mu S\,dt + \sigma S\,dW$ used to model asset price evolution. Ensures positive prices. *(Ch. 5)*

Information Ratio

Active return (portfolio minus benchmark) divided by tracking error. Measures reward per unit of active risk. *(Ch. 5)*

Ito's Lemma

A result in stochastic calculus giving the differential of a function of a stochastic process. Produces the $\mu - \sigma^2/2$ drift correction in GBM. *(Ch. 5)*

Maximum Drawdown (MDD)

The largest peak-to-trough decline in a portfolio's wealth index. A tail risk measure more intuitive than volatility. *(Ch. 5)*

Mean-Variance Optimization

Markowitz's framework for finding portfolios that maximize μ_p for a given σ_p (or minimize σ_p for a given μ_p). *(Ch. 5)*

Sharpe Ratio

Excess return per unit of total volatility: $(R_p - r_f)/\sigma_p$. The most widely used risk-adjusted return metric. *(Ch. 5)*

SLSQP

Sequential Least Squares Programming. A constrained nonlinear optimization solver used for portfolio optimization. *(Ch. 5)*

Sortino Ratio

Like Sharpe but uses only downside deviation. More appropriate when the return distribution is asymmetric. *(Ch. 5)*

Subadditivity

A property of coherent risk measures: $\rho(X + Y) \leq \rho(X) + \rho(Y)$. Diversification should not increase measured risk. VaR violates this; CVaR satisfies it. *(Ch. 5)*

Tracking Error

Annualized standard deviation of the return difference between portfolio and benchmark. Measures active risk. *(Ch. 5)*

VaR

Value at Risk. The loss threshold not exceeded with probability α. Three methods: parametric (Gaussian), historical (percentile), Monte Carlo. *(Ch. 5)*

SaaS & Business Metrics

ARR

Annual Recurring Revenue. $12 \times$ MRR. The standard SaaS top-line metric for annual planning. *(Ch. 4)*

CAC

Customer Acquisition Cost. Total sales and marketing spend divided by new customers acquired. *(Ch. 4)*

Gross Margin

Revenue minus cost of goods sold, divided by revenue. NovaCRM uses 80%. *(Ch. 4)*

Health Score

A 0–100 composite index combining six weighted SaaS metrics with linear scaling and traffic-light thresholds. *(Ch. 4)*

Logo Churn

The fraction of customers who cancelled in a period. Segment baselines: Starter 5.5%, Professional 3%, Enterprise 1.2% per month. *(Ch. 4)*

LTV

Customer Lifetime Value. Estimated as (ARPU $\times$ 12/Churn Rate) $\times$ Gross Margin. *(Ch. 4)*

LTV:CAC Ratio

The ratio of lifetime value to acquisition cost. $\geq 3\times$ is healthy; $< 1\times$ means losing money per customer. *(Ch. 4)*

MRR

Monthly Recurring Revenue. Sum of all monthly subscription fees. Excludes one-time charges. *(Ch. 4)*

MRR Waterfall

Decomposition of MRR change into New, Expansion, Contraction, and Churned components. Reveals which lever drives net movement. *(Ch. 4)*

NPS

Net Promoter Score. Promoters (9–10) minus Detractors (0–6) as a percentage. Range: -100 to $+100$. *(Ch. 4)*

NRR

Net Revenue Retention. Measures revenue retained and grown from the existing customer base. $> 100\%$ means the base grows without new logos. *(Ch. 4)*

Revenue Churn

Fraction of MRR lost to cancellations. Revenue churn exceeding logo churn signals loss of high-value accounts. *(Ch. 4)*

Rule of 40

Revenue growth rate (%) plus profit margin (%) should be ≥ 40. A heuristic for balancing growth and profitability. *(Ch. 4)*

Customer Analytics

Cohort

A group of customers defined by their first purchase month. Used to track retention over time. *(Ch. 2)*

Gini Coefficient

A measure of revenue concentration from the Lorenz curve. 0 = perfect equality, 1 = all revenue from one customer. *(Ch. 2)*

Kaplan-Meier Estimator

A non-parametric survival function estimator that handles right-censored data. Estimates $S(t) = P(\text{no repeat purchase by day } t)$. *(Ch. 2)*

Lorenz Curve

Plots cumulative revenue share vs. cumulative customer share (sorted low to high). The area between it and the diagonal is half the Gini. *(Ch. 2)*

Odds Ratio
: $\exp(\hat{\beta})$ from logistic regression. OR > 1 means the feature is associated with higher odds of the outcome. *(Ch. 2)*

Retention Matrix
: A cohort $\times$ period matrix showing the fraction of each cohort still active in each subsequent period. *(Ch. 2)*

RFM
: Recency, Frequency, Monetary – a customer segmentation approach using NTILE scoring on three dimensions. *(Ch. 2)*

Right-Censoring
: When the observation window ends before the event of interest occurs. KM handles this; naive counting does not. *(Ch. 2)*

Operations & Process

Bottleneck
: An agency–complaint type combination with unusually long resolution times. Identified by ranking mean/median resolution days. *(Ch. 6)*

Pareto Analysis
: Applying the 80/20 rule to identify which complaint types account for the majority of volume or delays. *(Ch. 6)*

Process Mining
: Analyzing event logs to discover actual process flows. Visualized as a Sankey diagram in this project. *(Ch. 6)*

Sankey Diagram
: A flow diagram where link widths are proportional to flow quantities. Used to trace complaint $\rightarrow$ agency $\rightarrow$ stage $\rightarrow$ response. *(Ch. 6)*

SLA (Service Level Agreement)
: A target deadline for resolving a service request. Compliance is Cumple/Incumple/Sin SLA. *(Ch. 6)*

SODA API
: Socrata Open Data API. Used to paginate and download NYC 311 data at 50K rows per page. *(Ch. 6)*

Technical & Architecture

ASGI
: Asynchronous Server Gateway Interface. The Python standard for async web servers; FastAPI runs on ASGI via Uvicorn. *(Ch. 7)*

Artifact Registry
: Google Cloud container image registry. Stores Docker images for Cloud Run deployment. *(Ch. 7)*

app.mount()
: FastAPI method to mount a sub-application at a path prefix. Used to host 5 project backends on a single service. *(Ch. 7)*

Cloud Run
: Google Cloud serverless container platform. Scales to zero; bills per request. Hosts the consolidated API, and a redirect-only container standing in for the retired Streamlit service. *(Ch. 7)*

Cold Start
: The latency incurred when Cloud Run spins up a new container instance. Mitigated by `min-instances=1` for production services. *(Ch. 7)*

GCS
: Google Cloud Storage. Parquet data files live in `gs://da-portfolio-data-assets` and are downloaded at build time. *(Ch. 7)*

Path Filters
: GitHub Actions trigger conditions that match changed file paths, enabling selective deployment (only rebuild what changed). *(Ch. 7)*

Rewrites (Next.js)
: A `next.config.js` pattern that proxies API requests through the Next.js server, keeping backend URLs private from the browser. *(Ch. 7)*

WIF
: Workload Identity Federation. A keyless authentication method for GitHub Actions to GCP, eliminating service account key management. *(Ch. 7)*